



AUTOMATED ACCEPTANCE TESTING FOR INDUSTRIAL ROBOTIC SYSTEMS

PAR MARCELA GONÇALVES DOS SANTOS

**THESIS PRESENTED TO THE UNIVERSITY OF QUEBEC AT CHICOUTIMI
WITH A VIEW TO OBTAINING THE DEGREE OF PHILOSOPHIÆ DOCTOR
(PH.D.) IN COMPUTER SCIENCE**

QUÉBEC, CANADA

© MARCELA GONÇALVES DOS SANTOS, 2025

ABSTRACT

Context: Between 2015 and 2020, the annual installations of industrial robotic systems increased by 9% each year. This rise in industrial robots has prompted researchers and practitioners to implement software engineering approaches within robotic domains to enhance quality and address failures. However, challenges and gaps related to software testing for robotic systems still exist. **Goal:** We proposed and evaluated a software testing approach called Automated Acceptance testing for Industrial Robotic Systems (AAT4IRS), which aims improve the effectiveness of testing by enhancing fault detection and test robustness. Additionally, we propose an evaluation guideline for applying mutation testing to industrial robotic systems that incorporate unpredictable actions, thereby increasing trust in simulation testing. **Method:** First, we conducted a systematic analysis of software engineering and software quality practices applied in robotic systems. Since our research focuses on the intersection of software testing and industrial robotic systems (IRS), this preliminary analysis provided valuable insights into how roboticists integrate software engineering techniques in their work. Secondly, we defined and validated an automated acceptance testing approach that enhances the effectiveness of testing by improving fault detection and test robustness. This proposed approach utilizes Behavior Driven Development (BDD) templates for acceptance testing. Lastly, we developed and evaluated a guideline for applying mutation testing to IRS, which includes the unpredictable actions typically observed in these systems. This incorporation of domain-specific details aims to improve trust in simulation testing. **Results:** The systematic mapping shows me that there is a consensus by the research community that software engineering techniques and approaches can improve the quality of robot software. Moreover, the majority of the studies focus on three areas that performed activities inherent to robotics software development (software design, construction and models and methods) . However, only 18% of the studies performed activities related to testing and quality. To evaluate the effectiveness of our software testing approach, we tested the generated test suites against mutants created from the original code. The test suites produced through the use of Automated Acceptance Testing for Industrial Robotic Systems (AAT4IRS) achieved an effectiveness score of 90%. In addition, the results observed in our experiment suggest that AAT4IRS has some benefits such as: (i) reduction in the effort to discover faults in IRS; (ii) improvement in traceability; (iii) better coverage of regression test; and (iv) improvement of coordination, collaboration, and documentation in robotic systems development teams. All these benefits needs to be evaluated in future work, but as they are known benefits by applying software testing in conventional system, we strongly believe that they also can also benefit robotic systems.

RÉSUMÉ

Contexte: Entre 2015 et 2020, les installations annuelles de systèmes robotiques industriels ont augmenté de 9 % chaque année. Cette hausse des robots industriels a incité les chercheurs et les praticiens à adopter des approches d'ingénierie logicielle dans le domaine de la robotique afin d'améliorer la qualité et de traiter les défaillances. Cependant, des défis et des lacunes persistent en matière de tests logiciels pour les systèmes robotiques. **Objectif:** Nous avons proposé et évalué une approche de test logiciel appelée Automated Acceptance Testing for Industrial Robotic Systems (AAT4IRS), qui vise à améliorer l'efficacité des tests en renforçant la détection des défauts et la robustesse des tests. De plus, nous avons présenté un guide d'évaluation pour l'application des tests de mutation aux systèmes robotiques industriels, intégrant des actions imprévisibles afin d'accroître la confiance dans les tests de simulation. **Méthode:** Tout d'abord, nous avons réalisé une analyse systématique des pratiques d'ingénierie logicielle et de qualité logicielle appliquées aux systèmes robotiques. Étant donné que notre recherche se situe à l'intersection des tests logiciels et des systèmes robotiques industriels (Industrial Robotic Systems - IRS), cette analyse préliminaire a fourni des informations précieuses sur la manière dont les roboticistes intègrent les techniques d'ingénierie logicielle dans leur travail. Ensuite, nous avons défini et validé une approche de test d'acceptation automatisé visant à améliorer l'efficacité des tests en renforçant la détection des défauts et la robustesse des tests. Cette approche repose sur des modèles de Behavior Driven Development (BDD) pour les tests d'acceptation. Enfin, nous avons développé et évalué un guide pour l'application des tests de mutation aux systèmes robotiques industriels, en intégrant les actions imprévisibles souvent observées dans ces systèmes. L'inclusion de ces détails spécifiques au domaine vise à renforcer la confiance dans les tests de simulation. **Résultats:** La cartographie systématique montre qu'il existe un consensus dans la communauté scientifique sur le fait que les techniques et approches d'ingénierie logicielle peuvent améliorer la qualité des logiciels robotiques. Par ailleurs, la majorité des études se concentrent sur trois domaines liés au développement des logiciels robotiques : la conception logicielle, la construction logicielle et les modèles et méthodes. Cependant, seules 18 % des études incluent des activités liées aux tests et à la qualité logicielle. Pour évaluer l'efficacité de notre approche de test logiciel, nous avons testé les suites de tests générées sur des mutants créés à partir du code original. Les suites de tests produites grâce à Automated Acceptance Testing for Industrial Robotic Systems (AAT4IRS) ont atteint un score d'efficacité de 90 %. De plus, les résultats de notre expérimentation suggèrent que AAT4IRS présente plusieurs avantages, notamment: (i) une réduction de l'effort nécessaire pour détecter les défauts dans les systèmes robotiques industriels; (ii) une amélioration de la traçabilité; (iii) une meilleure couverture des tests de régression; (iv) une amélioration de la coordination, de la collaboration et de la documentation au sein des équipes de développement des systèmes robotiques. Tous ces avantages devront être évalués plus en profondeur dans les travaux futurs, mais comme ces bénéfices sont déjà bien connus dans l'application des tests logiciels aux systèmes conventionnels, nous pensons fermement qu'ils peuvent également s'appliquer aux systèmes robotiques.

TABLE OF CONTENTS

ABSTRACT	ii
RÉSUMÉ	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
ACKNOWLEDGEMENTS	xi
CHAPTER I – INTRODUCTION	1
1.1 RESEARCH GOALS	7
1.2 SUMMARY OF CONTRIBUTIONS	9
1.3 THESIS OUTLINE	9
CHAPTER II – BACKGROUND AND RELATED WORK	10
2.1 FOUNDATIONS OF INDUSTRIAL ROBOTIC SYSTEMS	10
2.1.1 HARDWARE COMPONENT	12
2.1.2 SOFTWARE COMPONENT	18
2.2 FOUNDATIONS OF SOFTWARE TESTING	29
2.2.1 BASIC CONCEPTS	30
2.2.2 SOFTWARE QUALITY: ROBUSTNESS AND RELIABILITY	34
2.2.3 TEST AUTOMATION	36
2.2.4 BEHAVIOR-DRIVEN DEVELOPMENT	39
2.2.5 MUTATION TESTING	46
CHAPTER III – PRELIMINARY WORK	49
3.1 SOFTWARE ENGINEERING FOR ROBOTIC SYSTEMS: A SYSTEMATIC MAPPING STUDY	49
3.1.1 RESEARCH QUESTIONS	50
3.1.2 SEARCH STRATEGY	51

3.1.3	DATA EXTRACTION	53
3.1.4	DATA SYNTHESIS	54
3.1.5	SM RESULTS	60
3.1.6	RELATED WORK	66
3.1.7	DISCUSSIONS	67
3.2	SOFTWARE TESTING FOR ROBOTIC SYSTEMS	70
3.2.1	PRIMARY STUDIES	71
3.2.2	SECONDARY STUDIES	73
3.3	DISCUSSIONS AND FINAL REMARKS	75
CHAPTER IV – AAT4IRS SOFTWARE TESTING APPROACH		77
4.1	WORKFLOW OF THE AAT4IRS	77
4.1.1	STAGE 1 - WRITE FEATURES	80
4.1.2	STAGE 2 - WRITE SCENARIOS	82
4.1.3	STAGE 3 - IMPLEMENT TESTS	85
4.1.4	STAGE 4 - EXECUTE TESTS	86
4.2	CHAPTER FINAL REMARKS	88
CHAPTER V – APPLYING AAT4IRS TO PICK-AND-PLACE TASK		90
5.1	TOOLS, TECHNOLOGIES, AND LIBRARIES	91
5.2	CASE STUDY CONDUCTION	92
5.2.1	STAGE 1 - WRITE FEATURES	94
5.2.2	STAGE 3 - WRITE SCENARIOS	95
5.2.3	STAGE 4 - IMPLEMENT TESTS	96
5.3	EVALUATION	98
5.4	DISCUSSIONS	108
5.5	THREATS TO VALIDITY	110
5.6	CHAPTER FINAL REMARKS	111
CONCLUSION		113

REFERENCES	117
-------------------	-----

LIST OF TABLES

TABLE 2.1 :	Comparison of Online, Offline, and Hybrid Robot Programming Methods (Pan <i>et al.</i>, 2010 ; Heimann & Guhl, 2020 ; Zheng <i>et al.</i>, 2022)	23
TABLE 2.2 :	Key difference between Conventional System and IRS software	27
TABLE 2.3 :	Benefits of test automation and their impact on IRS. Based on (Naik & Tripathy, 2018 ; Graham <i>et al.</i>, 2008 ; ISTQ, 2021)	38
TABLE 3.1 :	List of primary studies	54
TABLE 3.2 :	List of Venues	62
TABLE 3.3 :	LIST OF INCLUDED STUDIES FOR EACH SWEBOK KNOWLEDGE AREAS.	63
TABLE 5.1 :	Original X Mutant Command for Robotic Systems	103
TABLE 5.2 :	List of mutants	103
TABLE 5.3 :	Mutant Score for each round	106

LIST OF FIGURES

FIGURE 2.1 – INDUSTRIAL ROBOTIC SYSTEM	11
FIGURE 2.2 – IR COMPONENTS	13
FIGURE 3.1 – OVERVIEW OF THE SEARCH STRATEGY AND STUDY SELECTION PROCESSES	52
FIGURE 3.2 – NUMBER OF STUDIES OF SE TECHNIQUE APPLIED IN ROBOTICS DOMAIN PER YEAR.	61
FIGURE 3.3 – NUMBER OF STUDIES AND SE TECHNIQUE APPLIED IN ROBOTICS DOMAIN.	65
FIGURE 4.1 – PROPOSED APPROACH TO APPLY AUTOMATED ACCEPTANCE TESTING FOR IRS.	80
FIGURE 5.1 – PICK-AND-PLACE TASK PERFORMED USING GAZEBO SIMULATOR.	93
FIGURE 5.2 – LOCALIZATION AND OPERATION THAT WE APPLY THE MUTATION.	103
FIGURE 5.3 – NOISE ADDED TO THE SENSOR READING.	106
FIGURE 5.4 – REPORT OVERVIEW.	107
FIGURE 5.5 – DETAILS FOR ONE FAILED TEST	108

LIST OF ABBREVIATIONS

CAGR compound annual growth rate

IFR International Federation of Robots

AAT4IRS Automated Acceptance Testing for Industrial Robotic Systems

AAT Automated Acceptance Testing

BDD Behavior-Driven Development

BR Business Requirement

IR Industrial Robots

ST Software Testing

AT Acceptance Testing

AC Acceptance Criteria

SM Systematic Mapping

SLR Systematic Literature Review

SUT System Under Test

IRS Industrial Robotic Systems

IRPL Industrial Robot Programming Language

AC Acceptance Criteria

IC Inclusion Criteria

ISTQB International Software Testing Qualifications Board

EC Exclusion Criteria

RQ Research Question

RS Robotic Systems

ROS Robot Operating System

RG Research Goal

RGs Research Goals

MT Mutation Testing

V&V Verification and Validation

RSE Robotics Software Engineer

SERS Software Engineering for Robotic Systems

SE Software Engineering

SQ Software Quality

ST Software Testing

SWEBOK Software Engineering Body of Knowledge

KA Knowledge Areas

ACKNOWLEDGEMENTS

There is a song that perfectly captures how I feel at this moment, and I would love to use it:

*"I wanna thank me for believing in me.
I wanna thank me for doing all this hard work.
I wanna thank me for having no days off.
I wanna thank me for never quitting."*

I am profoundly grateful to my academic advisor, Sylvain Hallé, and my co-advisor, Fabio Petrillo. Your unwavering support, insightful guidance, and patience have been instrumental in this research. I truly appreciate the knowledge you have shared and the impact you have had in my academic path.

To my family, both those I am connected to by blood and those life has gifted me, your love and support mean the world to me.

To my friends, thank you for your companionship, for the laughter, and for your unwavering support. From dancing and skiing together to help me navigate the long winters, to celebrating life under the summer sun, to deep conversations about the meaning of it all. You have made my PhD journey in the **real North** not only smoother but also richer, warmer, and full of unforgettable moments.

To my "brasilidade" and to The Art of Dance, you both play crucial roles in my PhD journey. Understanding my ancestry as a Brazilian living outside my country has been fundamental in maintaining my resolve to achieve my goal of becoming a doctor, and this was made possible thanks to dance. Not only through the dance classes I attended (thanks "Rabaterapia", my second home), but also in all the "Rodas de Samba." You are integral to my toolkit for maintaining mental wellness.

A special thank you to my therapist, whose guidance and support have been invaluable. Mental health is a crucial yet often overlooked aspect of academic life, especially for international students navigating cultural shifts, isolation, and the pressures of research. Seeking help is a sign of strength, and I am grateful for the space to reflect, heal, and grow throughout this journey.

Lastly, I want to acknowledge the spiritual practice whose name I may not know, but whose presence I have deeply felt. In moments when I was truly on my own, this connection, whether through mindfulness, meditation, or something beyond words, offered me comfort, clarity, and resilience. It reminded me that even in solitude, I was never truly alone.

CHAPTER I

INTRODUCTION

Robots are complex systems that integrate hardware and software components to perform specific tasks in different domains. Currently, robots are widely used across industries, in healthcare, and in critical missions, interacting with the physical world. They can be classified based on their operational environment (mobile or fixed) and their interaction mechanism (industrial or service), as well as their application field ([Ben-Ari & Mondada, 2018](#)).

IRS are systems composed of industrial robots, end-effectors (grippers, welding torches, painting heads, cutting tools, etc.) and equipment (sensors, vision systems, conveyors, etc.). Furthermore, IR are robots that carry out repetitive tasks—such as picking and placing, soldering, and cutting—with high precision, safety (substituting or aiding humans in hazardous tasks), and efficiency across various industries, including automotive, electrical/electronics, metal, and machinery sectors ([ISO/IEC 8373, 2012](#)).

The International Federation of Robots (IFR) is a non-profit organization that fosters connections within the global robotics community and aims to raise awareness of the beneficial impact of robotics on society and improve accessibility and interaction with robotics. Each year, IFR releases a report containing metrics, statistics, market analysis, forecasts, and case studies on robots. One of the metrics calculated by the IFR is the annual installation of industrial robots, which refers to the number of new industrial robots installed in factories and other industrial settings each year. The latest published data, for 2022, shows that robot installations reached a new record high of 553,052 units. This marks the second consecutive year that annual installations have exceeded 500,000 units, showing a 5% increase from the

previous year and compound annual growth rate (CAGR) of 7% from 2017 to 2022 ([IFR, 2024](#)).

Another important metric to consider in the industrial robotics market is the operational stock of industrial robots. In 2022, this number was 3,903,633 units, representing a 12% increase. Over the past five years, the operational stock of industrial robots has been growing at an average annual rate of 13% ([IFR, 2024](#)).

As these trends show, industrial robot installations increased significantly in the past few years. The growth forecast for global robot installations in 2023 is more than 590,000 (CAGR 7%) despite a slowdown in global economic growth. Some reasons for this growth include advancements in robotics technology (cloud computing, 5G mobile networks, machine vision, and artificial intelligence), the need for higher efficiency and productivity in manufacturing, and increasing labour costs. Indeed, there is a labour shortage, and by 2030, more than 85 million jobs could go unfilled globally because there are not enough skilled people to fill them. This has led to accelerated automation and increased use of robotics ([Ferry, 2018](#)).

The increase in the number of industrial robotic systems operating in widest range of environments also increases the necessity for these systems to handle failures and meet quality aspects.

In fact, robot failures can lead to catastrophic outcomes. For instance, a die-cast operator with 15 years of experience was tragically pinned between the rear end of a hydraulic robot and a safety pole ([Sanderson et al., 1986](#)). In another case, a robot operator setting up a stationary robot was crushed against a metal plate when the robot unexpectedly grabbed him ([Travis, 2015](#)). Additionally, a malfunctioning robot, designed to lift boxes of peppers, mistakenly identified a man as a box and crushed him against a conveyor belt ([Reynolds, 2023](#)). A study

indicated that from 1992 to 2017, at least 41 individuals lost their lives due to industrial robots (Layne, 2023).

Furthermore, according to research on the reliability of robotic automation published in the International Journal of Performability Engineering (Tsarouhas & Furlas, 2015), IRS are non-operational 12% of the time. This downtime is partly due to program and sensor faults, as well as other miscellaneous issues. The cost of IRS downtime depends on a variety of factors, but it is estimated to be tens of thousands of dollars in lost production.

Handling failures in IRS is, therefore, crucial for reducing accidents and minimizing the costs associated with lost production.

PROBLEM STATEMENT

For conventional systems, Software Engineering (SE) plays an important role in handling failures and improving software quality. The main goal of SE is to produce quality software. For IRS, an empirical study by García *et al.* (2020) assessed the state of the art and practice of software engineering in robotics. The respondents highlighted the difficulty and importance of programming robots to handle failures. 83% of respondents agreed or strongly agreed that robustness is a significant challenge in robotic systems. Robustness is the ability of a system to recover from various input errors and other failures (Naik & Tripathy, 2018). One definition of software failure in conventional systems is that it occurs when the customer's expectations are not met and/or when the software does not help the customer (Chillarege, 1996).

Software Testing (ST) is a process that can support the management of possible failures and unexpected environmental events over prolonged time frames. Afzal *et al.* (2020) conducted a qualitative study on challenges in testing robotics systems. They identified some testing challenges related to writing, designing, running, and automating tests for robotics

systems. According to the authors, one of the significant challenges is related to the **coordination and collaboration** among multiple teams (especially software and hardware teams) and documentation (there are very few standards and guidelines for practitioners to guide their testing). It stems from the need for adequate channels within many robotics companies, which often require bridging gaps between teams with diverse backgrounds, such as software and hardware teams.

Additionally, it is common to integrate and test third-party components without any accompanying documentation. Moreover, additional standards and guidelines for writing and designing tests for robotic systems are still needed. To exemplify, a representative quote from one of the respondents is *"There are so many people working on the same software stack. Sometimes there will be an inconsistency."*

Another challenge identified based on participants' responses concerns the **culture of testing**. One defining feature of the robotics community is its ability to bring together individuals from various disciplines, such as electrical and mechanical engineering. This diversity not only drives significant advancements in robotics but also poses certain challenges. To exemplify, a representative quote from one of the respondents is *"The world of robotics unites folks from different backgrounds. Folks from a software background might observe testing differently from those who are not."*

Running and automating tests for robotics systems is costly. Testing can be both risky and expensive, requiring developers and test teams to dedicate extensive time to executing test scenarios on robots. However, **time and resources**, including physical robots, are limited. To exemplify, a representative quote from one of the respondents is *"Full testing is expensive because you have to pay for the labour for someone to test. If there is downtime for robots, then there is a cost to pay people to wait for the fix."*

To this end, using simulated environments can reduce testing costs and expand opportunities for test automation. [Timperley et al. \(2018\)](#) contend that many real-world robotics bugs can be replicated and addressed in simulation environments. Moreover, simulations mitigate the risk of damaging equipment ([Bossecker et al., 2023](#)), eliminate the necessity for physical prototypes ([Roth et al., 2003](#)), and offer a cost-effective means to implement changes ([Robert et al., 2020](#)).

Simulation testing is considered a promising approach to automating tests in robotics ([Seshia et al., 2017](#); [Zheng et al., 2014](#)); for example, significant investments have been made to improve the simulation of self-driving cars. However, running and automating tests for robotics systems using simulators poses the crucial challenge of **distrust in simulation** ([Afzal et al., 2021](#)). To exemplify, a representative quote from one of the respondents is *"We mostly do field testing. That is what really affects what happens. The robot gets lots of impact from the environment. Simulation just doesn't reflect the real world."*

Some studies apply software testing in robotic systems to handle software failures. The study by [Nguyen et al. \(2023\)](#) analyzes robotic application requirements and acceptance criteria, explicitly focusing on robotic competitions and benchmarks. [Ashraf et al. \(2020\)](#) proposed coverage criteria for white-box testing to evaluate industrial robot tasks and a framework to generate test cases automatically to satisfy their defined coverage criteria. In the study conducted by [Bretl & Lall \(2008\)](#), the authors do not explicitly apply software testing techniques. However, they evaluate the robot's behavior. [Estivill-Castro et al. \(2018\)](#) proposed a robot simulator based on the Model-View-Controller software pattern and integrates simulation into a continuous integration paradigm to scale up testing integration with robot behavior.

[Erich *et al.* \(2019\)](#) presented a framework for automatically testing applications for collaborative robots. [Mossige *et al.* \(2015\)](#) presented cost-effective automated testing techniques to validate complex industrial robot control systems. [Alexander *et al.* \(2015\)](#) proposed the concept of situation coverage and evaluated it by testing a simple simulated autonomous road vehicle. [Yun Koo Chung & Sun-Myung Hwang \(2007\)](#) presented a testing process and evaluation elements for the software testing of intelligent robots.

To our knowledge, our work is the first to apply automated acceptance testing to industrial robotic systems using the BDD template and to evaluate the effectiveness of the approach using mutation testing.

There is a need for more specialized software testing approaches in robotics, particularly for IRS. Additional research is needed to address knowledge gaps in writing, designing, executing, and automating tests for IRS ([Afzal *et al.*, 2020](#)).

Taken together, these challenges show that current testing practices for Industrial Robotic Systems remain fragmented, resource-intensive, and strongly dependent on manual processes, ad hoc coordination, and undocumented acceptance criteria. Although prior work has proposed isolated testing strategies, none of these efforts have established a unified or standardized approach capable of ensuring the software-quality characteristics required for dependable robotic operation. This gap becomes even more critical when considering that the IRS increasingly operate in environments where reliability and robustness are essential for safety and operational continuity. Therefore, the limitations identified in the literature highlight the need for a theoretically grounded framework and that systematically addresses the lack of standardization in automated testing for IRS.

Problem Statement

IRS operate within dynamic environments but lacks a standardized framework for automating acceptance testing. This issue becomes even more significant when we consider software quality aspects, reliability and robustness, which are essential for preventing safety hazards and economic losses. Current workflows rely heavily on manual processes and informal acceptance criteria, resulting in inconsistent verification of software quality. This thesis addresses the need for a standardized, automated approach to acceptance testing for IRS to strengthen the overall dependability of robotic systems.

1.1 RESEARCH GOALS

As in traditional software systems, developers and stakeholders can collaborate to improve software quality through testing. As part of this thesis, we conducted a systematic literature review to understand better how SE (and its sub-areas) are applied in Robotic Systems (RS) from a software quality perspective. We found that only 18% of the studies explicitly investigated software quality concepts in robotic systems, and 5% investigated software testing.

We propose a software testing approach for IRS that adapts a well-established natural and common language template (Acceptance Testing (AT) from Behavior-Driven Development (BDD)) to help write, design, run and automate tests for IRS using simulation.

However, the distrust of simulation is a challenge observed in the [Afzal et al. \(2020\)](#) study. Some reasons for the distrust in simulation may arise from the fact that is hard to simulate the environmental complexity that occurs in physical world. For example, the noise in sensors can lead to *false positive/negative* test results. To improve the trust in simulation, we propose an evaluation using mutation testing.

Mutation testing is a software testing technique in which the original code is modified to create mutated versions or mutants. However, we observe that the code to control IRS contains few control structures variables or arithmetic operations, which are the typical points where mutation operators are applied. There are very few locations where mutations can be injected. These mutants provide no added value as they are unable to reveal any additional gap in the coverage achieved by the test suite. This has long been identified as a problem that reduces the effectiveness of mutation testing ([Untch *et al.* \(1993a\)](#)). So, our goal is to create a comprehensive set of mutants that accurately represents the solution space under examination and to provide informative value related to the effectiveness of our approach. We have also incorporated non-deterministic mutants that mimic the unpredictable actions observed in simulation and robotic systems.

Our overall research objective is translated into three specific Research Goals (RGs)

- **Research Goal (RG)1 - Systematic mapping of software engineering approaches for robotic system from a quality aspects perspective.** This SM aims to identify and classify the state-of-art of software engineering for robotic systems from a quality software perspective.
 - **Contribution:** A systematic analysis of software engineering areas applied in robotic systems. As the main object of the research is the intersection between software testing and IRS, the preliminary provides evidence to better understand how roboticists apply software engineering techniques in robotics. This work provided evidence that motivates this thesis.
- **RG2 - Definition and validation of the approach AAT4IRS.**

- **Contribution:** An approach that relies on BDD ([Smart \(2014\)](#)) templates for acceptance testing to enhance the reliability of robotic systems by improving fault detection.
- **RG3 - Evaluation guideline to apply mutation testing for IRS**
 - **Contribution:** A proposal and an evaluation of a guideline to apply mutation testing for IRS that incorporates unpredictable actions observed in IRS and also domain-specific characteristics to improve the trust in simulation-testing.

1.2 SUMMARY OF CONTRIBUTIONS

1.3 THESIS OUTLINE

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a background on the core concepts of IRS and software testing.
- **Chapter 3** presents a systematic mapping of software engineering for robotic systems. The results of our systematic mapping guide us to propose a software testing approach and framework to improve reliability of IRS.
- **Chapter 4** introduces the process of automating acceptance testing for industrial robotic systems using simulation and
- **Chapter 5** presents the application of the AAT4IRS process through proof-of-concept, performing a pick-and-place process, and validation using mutation testing with evaluation guidelines for applying mutation testing to IRS.
- The **Conclusion** of this thesis by highlights the research contributions and summarizes limitations and directions for future work.

CHAPTER II

BACKGROUND AND RELATED WORK

This chapter provides the necessary background to understanding the key concepts and challenges addressed in this thesis. It is structured into two main sections:

- **Foundations of Industrial Robotic Systems** – This section introduces the key principles of industrial robotics, including robotic programming, how the software for IRS differs from conventional software, robot positioning, and sensor technologies. These concepts are essential for understanding how robotic systems operate and how their software is developed and tested.
- **Foundations of Software Testing** – This section introduces fundamental software testing methodologies and sub-areas, with a particular focus on test automation, quality aspects that are essential for this thesis (robustness and reliability), as well as BDD and Mutation Testing. These methodologies, widely recognized in conventional software systems, provide a foundation for ensuring software correctness and reliability. By first presenting these well-established approaches, we create a basis for subsequently adapting them to the specific challenges of robotic systems, ultimately supporting the definition of our proposed solution.

2.1 FOUNDATIONS OF INDUSTRIAL ROBOTIC SYSTEMS

IRS are complex systems that integrate hardware and software components. Figure 2.1 shows an IRS composed of three industrial robots (A, B and C) that perform tasks to automate industrial processes through a software. Each IR has a gripper and the IRS has also a conveyor.

Furthermore, we can see a representation for a human (D) who interacts with the IR through the software component.

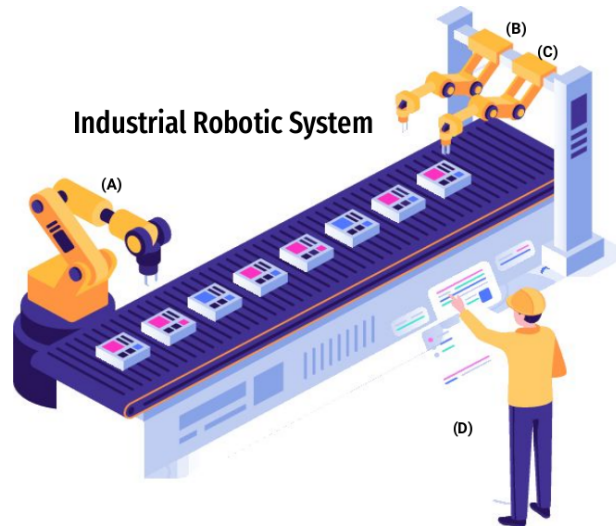


Figure 2.1 : Industrial Robotic System

IRs are widely used across manufacturing sectors to carry out repetitive and high-precision tasks that require speed, consistency, and endurance. According to [ISO/IEC 8373 \(2012\)](#), common tasks performed by industrial robots include pick-and-place operations, welding, soldering, painting, palletizing, assembling, and inspection. These tasks often involve interactions with the physical environment and must accommodate varying degrees of uncertainty in object positioning, sensor input, and timing.

Among these, pick-and-place is one of the most fundamental and commonly automated tasks, making it especially relevant in production lines, packaging systems, and laboratory automation. Since pick-and-place is the task used in our experiment, it is important to highlight its structure and relevance. According to [Mnyusiwalla et al. \(2020\)](#), the pick-and-place operation is typically divided into four distinct phases: (i) pre-grasping, in which the

gripper is moved to the object's initial position; (ii) grasping, where the robot securely grips the object, breaking contact with the original surface; (iii) transport, during which the object is moved to the destination; and (iv) placement, where the robot releases the object at the designated location. Each of these phases requires precise coordination of sensing, motion control, and timing, making pick-and-place a representative and meaningful task for evaluating robotic system performance.

To better understand the challenges of implementing and testing such tasks in IRS, it is necessary to examine the core elements that shape the behavior and reliability of these systems. They are: (i) hardware component, and (ii) software component.

2.1.1 HARDWARE COMPONENT

An IRS is composed of three primary components: an IR, the end effector, and auxiliary equipment. This section focuses on the hardware components of the IR, as depicted in Figure 2.2.

The discussion begins with the joints, which serve as pivotal points of motion that enable the robot to move and interact with its surroundings. These joints connect rigid segments known as links, forming the mechanical framework of the robot. Together, joints and links establish the robot's capacity to perform various tasks within its operational workspace.

In typical robotics applications, the position of the end effector (such as a gripper or tool) within the workspace, along with the orientation and configuration of the joints, is used to define the robot's state. These parameters are crucial for analyzing and controlling robotic systems.

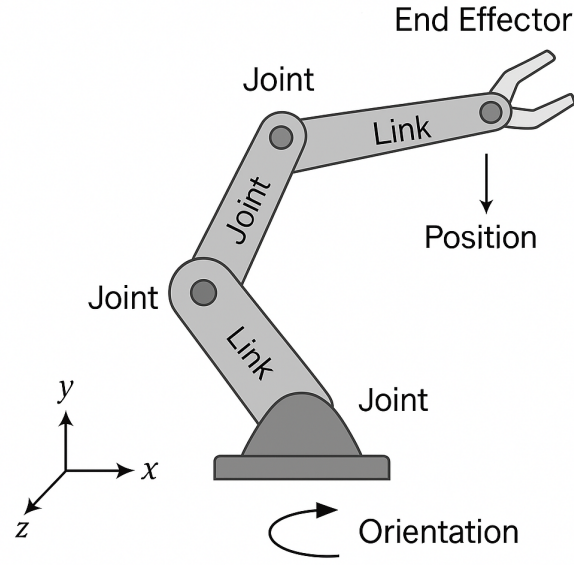


Figure 2.2 : IR components

Various positional systems are employed to describe the position and orientation of a robot in space, including Cartesian coordinates, Euler angles, quaternions, and axis-angle formulations. Each of these systems presents different methods for expressing spatial positioning, accompanied by unique advantages and limitations that depend on the specific application context. For instance, quaternions are often utilized to circumvent gimbal lock issues in rotational computations. While Euler angles offer an intuitive understanding, they may encounter singularities. Although axis-angle representations provide a compact method for describing rotations, they are less frequently adopted in industrial robotics programming (Craig, 2018).

In this thesis, we concentrate on the Cartesian coordinate system, as it offers an intuitive and practical geometric framework for analyzing robot motion within its workspace (Spong *et al.*, 2006). Although other representations may provide certain advantages in specific contexts, the Cartesian system aligns more closely with the goals of this research, which

focuses on validating IRS movement trajectories in relation to physical workspaces by applying automated acceptance testing.

The Cartesian coordinate system is the most intuitive and widely adopted framework in industrial robotics for defining spatial positions and trajectories. It provides a method to specify a point in space using a triplet (x,y,z) , corresponding to distances along the orthogonal X, Y, and Z axes. This system forms the foundation for motion planning and control in most industrial robots, as it aligns naturally with the physical layout of robotic workspaces (Craig, 2018). For example, an industrial robotic arm with a gripper (end effector) must move to specific coordinates (100 mm, 200 mm, 150 mm) to pick an object off a conveyor belt. Precise positioning is crucial, as even slight deviations can cause operational errors.

It is essential to recognize that while the Cartesian coordinate system allows for the specification of a point in space using three positional values (x, y, z) , it does not adequately capture a robot's orientation or joint configuration. For example, a robot arm may successfully reach a specified x, y, z location, but with an incorrect orientation that could hinder the execution of the intended task. However, modern industrial robot controllers and simulation libraries (e.g., ABB RobotStudio, KUKA KRL, Fanuc TP) incorporate inverse kinematics algorithms (Craig, 2018) that automatically determine joint angles required to achieve a target position and orientation. This capability allows engineers to specify spatial targets, while relying on the robot's control system to compute the necessary joint-level commands.

In this thesis, the primary focus is on verifying whether the robot's end effector accurately achieves the required spatial positions within the workspace, as criterion for acceptance testing in industrial settings. Thus, despite its limitations, the Cartesian framework is adopted as a practical and relevant approach for the validation tasks addressed in this work. Future ex-

tensions of this research could integrate considerations of orientation and joint-level kinematics to provide a more comprehensive analysis of robotic behavior.

The mechanical structure of a robot, which includes joints, links, and the end effector, serves as the foundation for movement and task execution. However, it is the integration of sensors that empowers the system to perceive and adapt. Sensors enable robots to detect and interpret a variety of environmental factors. This ability is essential for tasks that range from simple navigation to intricate decision-making processes. As robots are increasingly used in diverse fields such as manufacturing, healthcare, and autonomous vehicles, the reliability and accuracy of sensor data becomes critically important.

Sensors are critical components that measure various aspects of the environment. They can be broadly classified into two categories: proprioceptive sensors and exteroceptive sensors ([Ben-Ari & Mondada, 2018](#)).

Proprioceptive sensors monitor the robot's internal state, including joint positions, velocities, and torques. Common examples are encoders and potentiometers, which measure joint angles to ensure precise movement and positioning.

Exteroceptive sensors, on the other hand, gather information about the robot's external surroundings. This category includes:

- Temperature sensors, which help maintain optimal operating conditions and prevent overheating.
- Position sensors, essential for navigation and enhancing spatial awareness.
- Luminosity sensors, enabling robots to adapt to varying light conditions.

The integration of sensors empowers robots to operate safely and efficiently within dynamic environments. For instance, in settings with fluctuating lighting or high ambient noise levels, sensors allow robots to adjust their operations accordingly, maintaining performance and safety standards. Moreover, the data collected by these sensors are processed by the robot's control systems, facilitating informed decision-making and adaptation to new situations, thus enabling the execution of complex tasks. This sensory feedback loop is fundamental to the autonomy and intelligence of robotic systems.

Whenever a sensor is used to measure a physical quantity, it is essential to define and distinguish key performance metrics that describe the capabilities and limitations of an industrial robotic system (IRS). These metrics include range, resolution, accuracy, and precision, which together provide a comprehensive understanding of the system's measurement capabilities and performance ([Ben-Ari & Mondada, 2018](#)):

- Range refers to the span of values a sensor can measure. For instance, ultrasonic sensors used in indoor mobile robots typically have a range of up to 10 meters, whereas LiDAR sensors in autonomous vehicles can detect objects at distances exceeding 100 meters.
- Resolution denotes the smallest detectable change in the measured quantity. High-resolution sensors are crucial in applications requiring fine measurements, such as robotic assembly of small components, where even millimeter-level differences can impact task success.
- Accuracy indicates how close a sensor's measurement is to the actual or true value. High accuracy is vital in tasks where exact measurements are critical, such as positioning in robotic surgery or precision manufacturing.

- Precision refers to the ability of a sensor to consistently measure or detect data with high accuracy and repeatability. It is a critical characteristic that reflects how reliably a sensor can measure the same parameter multiple times under identical conditions. High precision is essential in robotics, as it directly impacts the robot's ability to perform tasks accurately. For example, in robotic surgery, even minor deviations in sensor readings can lead to significant errors in the accuracy and precision of the robot's movements, potentially compromising patient safety.

While the integration of sensors is crucial for allowing robots to perceive and interact with their surroundings. While, the accuracy and reliability of sensor data can deteriorate over time or fluctuate based on environmental conditions and manufacturing tolerances ([Hossain et al., 2024](#)). To maintain consistent and precise sensor readings, a calibration process is necessary.

Sensor calibration is the process of adjusting a sensor's readings to ensure they are accurate and consistent with known standards, such as industry benchmarks. Calibration involves comparing the sensor's output against a trusted reference and making necessary adjustments to align the readings. This process is essential for maintaining the integrity of sensor data over time, as sensors can drift and produce inaccurate readings due to factors such as wear and tear, environmental changes, or manufacturing defects.

Regular calibration is crucial for ensuring that sensors continue to provide accurate data throughout their operational lifespan. Without proper calibration, sensors may produce erroneous readings, leading to incorrect decisions and actions by robotic systems. For example, in a manufacturing setting, a miscalibrated sensor could result in defective products, causing financial losses and safety hazards. Therefore, implementing a routine calibration schedule is essential for maintaining the reliability and accuracy of sensor data in robotics.

Numerous case studies ([Lubrano et al., 2011](#); [Icli et al., 2020](#); [Li et al., 2021](#)) illustrate the critical relationship between sensor precision and calibration in robotics. For instance, in industrial robotics, companies such as FANUC and KUKA implement rigorous calibration protocols to ensure the precision of their robotic arms, which are used in high-stakes manufacturing environments ([IFR, 2020](#)).

Despite meticulous design and calibration, sensors in industrial robotic systems are inherently prone to measurement errors caused by various factors such as noise, drift, and environmental disturbances. These errors represent the discrepancy between the robot's actual state and the state reported by its sensors, which can potentially impact the system's internal estimation and control processes. Although proper calibration procedures can mitigate these errors, a certain degree of uncertainty is unavoidable and must be accounted for in the design and validation of robotic systems.

In addition to sensor inaccuracies, it is important to consider the concept of an error threshold in robotic applications. The error threshold determines the maximum allowable deviation between the robot's actual position and the position specified by the task requirements ([Ben-Ari & Mondada, 2018](#)). For instance, if a robot arm is programmed to place an object at a specific location, the task might establish an error threshold. Indicating that the robot's end effector must be within the error threshold of the target for the task to be deemed successful. If the deviation surpasses this threshold, the task is classified as unsuccessful, necessitating corrective actions.

2.1.2 SOFTWARE COMPONENT

IRS software plays a critical role in controlling robotic movements, processing data from sensors, and ensuring the precise execution of tasks. The role responsible for designing,

developing, and maintaining the software component that controls robots is the Robotics Software Engineer (RSE). They work closely with mechanical engineers and other team members to integrate software with hardware components and ensure the smooth operation of robots. Robot software engineers also test, debug, and optimize code to enhance the efficiency and functionality of robotic systems.

In addition to the technical aspects of the software component, RSEs must also consider the broader context in which robotic systems operate—particularly the Business Requirement (BR) that guide industrial applications. In contrast to conventional software systems, where BR are usually well-defined and formally documented, industrial robotic systems often prioritize task execution over detailed business-level specification. In many robotic applications, especially in manufacturing contexts, the primary objective is simply to ensure that the robot can perform the required physical task—such as welding, assembly, or pick-and-place—with sufficient precision and reliability.

However, there is a limitation in the accessibility of real-world industrial requirements, as noted by [Nguyen *et al.* \(2023\)](#). As such, we turn to the Robotic Grasping and Manipulation Competition’s Task Pool for direction. For example, the task design for “Pick Up and Place Using Tongs” ([Sun *et al.*, 2018](#)).

Although not explicitly articulated as formal business requirements, the competition rules establish key operational constraints and objectives—such as tool utilization, accuracy of placement, and management of task failures—that are directly comparable to business-level requirements in industrial applications. These rules encapsulate the expectations that robotic systems face in real-world scenarios. Therefore, they can be viewed as domain-specific articulations of business needs, structured in a manner suitable for the validation and assessment of robotic systems.

For the rules shown in the task “Pick Up and Place Using Tongs” ([Sun et al., 2018](#)), a business requirement might be stated as:

Business Goal - Pick Up and Place Using Tongs

The robot must be capable of transferring items from an input area to a designated zone with a positional tolerance of ± 5 cm, using only the provided end-effector tool, ensuring a minimum task success rate of 80% under time constraints, and skipping failed pickups to maintain cycle efficiency. .

To create the software component that allows a robot to perform the tasks, there are some programming methods depending on factors like the level of abstraction, the complexity of the system, and the tools at hand. According to [Heimann & Guhl \(2020\)](#), industrial robot programming methods can be categorized based on the interaction between the operator, who is responsible for programming and operating the robot, and the robot itself. These methods are classified as online, offline, or hybrid.

In the *online* method, the operator programs the robot directly on the shop floor using either the "lead-through method" (manually guiding the robot through a trajectory) or the "teach-pendant method" (guiding the manipulator to specific points, recording these points to create a trajectory, and then executing it). This process requires the production on the shop floor to be halted while the industrial robot system is programmed.

For example, an RSE stands next to a robotic arm on the factory floor, holding a teach pendant. They guide the robot through a series of movements—for example, picking a part from a conveyor and placing it on a shelf—pressing buttons to record positions and motions. Each step is executed and verified in real time. The robot stores this sequence as its task routine.

The *offline* method involves the RSE utilizing programming languages integrated with a simulator, allowing them to design, test, and refine robot behavior without using the physical robot. This setup supports more efficient development and safer testing environments by leveraging virtual representations of robotic systems and their workspaces.

In this context, it is important to consider the nature of the programming language, there are specialized Industrial Robot Programming Language (IRPL)s for robot control that include instructions for moving the robot’s arm(s) and accessing low-level resources (Pogliani *et al.*, 2020). Some IRPLs that are dedicated for a specific brand in which is possible to program only for the specific robots’ brand (ABB, 2010; KUKA, 2021).

However, there are ways to program robots using generic or general-purpose programming languages—like Python, C++, JavaScript. One way is to use Robot Operating System (ROS), a standard framework for developing robotic software. It is a collection of libraries, tools, and conventions. The main aim of ROS is to simplify the task of creating robot software. A system created with this framework consist of many different programs running simultaneously and communicating with one another, passing messages (Quigley *et al.*, 2015). ROS acts as a middleware between the robot program and the robot hardware.

In addition to the programming strategies presented so far, another essential component of the *offline* methodology adopted in this work is the use of simulators. These tools play a crucial role in both the development and validation of robotic behaviors prior to deployment. Simulators make it possible to test control algorithms, sensor configurations, and robot-environment interactions in a safe, cost-effective, and repeatable manner.

Simulators are software platforms that replicate the behavior of physical robots within a virtual environment. Typically, simulators offer support for physical dynamics, realistic 3D environments, and various robotic components, including sensors and actuators. Simu-

lator environments can lower testing expenses and expand opportunities for test automation. [Timperley et al. \(2018\)](#) contend that many real-world robotics bugs can be replicated and addressed in simulation environments. Moreover, simulations mitigate the risk of damaging equipment ([Bossecker et al., 2023](#)), eliminate the necessity for physical prototypes ([Roth et al., 2003](#)), and offer a cost-effective means to implement changes ([Robert et al., 2020](#)).

Simulating a robot entails replicating the behavior of a robotic system within a virtual environment, enabling thorough testing and development before physical deployment. This process involves multiple layers of modeling and integration. First, the robot itself must be virtually constructed, encompassing aspects such as its kinematics (movement and joint configurations), dynamics (mass, inertia, and force interactions), and embedded systems, which include sensors and actuators (e.g., wheels, arms, grippers). Simultaneously, the environment must be modeled to accurately reflect real-world characteristics, including obstacles, terrain topologies, surface friction, and lighting conditions. Lastly, physics simulation engines are employed to create realistic interactions, factoring in elements such as gravity, collisions, joint constraints, and force propagation ([Koenig & Howard, 2004](#); [Rohmer et al., 2013](#)).

A range of robotic simulators is available to support the development and testing of robotic systems. Among the most widely adopted platforms are Webots, recognized for its user-friendly interface and educational focus ([Michel, 2004](#)); CoppeliaSim, which provides robust scripting and multi-robot simulation capabilities ([Rohmer et al., 2013](#)); MuJoCo, favored in reinforcement learning ([Todorov et al., 2012](#)); and Unity-based simulators, which are increasingly utilized for vision-based tasks and synthetic data generation ([Le et al., 2014](#)). Among the available options, Gazebo emerged as the leading choice for robotic simulation due to its ability to replicate diverse robotic platforms equipped with standard sensors like cameras, GPS units, and IMUs. Despite operating independently, Gazebo seamlessly integrates with

ROS via the "gazebo_ros" package, enabling bidirectional communication (Quigley *et al.* (2015); Farley *et al.* (2022)).

Gazebo was selected for this research due to its strong integration with the Robot Operating System (ROS), its open-source nature, and its ability to simulate complex environments with realistic physics (Koenig & Howard, 2004) . Gazebo provides a scalable and modular architecture, a wide range of plugins for sensors and actuators, and an active user community, making it particularly suitable for iterative testing and integration of ROS-based robotic software.

Lastly, the *hybrid* method combines elements of both online and offline approaches. The operator works offline to develop the programming flow, calibrate the system, and validate it physically online (by running the program on the robot in a safe environment, and speed). The hybrid method leverages the advantages of both the online and offline programming methods.

The Table 2.1 presents a comparative overview of these three methods based on key technical and operational aspects.

Table 2.1 : Comparison of Online, Offline, and Hybrid Robot Programming Methods (Pan *et al.*, 2010; Heimann & Guhl, 2020; Zheng *et al.*, 2022)

Aspect	Online	Offline	Hybrid
Definition	Programming directly on the physical robot in real-time.	Programming in simulation environments before real-world deployment.	Combines simulation with real-world interaction or validation.

Environment	Real robot hardware.	Virtual environment (simulator).	Both virtual and real environments.
Hardware Requirement	Required during development and testing.	Not required until deployment phase.	Required for integration and real-world testing.
Development Cost	High (time-consuming and hardware-dependent).	Lower (faster, risk-free testing).	Moderate to high (requires both infrastructure types).
Programming Tools	Teach pendants, SDKs, terminal-based input.	Simulators (Gazebo, Webots, RoboDK), ROS, etc.	ROS, simulation platforms, physical testbeds.
Error Handling	Manual, physical debugging.	Iterative debugging in simulation.	Debugging in both environments.
Accuracy Before Deployment	High (tested on actual hardware).	Varies with simulation fidelity.	High (sim-to-real validation).
Reusability	Low (customized per robot/task).	High (portable, modular code).	High (reusable for deployment and retraining).
Expertise Required	Robotics hardware and control knowledge.	Robotics + simulation skills.	Robotics, simulation, and ML/AI integration.

Understanding the various methods for programming IRS is crucial. However, selecting a programming method is only one aspect of the overall process. The next step is validation, a critical phase in ensuring that robotic systems adhere to technical specifications and meet operational expectations defined in the BR.

In the field of robotics, validation is the process of confirming that a robotic system behaves as expected in its intended real-world environment and fulfills the defined requirements - both functional (what the robot does) and non-functional (how well it does it). Functionally, robots are designed to perform tasks such as object manipulation, navigation in an environment, and interaction with precision and reliability. Non-functional criteria, on the other hand, encompass safety, efficiency, adaptability, and strict adherence to ethical standards. For example, in industrial environments, robots must execute tasks safely, ensuring that human operators are not at risk, while complying with established safety standards such as [ISO \(2025\)](#).

These functional and non-functional aspects are directly connected to how we evaluate whether the robot meets expectations. Metrics like execution time, positioning accuracy, object manipulation, and obstacle avoidance are commonly used to assess performance. For example, reducing execution time is important in manufacturing, and accurate positioning is essential in tasks like assembly or surgery. Additionally, the robot must be able to manipulate objects without damage and navigate safely, especially in logistics and mobile applications. In general, energy efficiency, adaptability, and easy integration with other systems are also part of what we expect from a high-quality robotic solution.

The validation process in robotics can vary significantly depending on whether the system is programmed using online, offline, or hybrid methods. In the *online* method, the RSE must halt production, program the robot, and execute the program usually through real-time

execution and observation. The robot performs the task while RSE monitors parameters such as accuracy, execution time, and safety, adjusting the code as needed. In the *offline* method, validation occurs in a simulator. Only when the robot’s simulated behavior matches the previously expected behavior, the RSE sends software to the robot on the shop floor. This process heavily relies on the operator’s knowledge and experience regarding expected robot behavior. The simulation allows for repeatable and low-risk testing, but its results depend on how realistic the simulation is. Finally, the *hybrid* method combines both approaches: validation begins in the simulator, and the operator can fine-tune the program using the online method to make real-time adjustments as the robot interacts with the physical environment. This helps reduce development time and risk, while ensuring that the system behaves correctly when deployed.

Despite the differences in these methods, typically they share a common aspect: validation relies on the operator’s expertise and manual inspection of the robot’s behavior. If there are changes to the BR—whether through modifications or the addition of new features—the operator must update and re-validate the entire program, even if part of it was previously validated. This manual approach to validation significantly increases both the project timeline and costs.

Among the three methods for programming IR, another core element to understand the approach proposed in this thesis is how the software in IR differs from the software used in conventional system.

Conventional software systems are typically designed to operate in discrete, well-defined digital environments. Inputs, outputs, and system states are predictable and can often be specified with precision.

By contrast, IRS integrate hardware and software with the physical components that must operate in the physical world. The software in a robot needs to interact with a world that is continuous, dynamic, and full of uncertainty (Groover, 2007).

Furthermore, robots sense and act upon the physical world using components like cameras, force sensors, and motors—all of which are inherently noisy and imprecise . Unlike a web or mobile app, a robotic arm may misplace an object due to sensor noise or slight mechanical drift. These systems are also sensitive to timing—a delay of milliseconds in a control loop can cause a robot to miss or mishandle a task (Li, 2016).

Moreover, the state space in which robots operate (e.g., the real world) is vast and can not be exhaustively tested or simulated. As a result, emergent behaviors—unexpected actions that arise from complex interactions between components—are common and often difficult to predict or replicate (Afzal *et al.*, 2020).

Table 2.2 shows the key difference between conventional and IRS software related to the software characteristics.

Table 2.2 : Key difference between Conventional System and IRS software

Aspect	Conventional System	IRS
Environment	Fully digital, abstract, and predictable	Physical world: dynamic, continuous, and unpredictable
Inputs & Outputs	Clear, often discrete	From noisy, real-world sensors; can include images, distances, or forces

Timing	Timing may be abstracted or loosely managed	Highly time-sensitive; small delays can lead to failure
State Space	Finite and well-bounded	Practically infinite due to real-world variability

Engineering software for IRS requires adherence to rigorous quality requirements, including reliability and correctness (García *et al.*, 2020; Brugali & Scandurra, 2009; Marijan *et al.*, 2019). However, it is important to highlight that quality aspects for IRS need to have into account all specificity of these system (Table 2.2).

For example, robustness is a main challenge in IRS systems (García *et al.*, 2020) . Robustness is the system’s ability to maintain acceptable performance when facing uncertainties, disturbances, or unexpected changes (e.g., sensor noise, environmental variations). Since, reliability is the ability of a system to perform its intended function consistently and without failure over time and across multiple cycles. So, reliability in robotic systems requires robustness against uncertainties like sensor noise, unexpected inputs, and environmental change.

Correctness is often measured by performance or task success under varying conditions, rather than strict logical compliance (Marijan *et al.*, 2019). For example,in a industrial environment in which a robot’s task is to detect and move metallic parts from a moving conveyor to a target bin. Unlike conventional software, where correctness means producing the exact output for a given input, the robot’s success must account for real-world variations—such as slight misalignment of parts, lighting changes, and sensor noise. In this context, correctness might be defined as “placing the object within a 2 cm radius of the target center,” which represents a statistically defined acceptable error margin.

To summarize, all these difference between conventional and IRS software clarify the demand for specialized testing techniques capable to improve software quality aspects for IRS. Hence, there is a clear need for automated software testing solutions for industrial robots.

In conclusion, industrial robotic systems have challenges distinct from those encountered in traditional software systems. Due to their interaction with the physical world, these systems rely heavily on precise positioning and the integration of various sensor technologies. Robotic programming must take into account not only software logic but also factors such as timing, uncertainty, and safety. These considerations contribute to the increased complexity of developing and testing robotic software.

The following section presents the fundamentals of software testing, including basic testing concepts, test automation, BDD, and mutation testing.

2.2 FOUNDATIONS OF SOFTWARE TESTING

IRS integrate hardware and software to execute precise, safety-critical tasks. Their complexity arises from the need to coordinate multiple subsystems in real time, ensuring accuracy and reliability under varying conditions. Software plays a vital role in IRS, as it dictates the robot's behavior. However, developing robust robot software presents challenges, including managing uncertainties like sensor errors, communication delays, and unexpected environmental factors ([Afzal *et al.*, 2020](#)). Consequently, ensuring software quality in IRS is crucial. Without rigorous software quality assurance practices, the risk of system failures escalates, potentially resulting in operational downtime, financial losses, or safety incidents in industrial environments ([Ssanderson *et al.*, 1986](#); [Travis, 2015](#); [Reynolds, 2023](#); [Layne, 2023](#)).

To mitigate these risks, software engineering offers structured methodologies and best practices that steer the creation of high-quality software. This includes activities such as

requirements engineering, architectural design, code reviews, static analysis, and software testing ([Pressman, 2010](#); [Sommerville, 2000](#)).

Among these activities, software testing is a crucial activity for evaluating a system's performance in various conditions. It provides evidence that the implemented system meets its specifications and helps identify defects that could compromise the system's reliability during real-world operations. Reliability, in this context, refers to the system's capacity to consistently perform its intended functions over time, especially when faced with disturbances, environmental fluctuations, or internal changes. Testing plays an important role in improving confidence in the software's stability and fault tolerance ([Naik & Tripathy, 2018](#)).

In robotics, where software errors can lead to tangible physical consequences, the stakes for ensuring reliable behavior are significantly elevated. Consequently, testing practices must adapt to confront the distinct challenges posed by robotic systems, including sensor uncertainty, real-time constraints, hardware dependencies, and dynamic interactions with their environments ([Afzal *et al.*, 2020](#)).

2.2.1 BASIC CONCEPTS

Software testing is the process of executing a program to identify and rectify errors, ensuring that the software functions as intended. However, creating reliable software involves more than simply preventing crashes. It requires a structured and deliberate approach to detect, address, and prevent any issues that could compromise the system's stability, particularly in complex real-world environments ([Sommerville, 2000](#)).

To achieve this effectively, it is crucial to understand the differences between an error, a fault, and a failure. Each term represents a distinct stage in the life cycle of a potential issue: an error refers to a human mistake made during development, a fault is the manifestation of

that mistake within the code, and a failure occurs when that fault causes the software to behave incorrectly. Understanding the connections among these concepts is essential for developing software that is both reliable and resilient :

- **Error:** An error is a state of the system. In the absence of any corrective action by the system, an error state could lead to a failure which would not be attributed to any event subsequent to the error (Naik & Tripathy, 2018). For example, a developer misinterprets a requirement and writes the logic for picking up an object using the wrong coordinate system (e.g., world coordinates instead of local arm coordinates).
- **Fault:** A fault is the adjudged cause of an error (Naik & Tripathy, 2018). For instance, the robot arm's control module contains incorrect transformation calculations due to that mistake.
- **Failure:** A failure is said to occur whenever the external behavior of a system does not conform to that prescribed in the system specification (Naik & Tripathy, 2018). For example, when the robot runs this code during operation and places the object several centimeters away from the target location, missing the placement zone and causing a production delay.

In safety- or mission-critical systems like IRS, the impacts of failure can be significant. As a result, software engineering utilizes quality assurance processes aimed at preventing mistakes, identifying faults early on, and minimizing the risk of failures during operation (ISO/IEC 25010, 2011). Two core concepts within QA are verification and validation, which play complementary roles in achieving reliability:

- **Verification:** It involves checking that the software meets its specifications through methods such as code reviews, static analysis, and formal verification. Verification primarily targets internal consistency and correctness during the development process.
- **Validation:** It ensures that the final system meets the user's needs and performs correctly in its intended environment. Validation includes system testing, user acceptance testing.

Software testing encompasses both verification and validation activities. It involves systematically executing the software under controlled conditions to observe its behavior, identify faults, and assess its robustness. In the context of IRS, testing is vital for ensuring software reliability by evaluating its performance under edge cases, disturbances, and unexpected inputs. There are four distinct levels at which software testing may be performed (Naik & Tripathy, 2018).

In the first level we have **unit testing**. Testing team test programs units (procedure, functions, methods or classes) with the aim to ensure if the unit works or not according to a specification. **Integration testing** is the next level. In this level the Testing Team test the sub-system, they assemble the units with the objective to construct a reasonable system. The main goal here is verify if the interfaces between the units works well and without errors. **System testing** is the third level in the Verification and Validation (V&V). On this stage testing team evaluate the system as a whole regarding the requirements. We include all the external interfaces and simulated real life scenarios but it is performed by the development team inside the organization.

Acceptance testing, the testing applying on this thesis, confirms whether a system aligns with predefined Acceptance Criteria (AC), which are based on quality requirements. The AC acts as objective checkpoints to ensure that these quality requirements have been satisfactorily met throughout the development process. These requirements outline measurable

attributes essential for achieving the desired characteristics of the system, including functional correctness, performance benchmarks, and reliability standards.

There are two types of acceptance testing:

- **User acceptance testing:** the AT performed by the end user and ensure that the system satisfies the AC, for example a robotic arm control system on a manufacturing line running common production tasks using the new interface. A user acceptance testing would involve operators (the actual end users) checking if the robot responds correctly to inputs.
- **Business acceptance testing:** a type of AT focused specifically on validating that the software meets the broader business objectives and processes. For example, a plant manager in an industrial environment checks whether integrating this robotic system helps reduce production time, lowers costs.

Regression testing represents an essential phase in the verification and validation V&V model, conducted continuously throughout the development process. Its primary purpose is to ensure that new updates do not reintroduce faults into previously stable components of the system (Naik & Tripathy, 2018). In the context of industrial robotic systems, regression testing plays a crucial role in preserving the integrity of core functionalities—such as safety protocols, motion control, and sensor responses—amid the integration of new features or evolving requirements.

The software testing process begins with the selection of inputs, which may be derived from requirement specifications, the source code, or prior knowledge of the system. Once the inputs are selected, the corresponding expected outcomes for each input are defined. With both the selected inputs and their anticipated results in hand, the program can be executed. The

tester runs the program using these inputs, observes the actual outcomes, and compares them with the expected results to identify any discrepancies. Moreover, as stated in International Software Testing Qualifications Board (ISTQB) Glossary ([ISTQ, 2021](#)), the program and environment together form the System Under Test (SUT).

A pair with an input and its expected outcome are the **test cases**. However, the outcome of a test case is often more complex than a single value. It can include the values returned by the software, changes in internal or external state (such as updates to a database), and additional contextual information ([Naik & Tripathy, 2018](#)).

In the context of industrial robotics, the SUT could represent the software module responsible for controlling the movements of a robotic arm. During testing, inputs such as target coordinates, movement speed, and grip strength are provided, with the expected outcome being a successful pick-and-place operation. This testing process aims to verify whether the robotic arm performs as intended under these conditions, thereby confirming the correctness and reliability of the SUT.

2.2.2 SOFTWARE QUALITY: ROBUSTNESS AND RELIABILITY

SE provides methods, techniques, and tools to support the development of quality software. There are some software quality models ([McCall *et al.*, 1978](#); [Boehm *et al.*, 1976, 1984](#)). One of the most influential models, the McCall model, introduced eleven quality characteristics that have long guided software evaluation: Correctness, Reliability, Efficiency, Integrity, Usability, Maintainability, Testability, Flexibility, Portability, Reusability, and Interoperability. Many of these characteristics remain relevant today and appear, in updated form, in modern standards such as ([ISO/IEC 25010, 2011](#)).

This thesis was motivated by one of the main challenge in robotic systems, robustness ([Afzal et al., 2020](#)). Robustness and reliability (one of the software quality aspects defined at ([ISO/IEC 25010, 2011](#))) are related.

Reliability refers to a system's ability to perform its intended functions consistently under specified conditions. It is evaluated through indicators such as failure frequency and severity, mean-time-to-failure, output accuracy, and the system's capacity to recover from failures ([Pressman, 2010](#)).

Robustness, in turn, is defined as the ability of a system to cope with incorrect inputs and unexpected situations without compromising its essential behavior ([Naik & Tripathy, 2018](#)). Robust systems are capable of maintaining safe and stable operation even when the environment or the inputs diverge from what was originally anticipated.

Although reliability and robustness are distinct concepts, they are closely related. Reliability focuses on correct behavior under expected conditions, while robustness extends this expectation to unforeseen or adverse scenarios. By improving fault detection, our testing approach strengthens reliability, since the system fails less frequently and behaves more consistently. This increased reliability, in turn, contributes to robustness because a stable and predictable system is better equipped to maintain its essential functions when faced with unexpected inputs or environmental variations.

This thesis was motivated by robustness, identified in the literature as one of the main challenges in robotic systems ([Afzal et al., 2020](#)). Since reliability is a core quality characteristic defined in [ISO/IEC 25010 \(2011\)](#) and strongly linked to robustness in practice, our work focuses on improving reliability by enhancing fault detection through AAT4IRS

2.2.3 TEST AUTOMATION

Software testing is an activity that demands intensive energy and time by the stakeholders. Some of these activities can be performed automatically. Test automation involves utilizing scripts, tools, and frameworks to execute test cases automatically, assess results, and report outcomes with minimal human input. (Naik & Tripathy, 2018).

In a fundamental test scenario involving an IRS, the objective is to evaluate the robot's ability to accurately pick up an object from a designated location and place it onto a conveyor belt. In this manual testing approach, the tester activates the robotic system, positions the object at the pickup location, and engages the control interface to initiate the pick-and-place routine. Throughout this process, the tester observes the robot's performance to determine if it executes the task correctly, recording the outcomes manually. This method can be time-consuming and heavily dependent on human observation, increasing the likelihood of inconsistency or error—especially when the test is repeated multiple times.

In the automated variant of the same test, a script sends a command to the robot to initiate the pick-and-place routine. The environment automatically prepares the object at the pickup point. The robot performs the task, and the system records the final position of the object. These results are subsequently compared to the expected values to determine the accuracy of the robot's behavior.

A test framework, which provide reusable components and tools to support writing, running, and maintaining automated tests, manages this entire process: it oversees test execution, gathers data, applies evaluation criteria (such as tolerance thresholds), and stores the results. This methodology enables the tests to be conducted repeatedly without human intervention, yielding consistent and measurable outcomes. As a result, automated testing proves particu-

larly effective for regression testing and is instrumental in enhancing the reliability of robotic systems undergoing continuous development.

Test automation has some challenges that indicate the necessity of careful planning and ongoing maintenance. A primary concern is the initial investment of time and resources required to design, implement, and stabilize the testing infrastructure. Automated tests can also be sensitive to changes in the system under test, resulting in fragile test suite. Additionally, managing a large volume of test scripts can become expensive if not organized effectively.

Another frequent challenge is determining whether a test has passed or failed. Furthermore, test automation often demands specialized tools and skills, which can elevate the learning curve for both development and testing teams. In the context of IRS, the interaction with the physical world introduces variability, timing constraints, and non-deterministic behavior that can be difficult to replicate or simulate consistently. Establishing automated tests may necessitate the use of physical hardware, simulators, or hybrid environments, each requiring considerable setup and ongoing maintenance.

Additionally, defining test oracles poses a significant challenge when assessing physical outcomes, such as motion accuracy or object handling, where even minor variations can be critical. Furthermore, diagnosing test failures in robotic applications often entails navigating both software and hardware components, complicating the cause analysis. These elements highlight the necessity of automation in robotics, while also underscoring its technical complexities.

However, there are a lot benefits of test automation as increased productivity of tests, better coverage of regression tests, reduced duration of testing phases, reduced cost of software maintenance, increased effectiveness of test cases. With test automation, the organization can create a rich library of reusable test cases. So, test automation facilitates the execution of consistent set of test cases. These benefits are particularly significant in IRS, where

testing can often be time-consuming, resource-intensive, and challenging to replicate manually. To effectively demonstrate how these advantages translate into tangible value for robotic applications, Table 2.3 summarizes the key benefits of test automation along with their specific implications for IRS.

Table 2.3 : Benefits of test automation and their impact on IRS. Based on (Naik & Tripathy, 2018; Graham *et al.*, 2008; ISTQ, 2021)

Benefit	Description	Impact on IRS
Repeatability	Tests can be executed multiple times with consistent results.	Enables reliable validation of robotic behavior under repeated scenarios.
Scalability	Large test suites can be run quickly and efficiently.	Supports testing of multiple robotic tasks, environments, or configurations in less time.
Cost-effectiveness over time	Reduces manual effort and cost as the system evolves.	Minimizes testing overhead for frequent software updates and integration with new hardware.
Early fault detection	Faults are identified soon after they are introduced.	Prevents bugs from reaching the physical environment, reducing the risk of hardware damage.
Improved reliability	Promotes consistent system behavior across versions.	Helps ensure that robot actions remain predictable and safe during deployment.

Increased productivity	Frees testers from repetitive execution tasks.	Allows engineering teams to focus on improving robotic logic and adding meaningful test scenarios.
Reduced testing time	Speeds up the entire testing cycle.	Makes it feasible to test robotic applications in CI pipelines or after minor changes.
Lower maintenance costs	Automation detects regressions early and reduces debugging efforts.	Reduces downtime and costs associated with fault diagnosis on physical robotic setups.
Greater test effectiveness	Automated tests support precise validations and log complex system states.	Makes it possible to detect subtle robotic failures that might go unnoticed in manual testing.

2.2.4 BEHAVIOR-DRIVEN DEVELOPMENT

Traditional software development process starts with Step 1, the business owner articulates their needs to a business analyst, who then compiles a requirements document that clearly outlines these needs in a structured format (Step 2). Subsequently, the developer uses this document to create the software system (Step 3). Once the development phase is finished, a tester generates test cases based on the specified requirements, independent of the implementation (Step 4). Finally, a technical writer creates documentation by transforming the software into functional and technical resources for end users and stakeholders (Step 5) ([Smart, 2014](#)).

The traditional development process can lead to misunderstandings among different roles, as each participant interprets and modifies the requirements in their own way. This can result in discrepancies between what the business wants, what is actually implemented, and what is tested or documented.

Behavior-driven development (BDD) is a development methodology aimed at aligning the business and technical needs of software. BDD provides a common language based on simple, structured sentences expressed in English (or the native language of the stakeholders) to facilitate communication between project team members and business stakeholders (Smart, 2014).

Behavior-Driven Development (BDD) process starts with discussions between the business owner and business analyst to identify business needs (Step 1). Rather than filtering these needs through multiple layers, the business analyst, developer, and tester work together closely to refine the requirements (Step 2). These requirements are documented as structured scenarios in natural language, often utilizing a format like Gherkin. These scenarios function as both specifications and automated tests, providing guidance to the developer and ensuring that the output aligns with expectations (Step 3). The tester then leverages these same scenarios to define the test cases (Step 4), which helps maintain consistency between the business requirements and test validation. Ultimately, the automation of these tests enables continuous feedback on the system's behavior while also contributing to the application documentation (Step 5) (Smart, 2014).

To illustrate the structure of BDD scenarios, consider a straightforward example involving a bank account. A potential business goal might be:

Business Goal - Bank Account online solution
Ensure customers can manage their accounts easily

From the business goal, a set of corresponding business requirements can be identified, outlining what the system must accomplish to achieve that goal. Using the bank account example, where the objective is to enable clients to manage their finances more effectively, several business requirements emerge: the system must allow users to check their account balances; it must prevent overdrafts by validating available funds prior to any withdrawal or transfer; and it must permit authenticated users to transfer funds between their checking and savings accounts at any time.

The second activity in BDD is writing a features. Feature is a tangible, deliverable piece of functionality that helps the deliver the business goal using a business requirement as a guideline. For example, for the business requirement "The system must allow authenticated users to transfer funds between their checking and savings accounts at any time.", some features are: (i) transfer money between accounts; (ii) withdraw money. For instance, the feature "transfer money between accounts" broken down using the user story format appears as:

Feature - Bank Account online solution

Feature: Transferring money to a saving account

As a bank client

In order to manage my money more efficiently

I want to transfer funds between my accounts whenever I need to.

The next activity is to write the scenarios. Scenarios are examples that illustrate the features. Scenarios act as AC helping to define when a feature is considered complete. They also provide developers with a clear guideline, ensuring that the system is built to align precisely with the business requirements.

Most BDD tools use a Gherkin format to write scenarios. Gherkin has become the most widely adopted language for creating executable specifications in tools such as Cucumber ([Cucumber Project, 2024](#)), Behave ([Behave Project, 2024](#)), and SpecFlow ([SpecFlow Project, 2024](#)), because it offers a structured, readable syntax that is machine-executable ([Chandorkar et al., 2022](#); [Pucciani, 2023](#); [Santos et al., 2024](#)). Gherkin employs a straightforward and structured syntax based on natural language—typically English—incorporating keywords such as Feature, Scenario, Given, When, Then, And, and But. This structure enables scenarios to be both easily readable by humans and executable by machines.

Typically, in Gherkin the requirements relate to a feature are grouped into a single file called *feature file*. It contains a short description of the feature followed by a number of scenarios, or examples of how feature works. The template to describe scenarios in BDD is: **Given** [pre condition for the scenario and test environment], **When** [action under test], **Then** [expected outcomes] ([Smart, 2014](#)).

For the bank account example, a *feature file* for **transferring money to a saving account** appear as follows:

The last main activities in BDD is writing executable specifications, which involves defining test code for each scenario to verify that the application meets specific business requirements. A key principle of this step is the use of a shared vocabulary, ensuring that both developers and business stakeholders define examples using the same terminology. This common language minimizes misunderstandings and aligns expectations across teams. An executable specification is an automated test that verify if the application delivers a specific BR and runs whenever a change is made in application . On this way, the automated test in BDD servers as a AC and regression test ([Smart, 2014](#)).

Feature and Scenario - Bank Account online solution

Feature: Transferring money to a saving account

As a bank client

In order to manage my money more efficiently

I want to transfer funds between my accounts whenever I need to.

Scenario: Transferring money to a saving account

Given my current balance in my chequing account is \$2000.00

And my saving account has a balance of \$500.00

When I transfer \$1000.00 from my chequing account to my savings

Then I should have \$1000.00 in my chequing account

And I should have \$1500.00 in my saving account

As in this thesis the BDD tool used was *pytestbdd* (Pidsadnyi & Bubenkov, 2022), the executable specification for the scenario *Transferring money to a saving account* appears as follow:

```
@given(parsers.cfparse('my current balance in my chequing account  
is "{balance:Number}" ' ,  
    extra_types=dict(Number=float)))  
def setupInitialAccount(balance):  
    createAccount(balance)
```

Listing 2.1 : An example for the executable specification for Bank Account Application.

When *pytestbdd* runs the scenario, it will execute each step of scenario by finding the method associate with the step. As it is possible to see, the executable specifications is close to natural language, the key aspect here is about communication and validation. The test reports generated from automated BDD tests enhance communication by providing clear, objective

feedback on the system's implementation status and potential issues, fostering transparency and collaboration among all stakeholders. To summarize, with the use of common vocabulary, BDD improves the knowledge sharing, coordination, and collaboration between teams ([Irshad et al., 2021](#)).

Futhermore, BDD also reduce waste and costs in software development by improving communication and cutting down on rework. As everyone (developers, testers, and business analysts) talks early about requirements using a clear language like Gherkin, they can avoid misunderstandings before starting the actual work. It leads to fewer changes later in the process ([Smart, 2014](#)).

Beyond reduces waste and costs , BDD also enhance the safety of software changes. As scenarios serve as executable specifications that double as automated tests, ensuring new code changes do not change existing functionality through frequent runs, providing regression safety ([Wynne & Hellesoy, 2012](#)).

By keeping scenarios up to date and reflective of the expected system behavior, teams can evolve software with confidence, making developers more willing to update legacy code, which fosters innovation and continuous improvement—critical in industrial robotic systems.

Another benefits of BDD is related to faster releases. By aligning development, testing, and business validation from the beginning. By using a clear, shared language for BDD scenarios, teams reduce time spent on clarifying requirements and fixing miscommunications. These scenarios also function as automated acceptance tests . With aligned requirements, documentation, and testing, teams lower the manual effort needed for regression testing and release preparation ([Chelimsky et al., 2010](#)). It makes frequent and safe releases more achievable, even in complex systems like IRS.

One of the main challenges of using BDD is the need for strong involvement from business stakeholders throughout the development process. This collaboration is crucial for the application of BDD, but it can be difficult to maintain. Stakeholders might not always be available, or they may not have the time, technical knowledge, or interest to help write and review scenarios (Smart, 2014).

The need for strong collaboration in BDD not only presents a challenge but also highlights that BDD is a detailed process with key features and practices. According to Solis Pineda & Wang (2011), it is possible to summarize the six characteristics of BDD: (i) the use of ubiquitous language based on the business terminologies; (ii) iterative decomposition process for the high-level specifications; (iii) templates to write user stories and scenarios, (iv) automated acceptance tests; (v) readable specification code; and (vi) elaboration of behaviors based on the needs of the development phase.

In complex systems such as IRS, fully using the BDD process can often be impractical due to time limits, the availability of stakeholders, or the technical complexity of tasks. Therefore, this study focuses on specific parts of BDD that fit best with IRS, such as automated AT and clear scenario descriptions. This approach aims to improve clarity and support automation without relying completely on constant business involvement.

In conclusion, BDD provides a strong framework for aligning development efforts with business objectives through the use of clear, automated specifications. To further enhance confidence in the system's reliability, it is essential to take an additional step: evaluating the test suite's capacity to identify unexpected or faulty behavior. This is where mutation testing plays a crucial role in this study, as it assesses the effectiveness of the test suite created through AAT4IRS. The next section will explain how introducing small changes (mutations) into the code can improve system reliability.

2.2.5 MUTATION TESTING

Mutation testing is a software testing technique in which faults are introduced manually or automatically generating code variants. Usually, the variants result from applying an mutant operator to the code ([Andrews *et al.* \(2006\)](#)). These modifications include changing a mathematical operator, swapping the order of two statements, or replacing a conditional to the opposite.

For example, Figure 2.2 shows a simple pseudocode snippet for a function takes two numbers, a and b, and returns the larger of the two.

```
def findMaximum(a,b):  
    if a>b then  
        return a  
    else  
        return b
```

Listing 2.2 : Original Code.

In mutation testing, a possible mutant for this code may include a small, deliberate change in the code (Figure 2.3)

```
def findMaximum(a,b):  
    if a<b then //Mutant: Changed 'a > b' to 'a < b'  
        return a  
    else  
        return b
```

Listing 2.3 : Mutant Code.

Mutant analysis in software testing involves creating modified versions of a program, known as mutants, to evaluate the effectiveness of test cases. When a test case is executed

against a mutant and fails, the mutant is deemed "killed," indicating that the test case has successfully identified an error introduced by the mutation. According to [Jia & Harman \(2011\)](#), mutation testing can measure the effectiveness of a test suite in terms of its ability to detect faults. ([ISO/IEC 25010, 2011](#)) defines “effectiveness” as the “... accuracy and completeness with which users achieve specified goals” ([ISO/IEC 25010 \(2011\)](#)).

Mutation testing has gained recognition as a reliable measure of test suite quality. Research by [Papadakis *et al.* \(2018\)](#) *et al.* illustrates that high mutation scores are indicative of a test suite’s ability to uncover a wide array of faults. Similarly, [Jia & Harman \(2011\)](#) offer empirical evidence linking mutation scores to the identification of real-world software defects. A high mutation score signifies that the test suite is proficient at detecting faults introduced into the code. The primary objective of mutation analysis is to achieve the mutation score to 1 (or 100%), demonstrating that the test set is adequate for identifying all faults represented by the mutants. While there is no universally accepted threshold, the software testing literature typically considers a mutation scores exceeding 80% as indicative of good test quality, with scores above 90% to reflect exceptional fault detection capabilities ([Papadakis *et al.*, 2018](#); [Jia & Harman, 2011](#); [Untch *et al.*, 1993a,b](#)).

The mutation score can be used to evaluate how effective a test suite is at detecting faults. A higher mutation score indicates stronger fault detection capabilities and has been shown to significantly improve the overall ability to detect faults earlier. As a result, systems that are tested using mutation-based techniques demonstrate significantly improved reliability. According to IEEE Standard 610.12-1990 ([IEE, 1990](#)), software reliability is defined as “the probability of failure-free software operation for a specified period of time in a specified environment.” In this context, faults are the underlying causes of failures. Therefore, the more effectively a test suite identifies and eliminates faults before deployment, the less likely the system is to experience failures in operation, and the more reliable the software becomes.

This aspect is particularly beneficial in safety-critical and mission-driven fields, such as IRS, where identifying and addressing potential issues before they escalate can prevent serious consequences. By adopting these rigorous testing methods, we can ensure that systems function at their best, safeguarding both efficiency and safety in high-stakes environments.

CHAPTER III

PRELIMINARY WORK

This chapter outlines the preliminary work that laid the foundation for this thesis. The first section presents a systematic mapping study conducted to identify, classify, and evaluate the application of software engineering practices within the robotics domain. For this analysis, we utilized the Software Engineering Body of Knowledge (SWEBOK) as our reference framework. This study offers a comprehensive overview of current practices, challenges, and gaps at the intersection of software engineering and robotics. During our review, we identified a notable gap on software testing in robotic systems. This finding leads us to do a more detailed exploratory review of both primary and secondary studies related to testing practices in this area. Together, these sections provide valuable insights that support and contextualize the proposal for AAT4IRS which represents the primary contribution of this thesis.

3.1 SOFTWARE ENGINEERING FOR ROBOTIC SYSTEMS: A SYSTEMATIC MAPPING STUDY

In the last twenty years, there has been a growing trend of applying SE practices to robotics. This chapter describes a comprehensive study that aims to categorize and assess the current state of Software Engineering for Robotic Systems (SERS) .

Recent systematic mappings address software engineering areas in the RS in different SE areas as architecture. [Ahmad & Babar \(2016\)](#), requirements engineering, [Albuquerque et al. \(2017\)](#), safety for mobile robotic systems and [Bozhinoski et al. \(2019\)](#). Besides, there are surveys as the study performed by [Malavolta et al. \(2020\)](#) to elicit evidence-based architectural

guidelines for open-source ROS-based software for robots, and [García *et al.* \(2020\)](#) that performs a study focusing on software engineering in a service robotics perspective.

However, those previous studies neither address SE in general for robotic systems nor SWEBOK areas. Although conventional and robotic systems are different, most studies about software engineering and robotics apply some SE techniques described in SWEBOK. For this reason, we decided to use it as the base for our classification framework.

The aim for this study is to identify, classify and evaluate the software engineering systematic areas used in the robotic domain based on the SWEBOK guide. To do this, we meticulously examined 50 primary studies from the Scopus digital library as well as the two editions of the RoSE workshop, which focuses on software engineering in robotics and is hosted at ICSE, one of the largest software engineering conferences. The insights presented in this chapter provide valuable knowledge on the challenges and solutions in robotic software engineering, which in turn will support the development and enhancement of the AAT4IRS proposal.

In the forthcoming sections, we will present the research questions addressed in our Systematic Mapping (SM) study, the search strategy used to identify relevant studies, the process of data extraction and analysis, the findings in response to our research questions (SM results), and ultimately, a discussion of our discoveries and our concluding remarks.

3.1.1 RESEARCH QUESTIONS

We refined our research goal into three research questions:

RQ1: What are the demographics of publication of research studies about SE areas applied in the robotics domain? By answering this research question, we aim to understand

how the research area is studied and relevant venues where researchers published their results on the topic.

RQ2: What are the SWEBOK areas mentioned in the robotics domain? We studied the SWEBOK areas applied in robotics domains, including the 15 SWEBOK areas defined in the guideline [Bourque *et al.* \(2014\)](#).

RQ3: What are the main findings of SERS? We aim to provide information to help researchers with evidence-based indications about the directions related to software engineering practices.

3.1.2 SEARCH STRATEGY

In our study, we performed a hybrid search strategy proposed by [Mourão *et al.* \(2020\)](#) that combine the search in a digital library and a parallel snowballing forward and backward. A forward snowballing refers to identifying new papers based on papers that cite the paper examined, and backward snowballing is the process to evaluate the references for the paper in the set ([Wohlin, 2014](#)).

Figure 3.1 shows an overview of the search strategy and study selection that we performed in our study.

To conduct the automated search, we created and run a search query following the guidelines recommended by [Kitchenham & Brereton \(2013\)](#). Our search query is described next.

```
((robot*) AND ("software engineering"))
```

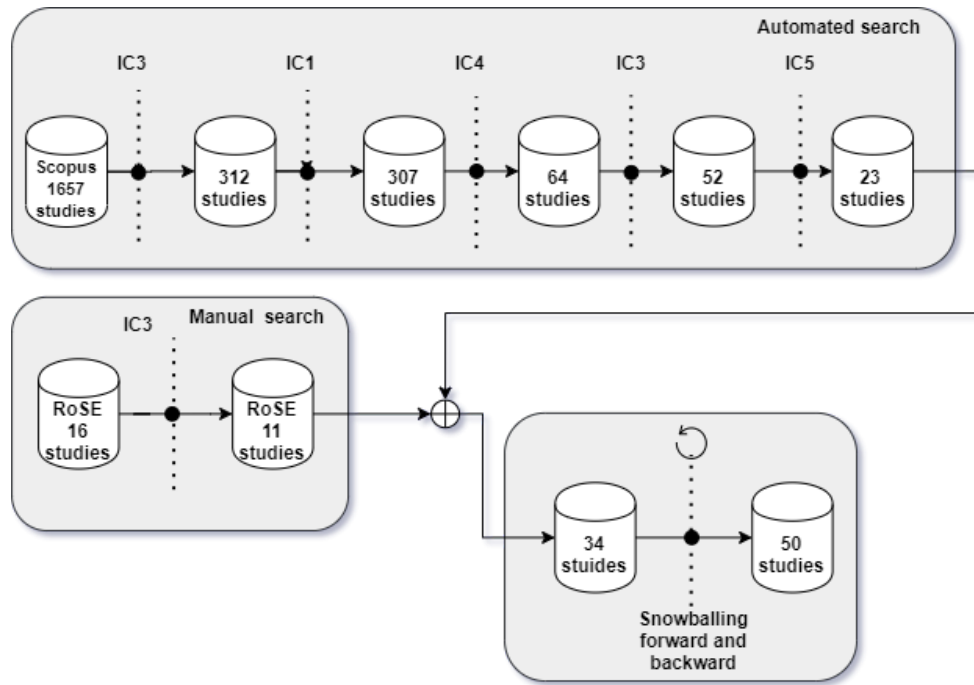


Figure 3.1 : Overview of the search strategy and study selection processes

We defined the inclusion and exclusion criteria during the protocol definition to obtain a set of relevant studies to answer our RQs. We include a study if it satisfies all inclusion criteria and will discard it if the study met any exclusion criteria.

Inclusion criteria (IC)

- **IC1:** The study must be a primary study;
- **IC2:** The paper must be an article or conference paper;
- **IC3:** The study mentions software engineering topic in robotic domain;
- **IC4:** The study was published in a venues that is ranked as A*, A and B for CORE Conference Ranking ([Education, 2021](#));
- **IC5:** The study must be a full paper.

Exclusion criteria (EC)

- **EC1:** The study published as an abstract;
- **EC2:** The study is not written in English;
- **EC3:** The study is a short paper, keynote, tutorial, challenge, and showcase;
- **EC4:** The study does not mention software engineering topic in robotic domain

A total of 1657 studies were turned from the automated search execution. We removed conference announcements and applied the selection criteria (IC and EC) on titles and abstracts, leading to 23 candidate studies. Next, we combined the results of a manual search performed in the last editions of the International Workshop on Robotics Software Engineering (RoSE) after applying IC3, remaining in a single list with 34 studies. Finally, we performed forward and backward snowballing in the set of paper and with the results and applied exclusion and inclusion criteria on title and abstract leading to 16 studies that were added to the seed set of 34 studies. Remaining a initial final set of studies with 50 papers. The studies included in the review are listed in Table 3.1.

3.1.3 DATA EXTRACTION

To extracted data from each primary study, we create a data extraction form. This form was created as a spreadsheet with specific fields to support our data extraction activity.

First, we collected keywords and concepts through the full reading of the papers and applied the classification in a primary set of 10 papers. When the researchers found some divergence, they discussed, evaluated and refined the framework. The process ended when there were no primary studies to analyze left.

The classification framework is composed of three different categories, one for each research question.

Publications Demographics (RQ1). The parameters that we use collect data about publications trends are: publication type and publication venue.

Software Engineering areas in Robotics Domain (RQ2). To answer research question 1 (RQ2), we used the software engineering body of knowledge (SWEBOK). We are using *SWEBOK Guideline* version 3 that is composed of 15 knowledge areas.

Main findings of RSE (RQ3). We highlight the results and conclusions of all studies evaluated. After that, we created clusters to conclude that they have similarities and described important aspects of robotics software engineering. We also eliminated the specific findings for one evaluated study because, in this case, the finding is not a global finding related to robotics software engineering.

3.1.4 DATA SYNTHESIS

We summarize the extracted data with a combination of content analysis and narrative synthesis. We did not identify inconsistencies within the data. We present the data synthesis and our findings in Section 5.3 with a catalogue offering research studies about the utilization of software engineering in the robotic domain and in what SWEBOK Knowledge Area the study is concentrated (Table 3.1).

Table 3.1 : List of primary studies

ID	Reference	Title	SWEBOK Area
P2	(Bäumel & Hirzinger, 2006)	Agile Robot Development (aRD): A pragmatic approach to robotic software.	SC

P7	(Rickert & Gaschler, 2017)	Robotics library: An object-oriented approach to robot applications.	SC
P9	(Zhong & DeLoach, 2011)	Runtime models for automatic reorganization of multi-robot systems.	SC
P14	(Jawawi <i>et al.</i> , 2007)	A component-oriented programming for embedded mobile robot software.	SC
P15	(Valero-Gomez <i>et al.</i> , 2013)	A new paradigm for open robotics research and education with the C++ OOML.	SC
P24	(Shepherd <i>et al.</i> , 2019)	Visualizing the Hidden “Variables in Robot Programs”.	SC
P25	(Gerasimou <i>et al.</i> , 2019)	Towards Systematic Engineering of Collaborative Heterogeneous Robotic Systems.	SC
P26	(Ritschel <i>et al.</i> , 2019)	Novice-Friendly Multi-Armed Robotics Programming.	SC
P29	(Maoz & Ringert, 2018)	On the Software Engineering Challenges of Applying Reactive Synthesis to Robotics.	SC
P32	(Beltrame <i>et al.</i> , 2018)	Engineering Safety in Swarm Robotics.	SC
P38	(Kudelski <i>et al.</i> , 2013)	RoboNetSim: An integrated framework for multi-robot and network simulation.	SC

P40	(Silva <i>et al.</i> , 2014)	Development of a flexible language for mission description for multi-robot missions.	SC
P45	(Weintrop <i>et al.</i> , 2018)	Evaluating CoBlox: A comparative study of robotics programming environments for adult novices.	SC
P46	(Nordmann <i>et al.</i> , 2015)	Modeling of Movement Control Architectures based on Motion Primitives using Domain-Specific Languages	SC
P49	(Alexandrova <i>et al.</i> , 2015)	RoboFlow: A flow-based visual programming language for mobile manipulation tasks	SC
P18	(Sanfilippo <i>et al.</i> , 2015)	Controlling Kuka industrial robots.	SC, SD
P33	(Ore <i>et al.</i> , 2018)	Towards Code-Aware Robotic Simulation: Vision Paper.	SC, SD
P35	(Álvarez <i>et al.</i> , 2006)	An architectural framework for modeling teleoperated service robots.	SC, SD
P37	(Han <i>et al.</i> , 2001)	Internet control architecture for internet-based personal robot.	SC, SD
P41	(Peng <i>et al.</i> , 2016)	EmSBot A modular framework supporting the development of swarm robotics applications.	SC, SD

P42	(Sánchez et al., 2011)	Introducing safety requirements traceability support in model-driven development of robotic applications.	SC, SD
P44	(Sauppé & Mutlu, 2014)	Design patterns for exploring and prototyping human-robot interactions.	SC, SD
P20	(Brugali & Hochgeschwender, 2018)	Software product line engineering for robotic perception systems.	SC, SMM
P22	(Ahn et al., 2012)	A framework-based approach for fault-tolerant service robots.	SC, SD, ST
P1	(Ramaswamy et al., 2014)	SafeRobots: A model-driven Framework for developing Robotic Systems.	SC, SD, SMM
P31	(Bozhinoski & Birattari, 2018)	Designing Control Software for Robot Swarms: Software Engineering for the Development of Automatic Design Methods.	SC, SD, SMM
P34	(Ernst et al., 2018)	Towards Rapid Composition with Confidence in Robotics Software.	SC, SQ, ST
P21	(Molina et al., 2020)	Building the executive system of autonomous aerial robots using the Aerostack open-source framework.	SC, SQ, SD

P3	(Flückiger & Utz, 2014)	Service oriented robotic architecture for space robotics: Design, testing, and lessons learned.	SD
P6	(Trivino <i>et al.</i> , 2009)	Towards an architecture for semiautonomous robot telecontrol systems.	SD
P8	(Kim <i>et al.</i> , 2006)	UML-based service robot software development: A case study.	SD
P11	(Messina <i>et al.</i> , 1999)	Component specifications for robotics integration.	SD
P17	(Minseong <i>et al.</i> , 2009)	Service robot for the elderly: Software development with the COMET/UML method.	SD
P27	(Witte & Tichy, 2018)	Checking Consistency of Robot Software Architectures in ROS.	SD
P43	(Pyo <i>et al.</i> , 2015)	Service robot system with an informationally structured environment.	SD
P47	(Kim <i>et al.</i> , 2005)	Re-engineering software architecture of home service robots: A case study	SD
P36	(Michael <i>et al.</i> , 2008)	Experimental Testbed for Large Multirobot Teams: Verification and Validation.	SD

P4	(Silva <i>et al.</i> , 2012)	Designing a meta-model for a generic robotic agent system using Gaia methodology.	SD, SMM
P5	(Akiki <i>et al.</i> , 2020)	EUD-MARS: End-user development of model-driven adaptive robotics software systems.	SD, SMM
P10	(Paraschos <i>et al.</i> , 2012)	Model-driven behavior specification for robotic teams.	SD, SMM
P13	(Coevoet <i>et al.</i> , 2017)	Software toolkit for modeling, simulation, and control of soft robots.	SD, SMM
P16	(Iborra <i>et al.</i> , 2009)	Design of service robots: Experiences using software engineering.	SD, SMM
P19	(Ortiz <i>et al.</i> , 2015)	Model-driven analysis and design for software development of autonomous underwater vehicles.	SD, SMM
P23	(McKee <i>et al.</i> , 2001)	Object-oriented concepts for modular robotics systems.	SD, SMM
P28	(Bardaro <i>et al.</i> , 2018)	A Use Case in Model-Based Robot Development Using AADL and ROS.	SD, SMM
P30	(BurguEo <i>et al.</i> , 2018)	Using Physical Quantities in Robot Software Models.	SD, SMM
P39	(Cho <i>et al.</i> , 2012)	An interaction-driven approach to identifying functional behaviors of service robot systems.	SD, ST

P12	(Abdelgawad <i>et al.</i> , 2017)	Model-based testing of a real-time adaptive motion planning system.	ST
P50	(Von Mayrhauser & Crawford-Hines, 1993)	Automated testing support for a robot tape library	ST
P48	(Carlson & Murphy, 2003)	Reliability analysis of mobile robots	SQ, ST

SC= Software Construction, **SD**= Software Design, **SMM**= Software Models and Methods, **SQ**= Software Quality, **ST**= Software Testing

3.1.5 SM RESULTS

RQ1: WHAT ARE THE DEMOGRAPHICS OF PUBLICATION OF RESEARCH STUDIES ABOUT SE AREAS APPLIED IN THE ROBOTICS DOMAIN??

We performed a hybrid search strategy that combines automated, manual and snow-balling. The manual search was performed on the Robotics Software Engineering Workshop (RoSE) realized editions (2018 and 2019). For this reason, we have 11 studies from a workshop even though a workshop does not have the Core Rank of at least B.

RoSE workshop studies are in our data set because it is a workshop focused on Robotics Software Engineering. Although the publication years are not a category that we use to answer the RQ1, the distribution in years is valuable to understand the importance of venues as RoSE.

In 2018, which is the year of the first RoSE edition, we have the added of 8 studies, and in 2019 we have in our set of studies only studies were published only in the **RoSE**.

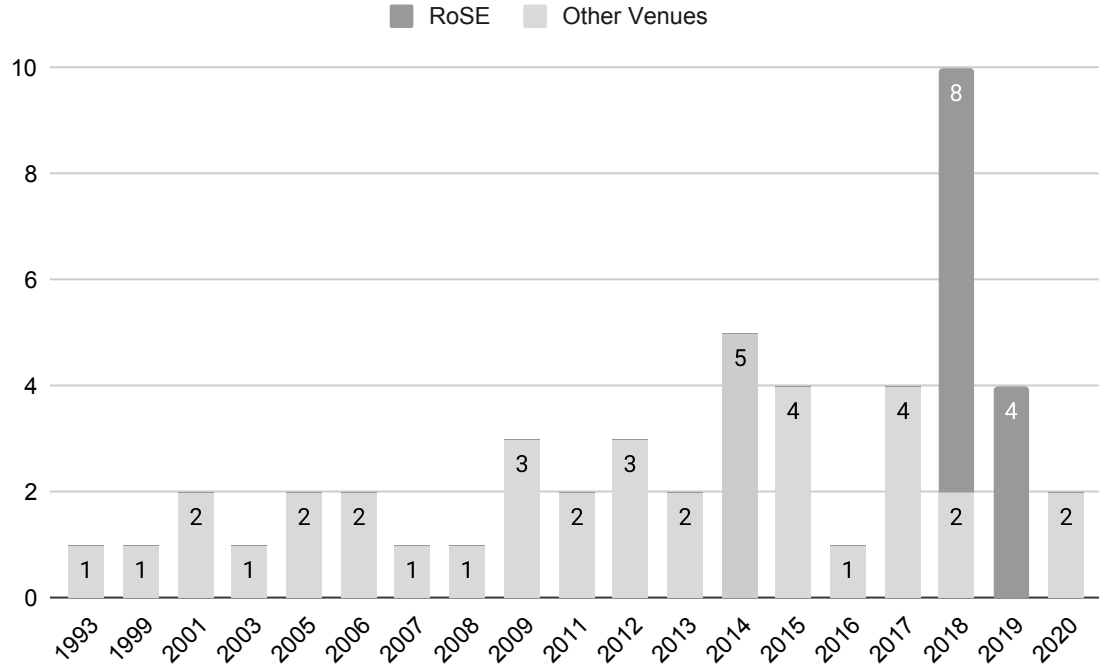


Figure 3.2 : Number of studies of SE technique applied in Robotics Domain per year.

Publications type We classified the primary studies by type (conference or journal paper). The publication type distribution is 26 (11 from RoSE) studies published as a conference paper and 24 in journals. If we evaluated the set of studies without the papers from RoSE, we have 15 studies from conference papers and 24 from journals.

Publications venues The studies evaluated are distributed in 19 different venues (Table 3.2) which research areas are Computer Science and Robotics. We observed a fragmentation in terms of publication venues, 10 of the venues have Robotics as the main research field, and nine are focused on Computer Science, and no venue addresses the intersection between robotics

and software engineering. We observed that the significant number of studies published in one venue was 4, and it happened in journals whose main research field is robotics. Besides, three conferences ranked as A* in the set of venues, and we evaluated six studies published in these conferences.

Table 3.2 : List of Venues

Venue Title	Type	CORE Ranking	Number of studies
Information Sciences	Journal	A	3
Journal of Field Robotics	Journal	A	1
Science of Computer Programming	Journal	A	1
International Journal of Advanced Robotic Systems	Journal	B	4
IEEE Robotics and Automation Magazine	Journal	B	4
Autonomous Robots	Journal	B	3
Advanced Robotics	Journal	B	2
Robotica	Journal	B	2
International Journal of Semantic Computing	Journal	B	1
Robotics and Autonomous Systems	Journal	B	2
International Conference on Software Engineering	Conference	A*	3
Conference on Human Factors in Computing Systems	Conference	A*	2
International Conference on Autonomous Agents and Multi-agent Systems	Conference	A*	1
IEEE International Conference on Intelligent Robots and Systems	Conference	A	3
International Conference on Engineering of Complex Computer Systems	Conference	A	1
International Symposium on Software Reliability Engineering	Conference	A	1
IEEE International Conference on Robotics and Automation	Conference	B	3
Technology of Object-Oriented Languages and Systems	Conference	B	1

Observation 1 - Demographics of publication

There is a distribution in venues with two different research areas, **Computer Science and Robotics**. We observed that RoSE play a critical role to the development for a interdisciplinary research field as Robotics Software Engineering, besides the studies are published in journals more than in conferences.

RQ2: WHAT ARE THE SWEBOK AREAS MENTIONED IN THE ROBOTICS DOMAIN?

We aim with RQ2 to provide an overview for researchers and practitioners about the established software engineering areas studied in robotic domains. As shown in Table 3.3, 5 of 15 SWEBOK knowledge areas appear on the studies evaluated for us.

Table 3.3 : List of included studies for each SWEBOK knowledge areas

Knowledge Area	References	Number of studies
Software Design	P1, P3, P4, P5, P6, P8, P10, P11, P12, P13, P16, P17, P18, P19, P21, P22, P23, P27, P28, P30, P31, P33, P35, P36, P37, P39, P41, P42, P43, P44, P47	31
Software Construction	P1, P2, P6, P7, P9, P14, P15, P18, P20, P21, P22, P24, P25, P26, P29, P31, P32, P33, P34, P35, P38, P40, P41, P42, P44, P45, P46, P49	27
Software Models and Methods	P1, P4, P5, P10, P13, P16, P19, P20, P23, P28, P30, P31	12
Software Testing	P12, P22, P34, P39, P48, P50	6
Software Quality	P21, P34, P48	3

The SWEBOK areas was the framework used to classify the studies. Thus, the SWEBOK is more cited in our set of studies was *Software Design* with 60% of the studies, and the techniques applied in the software design area are UML, model-driven development, component-based software engineering, architecture and description language.

The second SWEBOK area in our rank was *Software Construction* (54%), with most of the studies proposing a framework for solving software problems in the robotics domain.

Software Models and Methods with 24% of the studies is the third SWEBOK area. The SE techniques applied in this area are the meta-model, analysis of models, framework model.

Software Testing (12%) and *Software Quality* (6%) are the two of five SWEBOK areas cited with fewer studies evaluated. The SE techniques investigated are test generation method, code quality standard, requirement analysis and a fault-tolerance architecture.

Observation 2 - SWEBOK areas in Robotics domain

The majority of the studies are concentrated in 3 areas that performed activities inherent to robotics software development (*software design, construction and models and methods*) . Only 18% of the studies performed activities related to testing and quality, and the evaluated studies do not cover 10 SWEBOK areas, as shown in Figure 3.3.

RQ3: WHAT ARE THE MAIN FINDINGS OF SERS?

With the answers for RQ3, we aim to provide information to help researchers with evidence-based indications about the directions related to software engineering practices that are used.

The first finding is related to **the evidence in our set of studies that software engineering practices improve robotic systems quality**. According to [Iborra et al. \(2009\)](#), software engineering contributes to the quality improvement of robotic systems and reduces the effort related to the development. Besides, in the study performed by [Kim et al. \(2006\)](#), it is highlighted that it is impossible to resolve integration and developing issues in robotic

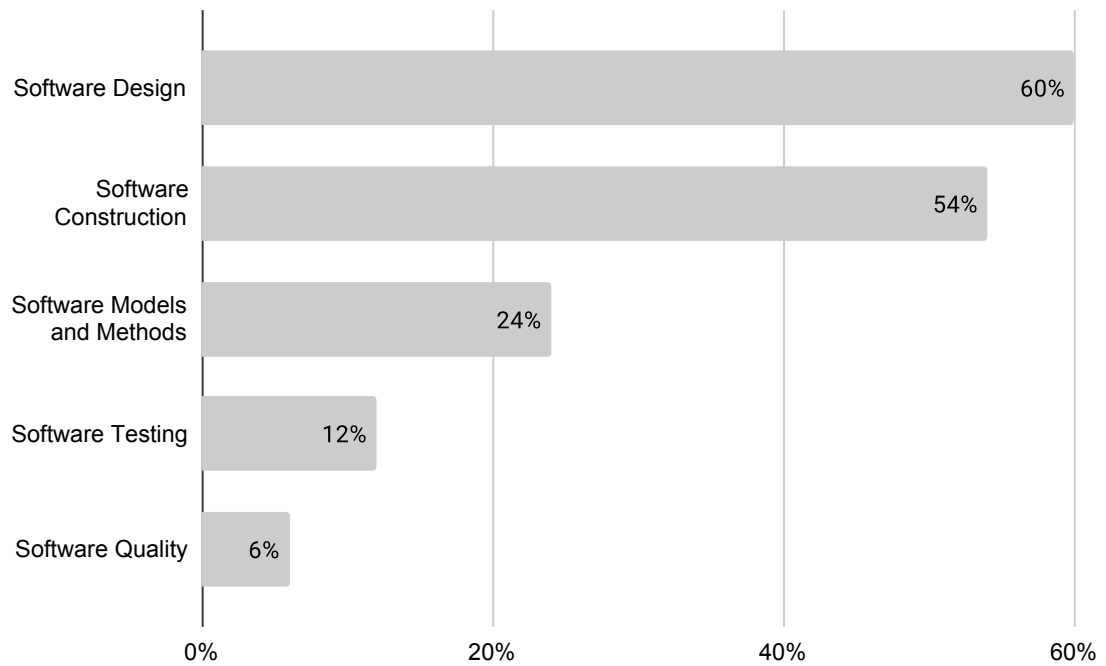


Figure 3.3 : Number of studies and SE technique applied in Robotics Domain.

systems without systematic and comprehensive software development method. Finally, the standard development is described as a key to the usability and sustainability of the robotic assets ([Flückiger & Utz, 2014](#)).

Another important finding is **how software engineering practices can help disseminate development for robotic systems**. There is evidence that applying approaches based on model-driven development can empower not professional programmers to develop robotics software without requiring advanced technical skills ([Akiki *et al.*, 2020](#)). In the study performed by [Gerasimou *et al.* \(2019\)](#), the Domain Specific Language concepts allow both software engineers and robotic experts to write robots software with more facility. Gonzales performed a study that allows engineers to create robotic platforms or other physical designs to exchange and collaborate with other researchers ([Valero-Gomez *et al.*, 2013](#)).

Observation 3 - Main Findings about SERS

There is a consensus by the research community that software engineering techniques and approaches can improve the quality of robot software.

3.1.6 RELATED WORK

Ahmad *et al.* [Ahmad & Babar \(2016\)](#) performed a systematic mapping study on software architectures for robotic systems on a set of 56 primary studies. The authors identified eight themes that support architectural solutions, the progress from object-oriented to component-based, and cloud robotics as an emerging solution. Our study differs from [Ahmad & Babar \(2016\)](#) in the following terms: (i) we apply a more general search process by considering studies related to all SE areas, so we use a search string with the expression "software engineering" instead of using specific terms, for example, architecture, framework or development; (ii) we use a known guideline, *SWEBOK guide* [Bourque et al. \(2014\)](#), as the framework to classify the studies.

In [Bozhinoski et al. \(2019\)](#), Bozhinoski *et al.* performed a systematic mapping study on safety for mobile robotic systems with a software engineering perspective. They analyzed 58 primary studies. Their study focuses on existing solutions that use software engineering methods or techniques to manage mobile robotic systems' safety. Our study does not analyse solutions in a specific type of robotic system. For this reason, we use the term `robotics*` in our search strategy. Furthermore, our study aims to analyze and identify all the studies that apply *Software Quality* techniques, as software quality fundamentals, software quality management processes, practical consideration, software quality tools [Fairley et al. \(2014\)](#).

Malavolta *et al.* [Malavolta et al. \(2020\)](#) performed a study to elicit evidence-based architectural guidelines for open-source ROS-based software for robots. They constructed a

data-set of 335 GitHub repositories containing real ROS-based systems and an online survey. Their study plays an important role in the mapping of Software Engineering for Robotics, given the increase of the ROS utilization by roboticists and architecture robotic researchers. Our study differs from their study because (i) we apply an empirical method, i.e., systematic mapping; (ii) we do not focus on a specific middleware (ROS).

[García et al. \(2020\)](#) have reported a survey study of robotics software engineering from a service robotics domains perspective. The survey evaluated the robotics system's state through 18 semi-structured interviews with industrial practitioners and a survey with 156 respondents. The authors provide a comprehensive picture of software engineering practices in the robotics domain, specific characteristics of robotics software engineering and the usual challenges with adopted solutions. The differences between them study and our are: (i) we apply a different methodology in our study, i.e., we apply a systematic mapping using the guideline proposed by [Kitchenham & Charters \(2007\)](#); (ii) we do not focus on a type of robots; (iii) our study has data from academic publications.

We considerate our study and the studies performed by [Malavolta et al. \(2020\)](#) , and by [García et al. \(2020\)](#) as complementary studies about robotics software engineering.

3.1.7 DISCUSSIONS

In this study, we evaluated Robotics Software Engineering to elicit how the SE areas are applied in the robotic domains and the gaps/opportunities for future work in the intersection of these two research fields.

We highlight two aspects of the study's demographics. First, we observed a **distribution related to the publications venues** (Table 3.2), the evaluated studies are published in 19 different venues. The studies are published in venues where the main research field is Robotics

(9) or Computer Science (9), further the workshop RoSE. Second, most journals focus on robotics, and most conferences focus on computer science. The venues ranked in the highest level are three conferences: International Conference on Software Engineering Conference, Conference on Human Factors in Computing Systems Conference, International Conference on Autonomous Agents and Multi-agent Systems Conference.

These facts lead us to conclude that there is not a concentration in a unique venue or area. Moreover, through observation, a dissonance between the methodology applied in the evaluated studies published in robotic venues concerned software engineering techniques and approaches. For example, in the studies that use model-driven development, 80% do not have empirical study as is trivial in a software engineering study.

In the context of SE Areas, we observed that the evaluated studies cited 5/15 SWEBOK Knowledge Areas: *Software Design*, *Software Construction*, *Software Models and Methods*, *Software Testing* and *Software Quality*. The top 3 areas mentioned are related to the Design and Implementation phase of the traditional approach of system development: *Software Design*, *Software Construction* and *Software Models & Methods*. However, 10/15 SE areas are not mentioned, and 5 of these studies are deeply researched by the software engineering community: *Software Requirements*, *Software Maintenance*, *Software Configuration Management*, *Software Engineering Management*, *Software Engineering Process*. We believe that there are gaps and research opportunities in these 5 SE areas for robotic domains.

The studies that cited topics related to the *Software Models and Methods* area also applied some topics related to the Software Design area (11/29). Besides, only five studies (P5, P10, P19, P20, P28) in *Software Models and Methods* declare they are working with Model-driven development (MDD). However, 80% of the studies related to MDD have evaluation issues. For instance, in the study performed by [Iborra et al. \(2009\)](#), it is unclear how the

authors evaluated the part of the proposal related to MDD. This observation suggests a **lack of empirical evaluation in the projects that apply MDD.**

Software construction is the second area more cited in the evaluated studies. According to the SWEBOK (Bourque *et al.*, 2014), software construction is a SE area related to coding, verification, unit testing, integration testing, and debugging. However, in the studies evaluated, the technique more applied is software construction tools with a focus on coding as the studies performed Rickert & Gaschler (2017) that describe a library approach named Robotics Library (RL), and by Álvarez *et al.* (2006) which purpose is presenting a reference architectural framework for service robot control applications.

Software testing was cited only in 12% of the studies. This result corroborated with other studies, for example, in the survey about testing robotic system performed by Afzal *et al.* (2020) . In this study, the author mentioned that robots interact with a physical component, which is a challenge for testing robotic systems.

Besides, only 6% of the studies evaluated are concerned *Software Quality*. We highlight that for our classification, it was important not only to cite quality requirements but also to develop topics related to *Software Quality* topics such as software quality fundamentals, software quality management processes, practical considerations, and software quality tools (Fairley *et al.*, 2014).

Threats to validity usually happen in a mapping study, and it was not different in our study. We highlight some of those threats and the mechanism that we applied to address them.

First, the main limitation is that our search string is restrictive concerning software engineering terms. This choice could lead us to primary studies that are not representative of state-of-the-art robotics software engineering. We mitigate this threat by applying a hybrid

search strategy proposed by [Mourão *et al.* \(2020\)](#); [Wohlin \(2014\)](#), with automated and manual search in the first stage and backward-forward snowballing.

Second, we considered only peer-reviewed studies published in venues with CORE Conference Ranking at least B ([Education, 2021](#)); this potential bias did not impact our study since the studies evaluated were reviewed in a rigorous process, which is a requirement for high-quality.

Another limitation of this work is the established categories. We could identify other categories if we decided, for example, to classify the approaches and techniques. However, we opted to follow the SWEBOK guideline because it is a known guideline used in SE. Besides, we could answer other RQs and provide a broad overview of SE for robotic systems.

3.2 SOFTWARE TESTING FOR ROBOTIC SYSTEMS

The systematic mapping presented in this thesis focuses mainly on software engineering and robotic systems. However, this section shifts the focus to software testing in the context of robotics. This happened after we analyzed the results from the mapping and noticed a clear gap: there are not many studies that directly connect software testing practices with robotic systems. Because of this, we decided to do a more focused and careful search for primary and secondary studies that deal with both topics. Even though we did not find enough material to build another full systematic mapping, this analysis still helped us understand better the challenges and research gaps in this area. These insights are important and helped guide the contributions made in this thesis.

3.2.1 PRIMARY STUDIES

[Hunecke *et al.* \(2025\)](#) made an important contribution to robotic acceptance testing by expanding BDD for testing object-sorting tasks with robotic arms. They created a specific language called RobBDD and used NVIDIA IsaacSim simulations to test complex pick-and-place tasks, like sorting colored cubes from a moving conveyor belt. Although their work and this thesis both aim to check robotic behavior in real-world situations, this thesis focuses more on test reliability and uses mutation analysis to evaluate how effective the tests are. Researchers showed that BDD scenarios can work in simulations, but their evaluation is mostly qualitative and does not include measurements of how well the tests can find faults. In contrast, this thesis uses mutation-based methods to measure and improve test quality, providing a clearer and more reliable way to test industrial robotic systems.

The study performed by [Nguyen *et al.* \(2023\)](#), the authors analyze robotic application requirements and acceptance criteria, explicitly focusing on robotic competitions and benchmarks. They aim with the study to address the challenges of representing and managing requirements in the context of robotic applications' increasing complexity and diversity. We consider this study as a complementary study. In both studies, the authors applied BDD to rewrite the requirements for industrial robotic systems. However, in the study presented, the authors highlight the application of Behaviour-Driven Development (BDD) as a means to express and manage requirements for robotic applications, emphasizing the potential for introducing automation into verifying and validating these requirements in robotics. In our study, we used BDD to rewrite the requirements for industrial robotic systems and implement executable tests using a simulator.

[Ashraf *et al.* \(2020\)](#) proposed coverage criteria for white-box testing to test industrial robot tasks and a framework to generate the test cases automatically to achieve the coverage

criteria defined by them. Differently, our research aims to automate software testing process but not the generation of test cases.

[Breitenhuber \(2020\)](#) propose an application-level testing framework for robot software applications that uses a known robotic software to describe the expected behavior of an application or its components. They focus on evaluating the component behavior in robotics systems that use the ROS framework in the application level testing. Besides, they apply the testing tools available in the ROS environment to test the components. In our study we aim to apply automated acceptance for industrial robotic systems in general, not only in ROS-based systems. Furthermore, in our approach, acceptance testing is automated and the business requirement is translated to the BDD template for acceptance criteria.

[Erich *et al.* \(2019\)](#) presented a framework for automatically testing applications for collaborative robots and demonstrated the proposal in a case study for automated testing of a pick and place application. The authors' proposal is a framework applied in a physical environment and at the level of Integration Testing. Our framework focus on acceptance testing, which lead us to consider the [Erich *et al.* \(2019\)](#) research and ours as complementary studies.

In the study performed by [Estivill-Castro *et al.* \(2018\)](#), the authors presented a proposal for a robot simulator following the Model-View-Controller software pattern. They use simulators with the stripped-GUI under a continuous integration paradigm for robots to scale up the testing integration with robot behaviour. In our research, the simulator will be part of the framework to improve reliability.

[Mossige *et al.* \(2015\)](#), in their study presented cost-effective automated testing techniques to validate complex industrial robot control systems in an industrial context and employ their methodology in continuous integration and constraint-based testing techniques. Although our

research also focuses on industrial robotic systems, we focus on improving a specific quality aspect, robustness.

[Alexander *et al.* \(2015\)](#) introduced the concept of situation coverage, which they evaluate through tests on a simulated autonomous road vehicle, comparing its effectiveness to random test generation. They note that the challenges of testing autonomous robots are similar to those faced in industrial robot testing, making our studies complementary. Situation coverage measures the proportion of possible situations tested by a given set, defined by starting states and rules for projecting future states, without a linear sequence. In contrast, our approach uses features and scenarios in natural language to guide test generation, seeking to cover representative scenarios through linear sequences. Both methodologies aim to enhance robotics software through effective testing, despite their differences.

In the study performed by [Bretl & Lall \(2008\)](#), the authors presented an algorithm to help to calculate the mass center of legged robots to achieve static equilibrium. The study does not explicitly use software testing techniques. However, the authors test the robot's behavior, which makes us consider their study as our related work. Our research aims to apply acceptance testing to improve robustness in industrial robotic systems.

In the study performed by [Yun Koo Chung & Sun-Myung Hwang \(2007\)](#), the authors presented a testing process and evaluation items for software testing of intelligent robots. They proposed a test case design methodology based on user requirements and ISO standards for software testing.

3.2.2 SECONDARY STUDIES

[Afzal *et al.* \(2020\)](#) presented a series of qualitative interviews with 12 robotics practitioners from 11 robotics companies and institutions that provide an overview of the challenges

of testing robots. In this research, the authors presented an overview of 12 testing practices: field robotics, logging and playback, simulation testing, plan-based testing, compliance testing, unit testing, performance testing, hardware testing, robustness testing, regression testing, and continuous integration and test maintenance. And some challenges, for example, software and hardware integration, distrust of simulation, culture testing. It is essential to highlight that testing practices mentioned can be considered the state-of-the-practice of software testing for robotic systems. This research shows us the state of the practice and gaps in the area.

In the research presented by [Gotlieb *et al.* \(2021\)](#), the authors present techniques for test generation, planning, and execution for industrial robots. They also performed the deployment of this technology into the real-world continuous integration process of a large robot manufacturing company. We can see this research as related work to our research concerning the field (testing industrial robotic systems). However, the decisions about how to construct the testing framework and how to measure the proposal are different.

[Andersson & Carlstedt \(2019\)](#) presented in their master's thesis a method that uses simulations to create a test set for detecting motion anomalies in new robot controller versions. They use auto-generation of test cases with an induced artificial bug, which is one difference between their research and ours. We aim to automate the execution of the tests and not the test generation. Besides, we aim to perform acceptance testing, which focuses on validation and not verification of the system.

The research performed by [Afzal \(2021\)](#) in her doctoral has focused on an automated testing framework using software-in-the-loop (low-fidelity) simulation to facilitate automated testing for cyber-physical systems. The proposal framework works in the automation of the oracle, automated test input generation. Some points that make ours and this research different:

(i) are [Afzal \(2021\)](#) proposed a framework to apply software testing in all cyber-physical systems; (ii) the level of test performed; and (iii) how they evaluate the framework.

[Wang \(2020\)](#) in his dissertation, studied the failures in long-term autonomous service robots and proposed methods to improve their reliability. He designed two reliability engineering methods to improve the robustness of service robots. In this way, his research focuses on a different class of robots compared to our research.

In summary, the analysis of software testing in robotic systems underscores that this intersection remains an evolving area with numerous challenges still to be addressed. The primary studies indicate promising avenues, such as the use of BDD, simulator-based testing, and frameworks specifically designed for industrial environments. However, limitations persist, particularly regarding the measurement of test effectiveness and the scarcity of quantitative evaluations. Our work builds on and expands these concepts by incorporating mutation analysis into the acceptance testing process to evaluate and enhance test reliability in IRS. Additionally, the secondary studies have provided us with valuable insights into the current state of practice in the field. These studies shed light on the testing strategies employed in industry, the common challenges encountered by practitioners, and the existing gaps in test automation and robustness. Collectively, these findings lay the groundwork for our proposed framework, AAT4IRS, which aims to address the shortcomings in automated acceptance testing, with a particular emphasis on fault detection and reliability in industrial robotic systems.

3.3 DISCUSSIONS AND FINAL REMARKS

This chapter has provided two complementary analyses that informed the development of this thesis proposal. First, we conducted a systematic mapping study to examine the

application of SE practices within the robotics domain. Following this, we undertook a more focused analysis to address the identified gap in software testing for robotic systems.

Our systematic mapping revealed that while software engineering concepts are being utilized in robotics, several critical areas of software engineering, including software requirements, quality, and testing, are significantly underrepresented. Most research tends to concentrate on design and construction, often employing tools and models but lacking robust empirical validation. Additionally, we observed that publications are dispersed across various venues, resulting in the absence of a centralized community. Furthermore, studies presented at robotics conferences rarely adopt rigorous software engineering methodologies. This highlights a disconnect between the two communities and underscores the necessity for more structured and validated software practices within robotics projects.

To better understanding the existing gap in software testing for robotic systems, we examined specific research focused on this area. Our review indicated that software testing in robotics is still in its early stages. While some studies propose frameworks and testing tools, particularly those utilizing simulations or automated methods, only a few provide quantitative results pertaining to test quality or reliability. Additionally, only a limited number of works have implemented BDD or acceptance testing within the context of IRS. Our research stands out by integrating BDD with mutation testing, thereby introducing more robust metrics and automation into the acceptance testing process for robotics. This approach addresses a gap that is often overlooked in the majority of existing studies.

By combining both perspectives, we were able to uncover significant research opportunities. These insights underscore the motivation for this thesis, which seeks to enhance the reliability of industrial robotic systems through structured, automated, and measurable acceptance testing techniques.

CHAPTER IV

AAT4IRS SOFTWARE TESTING APPROACH

This chapter presents the structured workflow of the AAT4IRS software testing approach, outlining the stages that guide the testing process from initial setup to execution. Additionally, we will address the common challenges encountered during the testing process, such as the complexity of system requirements, the scalability of test automation, and managing real-world variability.

As a contribution of this chapter, we provide a comprehensive overview of the AAT4IRS software testing approach, outlining its structured workflow essential for ensuring the reliability and performance of industrial robotic systems. Section 4.1 summarizes the workflow, detailing each stage (Stage 1: Getting Started, Stage 2: Write Scenarios, Stage 3: Implement Tests, Stage 4: Execute Tests), while Section 4.2 highlights the significance of well-defined processes in achieving effective testing outcomes.

4.1 WORKFLOW OF THE AAT4IRS

AAT4IRS is a specialized software testing approach that involves customizing and implementing automated acceptance templates based on the principles of BDD. This approach aims to improve fault detection in industrial robotic systems.

When testing software for industrial robotic systems, it is important to take into account the unique features and needs of these systems. According to [Afzal *et al.* \(2020\)](#), robotic systems differ from conventional software in several critical dimensions. Robots are complex systems composed of software and hardware. The hardware interacts with the physical world through sensors and actuators, which can lead to errors that are difficult to predict. Additionally,

defining what is ‘correct’ behavior is often challenging. To ensure the software meets the specific requirements and expectations of the IRS, it is essential to involve domain experts, stakeholders, and end-users in the testing process.

Thus, we propose an approach that addresses the specific challenges of robotic systems by validating whether the software satisfies the BR within the context of industrial robotics. This approach considers the differences between conventional software and robotic systems, ensuring that the behavior aligns with real-world expectations and domain-specific needs

Starting the AAT4IRS process necessitates a precise and clearly articulated set of BR to guarantee that the features developed provide genuine business value. Similar to conventional systems, in the context of IRS, the business requirements must outline the specific business needs or challenges that the software is intended to address.

For instance, an industrial robot could be employed in a supermarket cashier environment to automate the checkout and packing process by placing items into a tote. In this scenario, it is crucial to maintain the exact sequence of scanned items during the packing process, as this order may impact inventory tracking, payment accuracy, or customer preferences. Additionally, the packing process must be swift and reliable to prevent delays at checkout and enhance customer satisfaction.

These business requirements can be translated into test scenarios that assess whether the robot accurately preserves the item sequence and completes the task within the specified time frame. Such scenarios play a vital role in ensuring that the robot’s actions align with the operational objectives established at the business level.

Business Requirement
We need to ensure that the items on the conveyor belt selected by customers are identified by a scanner and dropped quickly in a bag them accordingly to prevent damage.

To implement AAT4IRS, it is essential to enrich the BR with additional details that reflect the unique characteristics of robotic systems. This includes specifying the robot's initial position and outlining acceptable margins for positional error. The requirements should be quantifiable, actionable, and include explicit acceptance criteria to ensure a shared understanding of the system's expected behavior among all stakeholders.

Continuing with the supermarket example, to verify that the robot can pack items in the correct sequence, the BR should also stipulate how precisely the robot must align with the tote, the speed at which the task should be completed, and the allowable level of deviation that will not compromise performance. These detailed criteria facilitate the development of automated acceptance tests that account for both functional expectations and the physical constraints of the environment.

Business Requirement
We need to ensure that the items on the conveyor belt selected by customers are identified by a scanner and dropped in a bag them from less than 35 mm.

The AAT4IRS workflow (Figure 4.1) follows a systematic process, with each stage driven by specific inputs and producing outputs that guide the subsequent steps. This approach ensures the cohesiveness and continuity of the testing process. The key stages of AAT4IRS are as follows: Stage 1 - Write Features, Stage 2 - Write Scenarios, Stage 3 - Implement Tests, and Stage 4 - Execute Tests.

Each stage of the process builds upon the work of the previous one. Stage 1 involves setting up the necessary environment for the subsequent stages. Stage 2 focuses on developing scenarios based on the environment established in Stage 1, which then guide the implementation of tests in Stage 3. Finally, in Stage 4, the tests are executed, producing the expected outcomes based on the groundwork laid in the earlier stages.

With the BR defined, we can take two paths. One starts with the development team *writing the mission* (the application layer defined in Chapter 2), essentially the system under test. The other path starts with the test using the Gherkin language to *write the feature*. So, according to the business decision, we can directly implement the code (mission) or we can apply AAT4IRS by follow the stages.

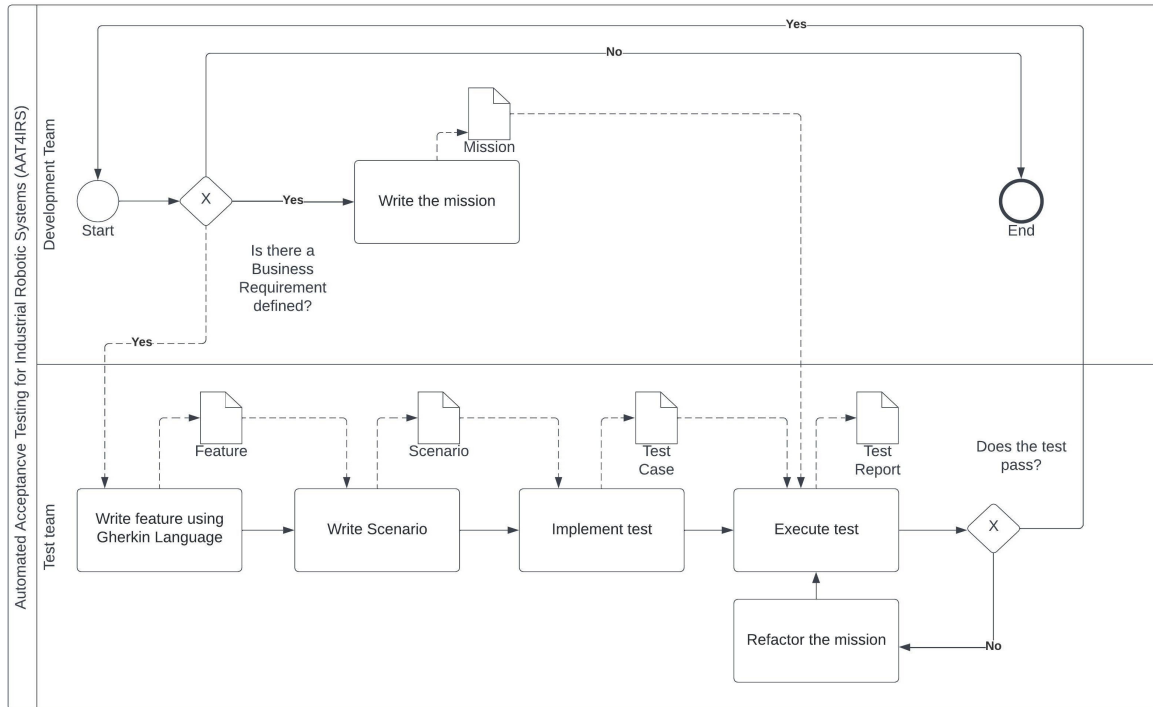


Figure 4.1 : Proposed approach to apply Automated Acceptance Testing for IRS.

4.1.1 STAGE 1 - WRITE FEATURES

A feature is an operational element that contribute to achieving the overall system objective. In an industrial environment that uses an IRS to automate processes, a feature represents a specific functionality to be automated using robotics, for example, pick-and-place operations, automated quality inspections, welding and assembly, real-time performance monitoring, and automated material handling.

Writing features serves as the basis for the AAT4IRS software testing approach because they define the functionality that the system will deliver and ensure alignment with the business goals. In AAT4IRS, the feature adapts the template defined in BDD and will follow this format: As an operator, I want [process to be automated] so that I can automate the [process] using an IRS.

Using the same example from a supermarket that had the BR defined before, we can write the feature Drop off an item using IRS

Feature - Drop off an item using IRS

Feature: Dropped off an item from conveyor belt to a bag

As an operator

I want to grip an item from a conveyor belt to a bag using an IRS

So that I can ensure that the items selected by customers are dropped quickly in a bag quickly from less than 35cm.

The features in AAT4IRS are structured using the BDD format, which follows the user story template. It is important to draw a parallel between the end-user in a traditional system's user story and the operator in an automated process using IRS, particularly in AAT4IRS. In a traditional system, a user story generally entails end-users directly engaging with the software to accomplish a specific objective. This user might be a website visitor, an employee utilizing an internal software tool, or anyone interacting with the system to meet a requirement.

Example (Conventional System User Story): "As a customer, I want to be able to search for products on the website so that I can easily find what I need."

In an automated process using IRS, the operator is the focal point of the user story. Thus, the operator's interaction with the system is crucial for ensuring that the robot's behaviour meets the business requirements.

Example (Robotics User Story): "As an operator, I want to be able to configure and monitor the robotic system so that it performs the task of item packaging on the production line without interruptions."

As shown in Figure 4.1 features represent the output of Stage 1 and serve as the input for Stage 2. Features provide the foundation for creating scenarios, with each scenario outlining specific behaviors the system should exhibit based on the defined feature. It is important to note that the role of features extends into the subsequent stages (scenarios and testing), making their initial definition a critical step. Without well-defined features, the AAT4IRS may lose its direction, leading to incomplete or ineffective testing.

4.1.2 STAGE 2 - WRITE SCENARIOS

Scenarios are examples that illustrate how features are intended to function. They act as acceptance criteria, defining when a feature is considered complete, while also guiding developers in building a system that aligns with business requirements.

In the AAT4IRS approach, scenarios act as a link between high-level features and the detailed testing process. While features define what the system should accomplish, scenarios outline how the system should behave in specific situations, providing a detailed view of its functionality.

Scenarios improve the collaboration among developers, testers, and operators by creating a shared understanding of expected outcomes. By utilizing a common language, such as scenarios template in BDD, teams can achieve alignment, reduce misinterpretation, and enhance communication. This unified language is particularly important in specially within robotics teams where diverse backgrounds can sometimes hinder coordination ([Afzal *et al.*](#)

(2020)). In AAT4IRS, scenarios act as a vital communication channel, bridging the gap between software and hardware teams, clarifying features, and serve a base for testing.

In order to define scenarios for AAT4IRS, it is essential to have a comprehensive understanding of the instrumental definition in industrial robotics. This definition encompasses key components such as sensors, actuators, and power supplies, which play a crucial role in the design and operation of robotic systems. Sensors are particularly important in automated processes using IRS. By integrating sensor readings and error thresholds into scenarios, the reliability for tests that will be implement using the scenarios will be improved. Furthermore, some aspects as the error threshold can be customized to meet business requirements and the instrumentation used.

Following the BDD template (Given-When-Then). Each condition can be classified as a Given, Then or When condition. The *Given* step involves outlining the the precondition environment for industrial processing using robots, which includes robot setup, environment preparation (e.g., conveyor belt setup), and sensor calibration, robot and object initial position. *When* step outlines the system under test. Lastly, in the "Then" step, we evaluate whether the system meets the expected behavior using the AC defined in the first activity. For example, we can use a sensor to evaluate the final position of a box. In the context of a BDD template, multiples conditions with the same type can be connected using the "and" connector.

In the context of defining business requirements and features, consider the example of a supermarket. An essential component in this scenario could be a position sensor capable of measuring each axis, accompanied by an error threshold that aligns with the business requirements. For instance, when looking at the feature *Dropping an item from the conveyor belt into a bag*, a potential scenario might unfold as follows:

Scenario: Dropped off an item in a bag

Given the robot is in initial position

And the conveyor belt is initialized

And the robot is at home with the position error threshold being less than or equal 100mm in the x-axis

And the robot is at home with the position error threshold being less than or equal 100mm in the y-axis

And the robot is at home with the position error threshold being less than or equal 100mm in the z-axis

And the item is in the conveyor with the position error threshold being less than or equal 150mm in the x-axis

And the item is in the conveyor with the position error threshold being less than or equal 150mm in the y-axis

And the item is in the conveyor with the position error threshold being less than or equal 150mm in the z-axis

When I finished the dropped off process

Then The item is in a bag in the same order that was placed on conveyor belt.

Writing clear scenarios is vital for outlining system behavior under various conditions. These scenarios serve as acceptance criteria and guide developers and testers, ensuring comprehensive validation of business requirements. By detailing interactions and expected outcomes, they facilitate effective testing. The next step follow the AAT4IRS workflow is the implementation phase in which the scenarios will be translated into executable tests.

4.1.3 STAGE 3 - IMPLEMENT TESTS

Stage 3 is the implementation of tests in order to turn scenarios into executable tests. Each scenario, as defined during the "Write Scenarios" stage, describes a specific interaction between the system and its environment. As AAT4IRS is based on BDD, the test implemented is the acceptance testing which verify the system behaves as described in scenarios. The acceptance test ensures that the system meets the specified business requirements and features.

By implementing tests, we must write the actual test code for each sentence defined in each scenario. This is typically done using a testing framework (e.g., NUnit, MSTest, or a BDD tool like pytest-BDD). The test code simulates the interactions defined in the scenario and assert that the expected outcomes occur as specified. This simulation can be written to be executed in a simulation environment or in physical robots.

Furthermore, another point of attention in the *Stage 3* is as the test code will be executed in the system under test, it will interact directly with the system. Therefore, the test code and the system under test need to use the same language, or a bridge between them must be implemented. We highlight that this bridge is not in the scope of this research.

For example, the sentence *the robot is at home with the position error threshold being less than or equal 100 mm in the x-axis* described in the scenario, is translated to a function to assert with the position on x-axis is according to the acceptance criteria defined. So the function will assert if the robot achieves the expected position using the reading sensors and also the error threshold defined according to the business requirement. To exemplify, the test code uses as a language *Python* and the library *pytest-bdd* to implement tests.

Scenario: Dropped off an item in a bag

Given the robot is at home with the position error threshold being less than or equal 100 mm in the x-axis.

```

//testDroppedItem.py
...
scenarios('../features/droppedItems.feature')
@given(parsers.cfparse('the robot is at home with the position error
    threshold being less than or equal "{errorThreshold}" in the x-axis
    ', extra_types=dict(Number=float)))
def robot_position_initialX(errorThreshold):
    positionRobotX = readSensor(1)
    assert (expectedRobotX - positionRobotX) <= errorThreshold

```

Listing 4.1 : The statement created in the feature is implemented by a test in the test file.

In the example for the *Given step*, a function called *readSensor* reads the position sensor for the X-axis. The function *robot_position_initialX* then uses this reading to assert whether it falls within the acceptable range, based on the error threshold defined in the acceptance criteria. As illustrated with the *Given* example, each sentence in the scenario will have its corresponding function implemented in the test code to ensure the behavior described is accurately tested.

To summarize, the *Stage 3* requires technical decisions that will directly impact the success of the testing process. Choosing the appropriate testing framework, tools is essential to ensure that the tests are reliable, efficient, and aligned with the overall system's goals. Moreover, this stage serves as the foundation for the next phase, *Execute Tests*, where the implemented tests are run against the system under test.

4.1.4 STAGE 4 - EXECUTE TESTS

It is essential to emphasize that all previous stages must be completed before moving on to Stage 4. In this phase, the tests developed are automated and executed, triggering

functions that interact with the system under evaluation. It is worth noting that AAT4IRS is not a development methodology but rather a software testing approach specifically tailored for industrial robots.

When AAT4IRS is used within a development framework, such as BDD, the initial execution of a test often results in failure. This outcome is expected because, in BDD, tests serve as guidelines for development.

In contrast to the BDD approach, a business decision might be made to develop the application layer first and only test afterward. This means the mission is tested only after the code has been fully developed. If the test is successful, the process moves forward to the next BR. If the test fails, the application layer is refined and adjusted until it passes the test.

Stage 4 of the AAT4IRS outputs a test report, an important communication tool for stakeholders that captures testing outcomes, highlights failures, and tracks progress over time, ensuring transparency and alignment with key business objectives.

This stage builds on the foundation established in Stage 3, where a framework for implementing tests is defined. Here, the focus shifts to creating a robust reporting structure that integrates seamlessly with the test code, enabling the generation of accurate, meaningful reports that reflect the current software status.

Automated testing frameworks like Cucumber, JUnit, pytest-bdd or custom scripts typically generate these reports. Automation provides a distinct advantage, producing reports efficiently with minimal manual input. Test reporting serves not only as documentation of testing results but also as a historical record that can guide future projects. Tracking testing progress over time enables stakeholders to evaluate testing strategies' effectiveness and make informed decisions for upcoming development cycles.

In summary, the test report generated in Stage 4 enhances stakeholder communication, increases transparency, and aligns testing with business goals.

4.2 CHAPTER FINAL REMARKS

Our proposed process shares similarities with applying automated AT to conventional systems. However, crafting features, scenarios, and test cases requires nuanced adaptations that aligns with the demands and objectives of industrial processes. To achieve this, we incorporate industry-specific language when formulating scenarios using the Given-When-Then structure. For example, the Given statement sets the initial conditions, such as the starting position of a robot. Furthermore, we employ instrumentation tailored to industrial settings to establish acceptance criteria. By using positional sensors across three axes, we validate positions and define acceptance criteria according to sensor characteristics, ensuring alignment with business objectives. Ultimately, our approach entails integrating domain-specific language to customize automated acceptance testing within the BDD standards framework.

By following our approach, the software of IR can undergo thorough acceptance testing to ensure that it meets the requirements and expectations of end users. It is important to involve domain experts and stakeholders throughout the AT process to ensure that the software aligns with the specific needs of the industrial robot application.

The software testing process for industrial robotic systems, as outlined in the AAT4IRS framework, is designed to be structured and systematic, ensuring a thorough evaluation from the initial definition of features to the execution of tests. This approach emphasizes the importance of starting with well-defined features and scenarios, which are crucial for aligning testing efforts with overarching business objectives. By establishing clarity at this

stage, the risk of misinterpretation is significantly reduced, laying a strong foundation for the development of relevant and comprehensive test cases.

The workflow further translates these requirements into actionable scenarios that reflect real-world applications. This specificity is vital, as it ensures that each test is focused on critical aspects of the system's functionality, thereby enhancing test coverage and relevance. By detailing scenarios that stem directly from the defined features, the testing process becomes more targeted and effective.

A key component of this testing process is the incorporation of automation. Automation not only accelerates the testing timeline but also facilitates consistent and repeatable evaluations. This efficiency is essential in industrial settings, where time and reliability are paramount.

As tests are executed, they generate valuable feedback loops for both development and testing teams. The insights gained during test execution provide real-time information about system behavior, allowing teams to identify areas for improvement and enhance overall software quality. This iterative feedback mechanism is crucial for refining the system and ensuring it meets the required standards.

In summary, the AAT4IRS framework ensures that all testing activities are strategically aligned with business goals. It provides a robust structure that promotes consistency, reliability, and effectiveness throughout the software testing process for industrial robotic systems. By following this systematic approach, organizations can achieve a higher level of confidence in their robotic systems, ultimately leading to improved performance and operational success.

CHAPTER V

APPLYING AAT4IRS TO PICK-AND-PLACE TASK

We propose a ST approach called AAT4IRS that uses natural language to write the features and scenarios to be tested described in the Chapter 4. In this chapter, we evaluated AAT4IRS through a proof-of-concept, performing a pick-and-place process in simulation. The choice of a single simulated use case is intentional and aligns with the goal of demonstrating the feasibility and initial effectiveness of the approach, rather than providing exhaustive validation across multiple industrial tasks.

The use of industrial robots in pick-and-place scenarios is common occurrence in competitions and benchmarking exercises. Our decision to draw inspiration from a robotic competition is rooted in the limited accessibility of real-world industrial requirements, as noted by [Nguyen *et al.* \(2023\)](#). As such, we looked to the Robotic Grasping and Manipulation Competition’s Task Pool for direction. In particular, we turned to a specific task outlined in their competition framework: “Pick Up and Place Using Tongs” ([Sun *et al.* \(2018\)](#)). We focus on the off-line method, which makes use of simulation. Finally, we applied mutation testing to measure the effectiveness of AAT4IRS.

This chapter is organized as follows: (i) a detailed description of the software and hardware resources essential for performing the case study, (ii) an empirical study, (iii) evaluation of the effectiveness of the proposed approach, (iv) discussion of the evaluation findings, highlighting key observations, challenges encountered, (v) enumeration of potential limitations that may impact the validity of the case study, and (vi) the chapter final remarks.

5.1 TOOLS, TECHNOLOGIES, AND LIBRARIES

The robot model used was the Gen3 from Kinova ([Robots \(2022\)](#)), and the end-effector is a gripper, the Robotiq-2f-85 from (Robotiq [Robotiq \(2022\)](#)). This model includes the following sensors: torque, position, current, voltage, temperature, accelerometer and gyroscope. We use the position sensor, which provides the arm's position along the three axes (x,y,z). To communicate with the Kinova Gen3 robot, we use the API *Kinova Kortex* integrated with ROS through the package *ROS Kortex*. Thus, ROS Kortex allow us to use the ROS mechanisms (communication and file system) and Gazebo as a simulator to perform the experiments using Kinova Gen3 and Python.

Although the experiments conducted in this thesis utilized pytest-bdd and the ROS Kortex API, it is crucial to compare existing testing frameworks and tools commonly used in robotics and software testing: ROSTest ([RosTest, 2025](#)) and Robot Framework. Our objective is to clarify the scope of AAT4IRS and highlight the specific gaps it addresses.

ROSTest ([RosTest, 2025](#)) is the native testing framework for the ROS. It supports integration and system testing through ROS nodes, launch files, and communication interfaces. However, ROSTest does not provide a structured way to write acceptance tests in natural language. The focus of ROSTest is to verify ROS components rather than validating system-level acceptance criteria or business requirements. In contrast, AAT4IRS uses natural language (BDD/ Gherkin) to describe expected robot behavior, enabling an accessible specification for multidisciplinary teams.

Robot Framework ([Framework, 2025](#)) is an open-source, keyword-driven automation framework used for acceptance testing and robotic process automation. Although the name "Robot Framework" is not suitable for robotics systems. In our thesis, we consider the definition of a robot in [ISO/IEC 8373 \(2012\)](#): a programmed actuated mechanism with a

degree of autonomy to perform locomotion, manipulation or positioning. In this context, it is not a robot: software ("bots", AI, Robotic Process Automation), voice assistants, autonomous cars, ATMs, and intelligent washing machines.

We used the library *pytest-bdd* (Pidsadnyi & Bubenkov (2022)), the most popular framework in Python that implements a subset of the Gherkin language to enable the automating of project requirements testing. The project structure for *pytest-bdd* is based on *pytest* and the following conventions are recommended:

- All test code should appear under a test directory named “tests”.
- Feature files should be placed in a test subdirectory named “features”.
- Step definition modules should be placed in a test subdirectory named “step_defs”.

The robot’s mission performed in our experiment is to transfer a box from its initial position (point A) on the conveyor to its destination at the delivery table (point B) (Figure 5.1). The robotic arm must precisely navigate to the target location and use its gripper to securely grasp the object while ensuring proper alignment. Upon successful pickup, the robotic arm carefully places the box on the delivery table before returning to its original position. The instrumentation necessary to run our experiments besides the IR are sensors to read the IR, box color, and the box positions.

5.2 CASE STUDY CONDUCTION

For our experiment, we have an industrial environment with an IRS performing a pick-and-place process (an industrial process in which an industrial robot picks up an object from one location and places it in another).

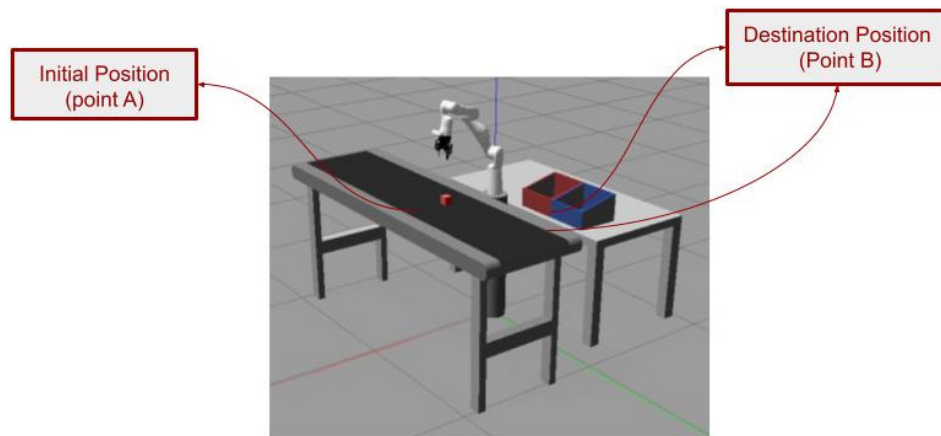


Figure 5.1 : Pick-and-place task performed using Gazebo simulator.

Following the approach defined at Chapter 4, we write the *mission* for the IR to meet the follow BR:

Business Requirement

We need to ensure that the box with a specific color on the conveyor belt dropped in a the delivery point, respecting the box color.

To enhance the BR as outlined in the AAT4IRS workflow (Figure 4.1), we must incorporate critical data regarding the box's final position across the three axes (X, Y, Z). Additionally, we need to specify the initial positions of both the robot and the box on the same axes. Furthermore, it is essential to define the acceptable error thresholds for each axis to ensure optimal performance and accuracy in robotic operations.

Business Requirement

We need to ensure that the box with a specific color on the conveyor belt dropped in a the delivery point, respecting the box color. The robot needs to be positioned correctly before the pick-and-place process can begin within a threshold of **0.02 cm** in all three axes. For the box the threshold is **0.01cm** also for the three axes. And finally, the final position of the box should not exceed **0.02 cm** in any of the three axes.

The AAT4IRS workflow, as shown in Figure 4.1 and described in Chapter 4, outlines a pivotal business decision-making process. In the case study presented in this dissertation, this mission was successfully wrote, leading to the creation of a test suite utilizing AAT4IRS. This organized approach ensures that the mission is aligned with business objectives and rigorously tested for its effectiveness.

The *pytest-bdd* library was chosen to facilitate BDD in the *pick-and-place* task presented in this dissertation. This library allowed for the clear expression of test scenarios, promoting effective collaboration among team members. So, each of the four stages described in AAT4IRS was implemented using Python and *pytest-bdd*.

5.2.1 STAGE 1 - WRITE FEATURES

For the BR used on this case study, one feature that should be implemented is the *Send box to the target with IRS*. In the project structure, the folder *features* with a file *sendBox.feature* contents the description for the feature (Listing 5.1).

```
//testPickAndPlace.py
...
Feature: Send box to the target with IRS
  As a operator ,
  I want to send the box with a specific color to the the delivery
    point using the robot
  So that I can automate the process using a industrial robotic system
```

Listing 5.1 : Feature file for the pick-and-place task.

5.2.2 STAGE 3 - WRITE SCENARIOS

After write the features, we write the scenarios that are examples for the feature and are concerned with the acceptance criteria defined in the business requirement. The first step is to establish the initial environment for our experiment, which involves initializing both the robot and conveyor, moving the robot to the home position, and placing the box in a specific location. Given that each position has three axes, we will compose a *Given* statement for each axis. So, we add this content to the file *.feature* that will have the feature and scenario together (Listing 5.2).

```
//testPickAndPlace.py
...
Feature: Send box according to the box color to the target with the
        same color
        As a operator ,
        I want to send the box with a specific color to the the delivery
        point using the robot
        So that I can automate the process using a industrial robotic system

Scenario: Move the box with a specific color from conveyor to the
        target with the same color
        Given the industrial robotic system is initialized
        And the conveyor is initialized
        And the robot is at home with the position error threshold being
        less than or equal "0.02" cm in the x-axis
        And the robot is at home with the position error threshold being
        less than or equal "0.02" cm in the y-axis
        And the robot is at home with the position error threshold being
        less than or equal "0.02" cm in the z-axis
```

```

    And the box is in the conveyor with the position error threshold
    being less than or equal "0.01" cm in the x-axis

    And the box is in the conveyor with the position error threshold
    being less than or equal "0.01" cm in the y-axis

    And the box is in the conveyor with the position error threshold
    being less than or equal "0.01" cm in the z-axis

    When the pick-and-place finished

    Then the box should be in the specific target with the position
    error threshold being less than or equal "0.02" cm in the x-axis

    Then the box should be in the specific target with the position
    error threshold being less than or equal "0.02" cm in the y-axis

    Then the box should be in the specific target with the position
    error threshold being less than or equal "0.02" cm in the z-axis

```

Listing 5.2 : Scenariofile for the pick-and-place task.

5.2.3 STAGE 4 - IMPLEMENT TESTS

In order to proceed, we must carry out a test that involves creating a function for each statement in the scenario. The criteria for accepting is related to the robot and box within specified position limits. As the acceptance criteria are based on data from sensors that monitor the positions of the robot and box, we incorporated sensor readings into the test script to create assertions (as shown in Listing 5.3 and 5.4). Once the test suite was developed, we ran it against the original code and successfully passed all tests.

```

//testPickAndPlace.py
...
scenarios('../features/sendBox.feature')
@given("the industrial robotic system is initialized")
def setup_robot():

```

```

@given("the conveyor is initialized")
def setup_conveyor():
    @given(parsers.cfparse('the robot is at home with the position error
        threshold being less than or equal "{errorThreshold:Number}" cm in
        the x-axis'
        , extra_types=dict(Number=float)))
    def robot_position_initialX(errorThreshold):
        @given(parsers.cfparse('the robot is at home with the position error
            threshold being less than or equal "{errorThreshold:Number}" cm in
            the y-axis', extra_types=dict(Number=float)))
        def robot_position_initialY(errorThreshold):
            @given(parsers.cfparse('the robot is at home with the position error
                threshold being less than or equal "{errorThreshold:Number}" cm in
                the z-axis', extra_types=dict(Number=float)))
            def robot_position_initialZ(errorThreshold):
                @given(parsers.cfparse('the box is in the conveyor with the position
                    error threshold being less than or equal "{errorThreshold:Number}"
                    cm in the x-axis', extra_types=dict(Number=float)))
                def box_positionAX(errorThreshold):
                    @given(parsers.cfparse('the box is in the conveyor with the position
                        error threshold being less than or equal "{errorThreshold:Number}"
                        cm in the y-axis', extra_types=dict(Number=float)))
                    def box_positionA(errorThreshold):
                        @given(parsers.cfparse('the box is in the conveyor with the position
                            error threshold being less than or equal "{errorThreshold:Number}"
                            cm in the z-axis', extra_types=dict(Number=float)))
                        def box_positionA(errorThreshold):
                            @when("the pick-and-place finished")
                            def pickplace_mission():
                                @then(parsers.cfparse('the box should be in the specific target with
                                    the position error threshold being less than or equal "{

```

```

    errorThreshold:Number}" cm in the x-axis',extra_types=dict(Number=
    float)))
def box_positionBX(errorThreshold):
@then(parsers.cfparse('the box should be in the specific target with
    the position error threshold being less than or equal "{
    errorThreshold:Number}" cm in the y-axis',extra_types=dict(Number=
    float)))
def box_positionBY(errorThreshold):
@then(parsers.cfparse('the box should be in the specific target with
    the position error threshold being less than or equal "{
    errorThreshold:Number}" cm in the z-axis',extra_types=dict(Number=
    float)))
def box_positionBZ(errorThreshold):

```

Listing 5.3 : All the statements created at feature are contemplated by a test in the test file.

```

    @given(parsers.cfparse('the robot is at home with the position
    error threshold being less than or equal "{errorThreshold:Number}"
    cm in the x-axis'
    , extra_types=dict(Number=float)))
def robot_position_initialX(errorThreshold):
    positionRobot = readSensor(1)
    assert round(abs(0.5761017203330994 - positionRobot[0]),2) <=
    errorThreshold

```

Listing 5.4 : An example for the test using the acceptance criteria.

5.3 EVALUATION

We evaluate AAT4IRS through mutation testing. The mutants defined taking into account the fact that the code for a robotic system is not just any piece of procedural code, but

a specific type of program that manipulates robotic components interacting with the physical world.

Figure 5.5 shows a pseudo-code for a possible program in which the IR performs the pick-and-place task. The robot picks the box, lifts it from the ground by some amount, and does the reverse operations after turning to two possible angles, depending on the box's color determined by some visual sensor. The code is mostly a linear sequence of *send* and *read* commands, which, in the context of ROS, give commands to the robot and probe the state of the environment, respectively. The program has few control structures, variables or arithmetic operations, which are the typical points where mutation operators are applied. There are very few locations where mutations can be injected.

```
send('pick')
send('lift/5')
if read('color')== 'red' then
    send('turn/90')
else
    send('turn/270')
endif
send('drop/5')
send('release')
```

Listing 5.5 : A simple robot program.

We note that in the robot code there are a few open space to apply the traditional mutation testing. The code is mostly a linear sequence of send and read commands, which, in the context of ROS, give commands to the robot and probe the state of the environment, respectively. The program has few control structures, variables, or arithmetical operations, which are the typical points where mutation operators are applied.

```
#!/usr/bin/env python3
```

```

import datetime
import time

from kortex_driver.srv import *
from kortex_driver.msg import *

from RobotActions import ExampleCartesianActionsWithNotifications
from std_msgs.msg import String

import rospy

arm = ExampleCartesianActionsWithNotifications()

if __name__ == "__main__":

    arm.listener()
    arm.example_send_gripper_command(0)
    arm.example_home_the_robot()

    arm.example_send_cartesian_pose(0, 0.2, 0, 0, 0, 0)
    arm.example_send_cartesian_pose(0, 0, 0, 80, 0, 0)
    arm.example_send_cartesian_pose(0, 0, -0.29, 0, 0, 0)
    arm.example_send_gripper_command(1)

    time.sleep(1.5)
    arm.example_home_the_robot()

    arm.example_send_cartesian_pose(0, 0, 0, 0, 0, 80)
    arm.example_send_cartesian_pose(0, 0.45, 0, 0, 0, 0)

    arm.example_send_cartesian_pose(-0.9, 0, 0, 0, 0, 0)
    arm.example_send_cartesian_pose(0, 0, 0, 80, 0, 0)
    arm.example_send_cartesian_pose(0, 0, -0.3, 0, 0, 0)

```



```

if int(arm.sensor) % 2 == 0:

    arm.example_send_cartesian_pose(0, -0.2, 0, 0, 0, 0)

arm.example_send_gripper_command(0)
time.sleep(1)
arm.example_home_the_robot()

```

Listing 5.6 : Mission code.

Most of the program’s logic is nested inside string parameters; applying “dumb” mutation operators on these strings are likely to generate invalid robot commands (e.g. `pic` instead of `pick`, line 1), resulting in mutants that will fail every test and provide no informative value.

Assuming the mutation on send commands is conscious of the syntax of each possible invocation, typical alterations on parameters, such as “off-by-one” modifications, may result in situations that are physically impossible to execute. On the contrary, other “off-by-one” modifications may result in mutants that are guaranteed to pass every test.

A direct application of mutation operators for arbitrary procedural code would therefore produce a suite of mutant programs such that a large proportion of them are either guaranteed to be killed, or are equivalent to the original program. In either case, these mutants provide no added value as they are unable to reveal any new gap in the coverage achieved by the test suite. This has long been identified as a problematic that reduces the effectiveness of mutation testing ([Untch *et al.* \(1993b\)](#)), and is exacerbated by the fact that tests on robotic systems are usually lengthy and may require heavy preparations before each run (if executed on actual physical components).

Better yet would be to define mutation operators taking into account the fact that the robot program is not just any piece of procedural code, but a specific type of program that manipulates robotic components interacting with the physical world.

In this discussion, we will explore the implications of angular values, specifically the numbers 90 and -270, and their potential confusion in the context of directional commands. The numbers 90 and -270 can be interpreted as indicators of "left" and "right" movements, respectively. This interpretation is significant because it highlights a common pitfall for developers: the likelihood of confusing these values. Such confusion is more probable than introducing a minor angular error, such as mistaking 90 for -90.

Moreover, as discussed in Chapter 2 sensor readings are inherently subject to noise. This noise can introduce additional variability in the data. Consequently, another category of mutants arises from the manipulation of sensor readings. By simulating various noise levels and types, we can create mutants that reflect the potential inaccuracies in real-world sensor data.

Table 5.1 illustrates the possible transitions between the original code and the mutants we have identified. Each transition represents a specific mutation that can be tested.

Our approach for assessing our methodology involves following the guidelines specified in Table 5.1. It is worth mentioning that we customize these guidelines to align with the language and instrumentation utilized in robotics and simulation. Our evaluation entails generating mutants that pertain to rotation (orientation modifications) and translation (position adjustments) operations for the robot, as well as variations in the initial and final positions for both the robot and the box. Further, we create mutants for the gripper operations, comprising opening and closing actions (Figure 5.2).

Table 5.1 : Original X Mutant Command for Robotic Systems

Original	Mutant
rotate left by X	rotate right by X
rotate right by X	rotate left by X
translate forward by X	translate backwards by X
translate backwards by X	translate forward by X
translate forward by X	translate forward to X
translate backwards by X	translate backwards to X
do command X	do nothing
do command X	do command X twice
distance value is X	distance value is -X (reverse direction)
distance value is X	distance value is X + noise

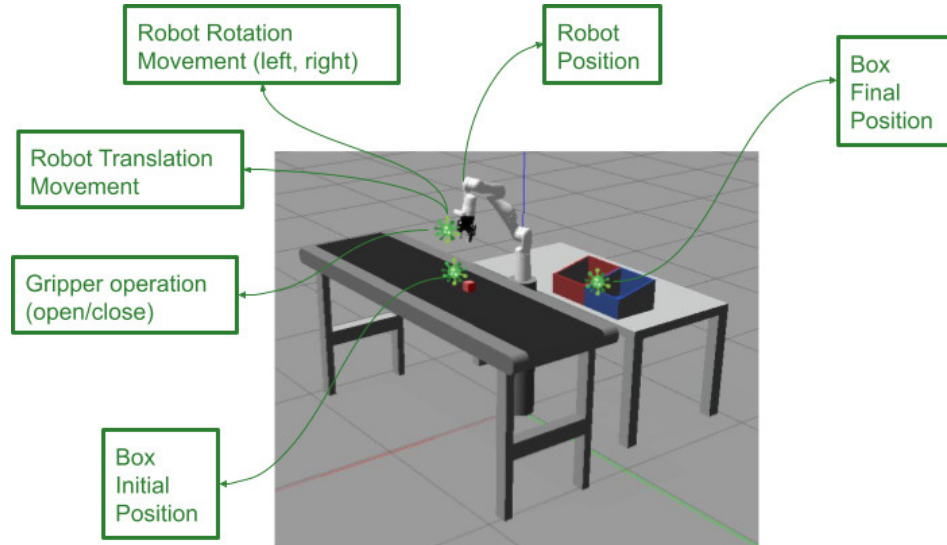


Figure 5.2 : Localization and operation that we apply the mutation.

Table 5.2 shows the 26 mutants created using our guideline and the adaptation needs for the specific robot used in our experiment.

Table 5.2 : List of mutants

Mutant Number	Number	Description
---------------	--------	-------------

#1	Translation	Change the y-value in translation
#2	Rotation	Change the angle orientation in rotation
#3	Translation	Change the z-value in translation
#4	Gripper Operation	Do not change gripper status
#5	Gripper Operation	Change the gripper status twice
#6	Gripper Operation	Do not change gripper status with the opposite expected operation
#7	Rotation	Change angle orientation in rotation
#8	Translation	Change x-value in translation
#9	Robot Initial Position	Sensor reading with opposite expected value for x-component
#10	Robot Initial Position	Sensor reading with opposite expected value for y-component
#11	Robot Initial Position	Sensor reading with opposite expected value for z-component
#12	Robot Initial Position	Sensor reading with noise in x-component
#13	Robot Initial Position	Sensor reading with noise in y-component
#14	Robot Initial Position	Sensor reading with noise in z-component
#15	Box Initial Position	Sensor reading with opposite expected value for x-component

#16	Box Initial Position	Sensor reading with opposite expected value for y-component
#17	Box Initial Position	Sensor reading with opposite expected value for z-component
#18	Box Initial Position	Sensor reading with noise in x-component
#19	Box Initial Position	Sensor reading with noise in y-component
#20	Box Initial Position	Sensor reading with noise in z-component
#21	Box Final Position	Sensor reading with opposite expected value for x-component
#22	Box Final Position	Sensor reading with opposite expected value for y-component
#23	Box Final Position	Sensor reading with opposite expected value for x-component
#24	Box Final Position	Sensor reading with noise in x-component
#25	Box Final Position	Sensor reading with noise in y-component
#26	Box Final Position	Sensor reading with noise in z-component

For the noise added to sensor readings, we added the noise shown in Figure 5.3. This noise is a simulation for a Gaussian noise, normally distributed and is often used to model measurement errors or communication noise. As we can see, the lowest value for this sign is around *0.053* and the bigger is around *0.39*.

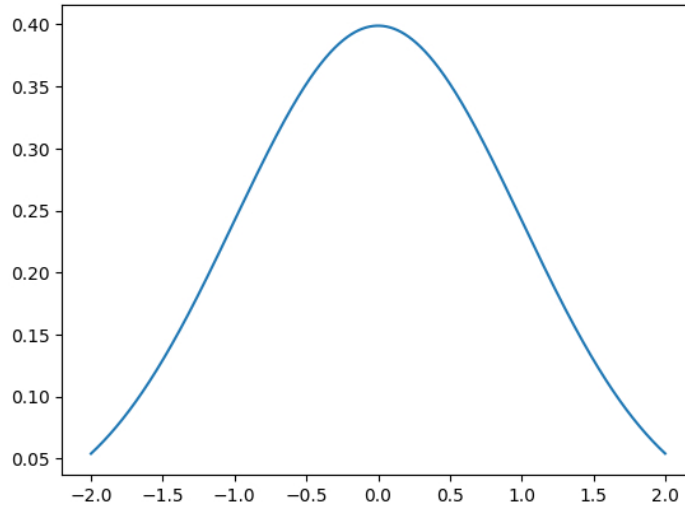


Figure 5.3 : Noise added to the sensor reading.

In order to guarantee the randomness for the noise added and diversity of the boxes, we conducted the test suite for each of the 26 mutants five times. The mutant scores for each round are presented in the table 5.3.

Table 5.3 : Mutant Score for each round

Round	Mutant Score
#1	92%
#2	85%
#3	92%
#4	88%
#5	88%

Figures 5.4 and 5.5 presented a report for one of the rounds in which we have a test failed. The test report is the outcome for Stage 4 in AAT4IRS. For this example, the report shows that three tests failed, and two passed. Figure 5.5 shows the reason why simulation three did not pass, the box final position was not the expected.

Report.html

Report generated on 11-Sep-2022 at 17:28:17 by [pytest-html](#) v3.1.1

Environment

Packages	{"pluggy": "1.0.0", "py": "1.11.0", "pytest": "7.1.2"}
Platform	Linux-5.15.0-46-generic-x86_64-with-glibc2.29
Plugins	{"bdd": "5.0.0", "html": "3.1.1", "metadata": "2.0.2", "repeat": "0.9.1"}
Python	3.8.10

Summary

5 tests ran in 318.44 seconds.

(Un)check the boxes to filter the results.

☒ 2 passed, ☒ 0 skipped, ☒ 3 failed, ☒ 0 errors, ☒ 0 expected failures, ☒ 0 unexpected passes

Results

[Show all details](#) / [Hide all details](#)

▲ Result	▼ Test	▼ Duration	▼ Links
Failed (show details)	tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[3-5]	38.21	
Failed (show details)	tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[4-5]	31.00	
Failed (show details)	tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[5-5]	30.98	
Passed (show details)	tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[1-5]	108.75	
Passed (show details)	tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[2-5]	108.73	

Figure 5.4 : Report Overview

```
Failed (hide details) | tests/steps_defs/test_PickandPlaceAutomation.py::test_move_box_from_a_to_b[3-5] | 118.76 |
request = <FixtureRequest for <Function test_move_box_from_a_to_b[3-5]>>
_pytest_bdd_example = {}

@pytest.mark.usefixtures(*args)
def scenario_wrapper(request, _pytest_bdd_example):
    scenario = templated_scenario.render(_pytest_bdd_example)
> _execute_scenario(feature, scenario, request)

../../../../local/lib/python3.8/site-packages/pytest_bdd/scenario.py:174:
../../../../local/lib/python3.8/site-packages/pytest_bdd/scenario.py:144: in _execute_scenario
    _execute_step_function(request, scenario, step, step_func)
../../../../local/lib/python3.8/site-packages/pytest_bdd/scenario.py:114: in _execute_step_function
    return_value = step_func(**kwargs)

-----

@then("the box should be in the position B")
def box_positionB():
    pyautogui.hotkey('ctrl', 'alt', 't')
    time.sleep(2)
    pyautogui.write('roslaunch caseStudy2 readingGazeboBoxFinal.py')
    pyautogui.press('enter')
    time.sleep(5)
    finalizeSystem(1)
    with open('/home/mdossantos/catkin_workspace/src/caseStudy2/'
              'tests/steps_defs/boxPositionXFinal.txt',
              'r') as input_file:
        for line in input_file:
            pass
            text = line
> assert round(abs(abs(-0.32389056830347945) - abs(float(text))),2) <= 0.02
E   AssertionError: assert 0.29 <= 0.02
E   + where 0.29 = round(0.28943194864319394, 2)
E   +   where 0.28943194864319394 = abs((0.32389056830347945 - 0.6133225169466734))
E   +   where 0.32389056830347945 = abs(-0.32389056830347945)
E   +   and 0.6133225169466734 = abs(0.6133225169466734)
E   +   where 0.6133225169466734 = float('0.6133225169466734\n')

tests/steps_defs/test_PickandPlaceAutomation.py:72: AssertionError
```

Figure 5.5 : Details for one failed test

5.4 DISCUSSIONS

Our findings demonstrate a strong level of test effectiveness, with an average mutation score of 90%. This metric signifies the test suite’s capability to identify faults introduced by code mutations, serving as a crucial indicator of quality in software testing. However, to gain deeper insight into how this effectiveness impacts system reliability, we examined the specific instances in which mutants persisted. Analyzing the survived mutants in all 130 executions, the mutants that survived was **#5**, **#10**, **#14**, **#20** and **#24** in different rounds.

For instance, mutant **#5** introduced a redundant gripper action, causing the robot to close the gripper twice instead of once. While this did not hinder the robot’s ability to complete its task, the additional motion could affect the overall timing of the mission. This highlights a significant nuance: although a test may effectively identify functional faults, it is essential

to incorporate non-functional aspects, such as performance and timing, into the acceptance criteria to ensure reliability in real-world scenarios.

Mutant #10 similarly reversed the error value in the y-axis component of the sensor reading. Although the deviation remained within the specified threshold of 0.02 cm, this underscores a potential blind spot in the test oracle. When establishing what constitutes “acceptable” behavior, it is essential to consider both the sensor’s resolution and the magnitude of the introduced error. This reinforces the notion that acceptance criteria must be closely aligned with both business objectives and the technical capabilities of the system to ensure reliable outcomes.

Lastly, the surviving mutants #14, #20, and #24 resulted from randomized noise within the acceptable threshold. While our test suite did not detect them as faults, this reinforces that reliability in robotic systems is not only about detecting any deviation, but about ensuring that those deviations remain within safe and acceptable margins. The experiment confirms that effective tests properly aligned with sensor accuracy and operational thresholds contribute directly to reliability. Therefore, our mutation analysis not only measures effectiveness but also provides a practical way to validate and refine the reliability of robotic software.

Although our results show a high level of test effectiveness as demonstrated in previous studies that used mutation score as a metric, we acknowledge that this metric alone provides a limited statistical perspective on test quality. To address this concern, we expanded our analysis and examined the characteristics of the surviving mutants across all executions.

We can identify fault types, understand why they went undetected, and evaluate their impact on robot behavior. Finally, we can examine how acceptance criteria, sensor thresholds, and non-functional aspects influence the reliability of robotic systems.

5.5 THREATS TO VALIDITY

We highlight some of these threats and the mechanism we applied to address them.

Mutants generation. We created mutants with just one transition for each mutant. This fact was acceptable because of the nature of the acceptance test for each scenario. The tests implementing each sentence used in our approach (Given-Then-When) are executed in sequence. So, when one of the tests fails, the process is stopped. In other words, if the test that implements the *given* sentence fails, the process stops and the test report is generated. Furthermore, no more tests are executed. Therefore, the unique transition for each mutant was an acceptable method to create mutants to evaluate our approach.

Input data. As observed in the experiment, sensor characteristics was the reason for the survived mutant. Our approach is not concerned with the input data, but we strongly suggest that domain experts must also choose the input data. Besides, our experiment performed a well-known process in robotic fields, and we used data close to accurate data. Thus, further experiments with more realistic input data will be necessary to confirm the effectiveness of our approach.

Safety A potential safety-related threat arises from the fact that our experiment was conducted in simulation, which does not ensure safe behavior in real-world environments with physical robots. We understand that a safety additional layers based on industrial standards (ISO, 2025) need to be incorporate. While these considerations were outside the scope of our evaluation, they are essential for implementing AAT4IRS in safety-critical industrial settings.

5.6 CHAPTER FINAL REMARKS

This chapter outlines the application of AAT4IRS to improve fault detection in Industrial Robotic Systems. As mentioned, one of the challenges to apply software testing for robotic systems is related to the communication and collaboration, culture of testing. We implemented our experiment using ROS, Gazebo, and pytest-bdd, a Python library dedicated to BDD. To evaluate the effectiveness of our software testing approach, we tested the generated test suites against mutants created from the original code. The test suites produced through the use of AAT4IRS achieved a mutation score average of 90%.

In our assessment, we utilized mutation testing to generate mutants that accurately reflect the complexities of the robotic landscape. Our methodology involved creating mutants that targeted the non-deterministic elements that are inherently present in robotic systems, such as fluctuations in sensor readings, as well as mutants that accounted for linguistic subtleties. By employing this nuanced approach, we were able to gain valuable insights into the robustness and flexibility of our proposed methodology within the constantly evolving field of robotics.

When evaluating business requirements for industrial robotic systems, it is crucial to consider both the robot's physical attributes and overall business objectives. Achieving alignment across all teams involved in the project, which may include individuals from various backgrounds, is essential when establishing acceptance criteria. Additionally, utilizing live documentation made possible by the AAT4IRS implementation can help foster collaboration among teams, allowing for more effective problem-solving when facing the complex challenges associated with these types of projects.

In addition, the results observed in our experiment **suggest** that AAT4IRS has some benefits such as: (i) a reduction in the effort to discover faults in IRS; (ii) improvement in traceability; (iii) better coverage of regression tests; and (iv) improvement of coordination,

collaboration, and documentation in robotic systems development teams. All these benefits need to be evaluated in future work, but since they are known benefits of applying software testing in conventional system, we strongly believe that they can also be for robotic system.

CONCLUSION

As the complexity of tasks performed by IR increases, along with the interactions between robots and humans, ensuring the reliability and quality of their software becomes increasingly essential. One effective approach to enhancing quality is the application of software testing in IRS. However, several gaps and challenges arise at the intersection of robotics and software testing. For instance, there are issues related to coordination and collaboration among teams with diverse backgrounds, varying levels of knowledge about software testing among individuals from different fields, as well as constraints in time and resources. Additionally, there is often distrust towards simulation. Furthermore, IRS, like all robotic systems, must contend with unpredictable sensor data and real-world interactions.

To address this gap, this thesis introduces and evaluates an approach called AAT4IRS. This method adapts well-established BDD templates to create AT in natural language, utilizing simulation tools to automate the validation of robotic tasks. To assess the effectiveness of the approach, we employ mutation testing, taking into account the inherent unpredictability associated with IRS.

As part of our research, we conducted a systematic mapping study (**RG1**) to explore the application of software engineering practices in robotic systems, particularly from a quality perspective. The study revealed that while the significance of software engineering in robotics is well recognized, certain areas remain underexplored or implemented without systematic evaluation. Most of the literature primarily addresses the design, modeling, and construction phases. Notably, only 18% of the studies explicitly focused on software testing or quality assurance activities. This finding underscores a gap in the field and emphasizes the need for dedicated strategies to enhance testing in IRS. The insights gained from this mapping study

served as a foundation for the design of AAT4IRS, further confirming the lack of structured testing methodologies tailored to the unique challenges posed by robotic systems.

Building on these findings, we proposed and validated AAT4IRS as a key component of **RG2**. This approach leverages BDD templates, which structure acceptance tests in a natural language format (Given–When–Then). We applied this method to a pick-and-place robotic task within a simulated environment, and the resulting test suites were evaluated using mutation testing. The test cases achieved an average mutation score of 90%, demonstrating a high fault-detection capability. These results illustrate that automated acceptance testing, utilizing natural language and enhanced through simulation, can significantly improve the reliability of robotic software.

To validate our approach in addressing the challenge of distrust in simulation, we developed a mutation testing guideline as part of **RG3**. During our evaluation, we discovered that traditional mutation operators commonly used in conventional software testing were ineffective for IRS. This ineffectiveness arises from the fact that robotic code typically includes fewer control structures and arithmetic operations. Consequently, conventional mutants failed to represent meaningful faults leading to a non-informative failing tests. To address this issue, we introduced domain-specific mutants and non-deterministic mutation operators that mimic the real-world uncertainties frequently encountered in robotic systems, such as sensor noise, environmental variations, and timing deviations. These new mutants reflect faults in IRS and facilitate the evaluation of whether test suites can detect failures. By integrating these elements into mutation testing, our guideline enhances the trust in simulation-based testing, establishing it as a more reliable tool for validating robotic software.

This work contributes to bridging the gap between software testing and robotics. We propose an approach that adapts the AT template from BDD using simulation, and we evaluate

it through domain-specific mutation testing. Our goal is to enhance software testing strategies for IRS.

While the results presented are promising, it is important to acknowledge certain limitations. First, we do not test in a real-world scalable environment. To address this, we utilized a significant industrial scenario, carefully modeled through ROS and Gazebo. We incorporated physical constraints and sensor noise, which enabled a controlled yet representative evaluation of robotic behavior. Furthermore, our approach was evaluated using a single type of robot and task. However, this task was chosen due to its substantial adoption within industrial settings, providing valuable insights for validating robotic systems. Additionally, to ensure methodological rigor and enable an objective assessment of fault detection, we applied the mutation score, a consolidated metric in the software testing literature, which allowed us to quantify the effectiveness of the generated test suites.

Future research could investigate the application of AAT4IRS within physical robot validation, empirically demonstrating scalability and adaptability, and assessing its effectiveness through interviews with experts. Enhancing this approach to accommodate orientation and joint-level constraints better may further refine validation precision. Furthermore, incorporating tools that enable the semi-automatic extraction of business requirements or facilitate the generation of test scenarios from logs or user stories would make the process more accessible to multidisciplinary teams. Lastly, expanding the mutation testing framework to automatically generate mutants and include timing-related faults, additional hardware failures, and more complex robot behaviors would significantly enhance the evaluation's effectiveness. Furthermore, a comparative analysis with other verification methods (fault injection, model checking) and benchmarking AAT4IRS mutation results against alternative techniques will be included as future work.

This thesis shows that the integration of AT and mutation testing, utilizing the unique characteristics of IRS, enhances software quality. This approach provides a foundation in software testing for IRS.

REFERENCES

- (1990). IEEE Standard Glossary of Software Engineering Terminology. *IEEE Std 610.12-1990*, 1–84. doi: [10.1109/IEEESTD.1990.101064](https://doi.org/10.1109/IEEESTD.1990.101064)
- (2025, february)*Robotics — Safety requirements — Part 1: Industrial robots*. Edition 3, reference number 73933. © ISO 2025.
- ABB (2010). *Technical reference manual. RAPID Instructions, Functions and Data types*. <https://library.e.abb.com>. Accessed: 2020-12-02.
- Abdelgawad, M., McLeod, S., Andrews, A. & Xiao, J. (2017). Model-based testing of a real-time adaptive motion planning system. *Advanced Robotics*, 31(22), 1159–1176. doi: [10.1080/01691864.2017.1396921](https://doi.org/10.1080/01691864.2017.1396921)
- Afzal, A. (2021). Automated Testing of Robotic and Cyberphysical Systems. doi: [10.1184/R1/16645639.v1](https://doi.org/10.1184/R1/16645639.v1)
- Afzal, A., Goues, C. L., Hilton, M. & Timperley, C. S. (2020). A Study on Challenges of Testing Robotic Systems. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 96–107. doi: [10.1109/ICST46399.2020.00020](https://doi.org/10.1109/ICST46399.2020.00020)
- Afzal, A., Katz, D. S., Le Goues, C. & Timperley, C. S. (2021). Simulation for Robotics Test Automation: Developer Perspectives. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 263–274. doi: [10.1109/ICST49551.2021.00036](https://doi.org/10.1109/ICST49551.2021.00036)
- Ahmad, A. & Babar, M. A. (2016). Software architectures for robotic systems- A systematic mapping study. *Journal of Systems and Software*, 122, 16 – 39. doi: <https://doi.org/10.1016/j.jss.2016.08.039>
- Ahn, H., Loh, W. K. & Yeo, W. Y. (2012). A framework-based approach for fault-tolerant service robots. *International Journal of Advanced Robotic Systems*, 9, 1–10. doi: [10.5772/54023](https://doi.org/10.5772/54023)
- Akiki, P. A., Akiki, P. A., Bandara, A. K. & Yu, Y. (2020). EUD-MARS: End-user development of model-driven adaptive robotics software systems. *Science of Computer*

Programming, 200, 102534. doi: [10.1016/j.scico.2020.102534](https://doi.org/10.1016/j.scico.2020.102534)

Albuquerque, D., Castro, J., Ribeiro, S. & Heineck, T. (2017). Requirements Engineering for Robotic System: A Systematic Mapping Study. *WER*.

Alexander, R., Hawkins, H. & Rae, A. (2015). Situation coverage – a coverage criterion for testing autonomous robots.

Alexandrova, S., Tatlock, Z. & Cakmak, M. (2015). RoboFlow: A flow-based visual programming language for mobile manipulation tasks. *Proceedings - IEEE International Conference on Robotics and Automation, 2015-June*(June), 5537–5544. doi: [10.1109/ICRA.2015.7139973](https://doi.org/10.1109/ICRA.2015.7139973)

Álvarez, B., Sánchez-Palma, P., Pastor, J. A. & Ortiz, F. (2006). An architectural framework for modeling teleoperated service robots. *Robotica*, 24(4), 411–418. doi: [10.1017/S0263574705002407](https://doi.org/10.1017/S0263574705002407)

Andersson, S. & Carlstedt, G. (2019). *Automated Testing of Robotic Systems in Simulated Environments*. Mälardalen University, School of Innovation, Design and Engineering.

Andrews, J., Briand, L., Labiche, Y. & Namin, A. (2006). Using Mutation Analysis for Assessing and Comparing Testing Coverage Criteria. *IEEE Transactions on Software Engineering*, 32(8), 608–624. doi: [10.1109/TSE.2006.83](https://doi.org/10.1109/TSE.2006.83)

Ashraf, A. K., D’Souza, M. & Jetley, R. (2020). Coverage Criteria based Testing of Industrial Robots. In *2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*, 16–21., Hong Kong, China. IEEE. doi: [10.1109/CASE48305.2020.9217031](https://doi.org/10.1109/CASE48305.2020.9217031)

Bardaro, G., Semprebon, A. & Matteucci, M. (2018). A Use Case in Model-Based Robot Development Using AADL and ROS. In *2018 IEEE/ACM 1st International Workshop on Robotics Software Engineering (RoSE)*, 9–16.

Bäumel, B. & Hirzinger, G. (2006). Agile Robot Development (aRD): A pragmatic approach to robotic software. *IEEE International Conference on Intelligent Robots and Systems*, 3741–3748. doi: [10.1109/IROS.2006.281756](https://doi.org/10.1109/IROS.2006.281756)

Behave Project (2024). *Behave Documentation*. <https://behave.readthedocs.io/en/stable/>. Accessed: 2025-06-13.

Beltrame, G., Merlo, E., Panerati, J. & Pincirolì, C. (2018). Engineering safety in swarm robotics. *Proceedings - International Conference on Software Engineering*, 36–39. doi: [10.1145/3196558.3196565](https://doi.org/10.1145/3196558.3196565)

Ben-Ari, M. & Mondada, F. (2018). *Elements of Robotics*. Cham: Springer. doi: [10.1007/978-3-319-62533-1](https://doi.org/10.1007/978-3-319-62533-1)

Boehm, B., Brown, J., Kaspar, H., Lipow, M., Macleod, G. & Merrit, M. (1984). *Specification of software quality attributes*. Prepared by Boeing for RADC.

Boehm, B. W., Brown, J. R. & Lipow, M. (1976). Quantitative Evaluation of Software Quality. In *Proceedings of the 2Nd International Conference on Software Engineering*, ICSE '76, 592–605., Los Alamitos, CA, USA. IEEE Computer Society Press.

Bossecker, E., Sousa Calepso, A., Kaiser, B., Verl, A. & Sedlmair, M. (2023). A Virtual Reality Simulator for Timber Fabrication Tasks Using Industrial Robotic Arms. In *Proceedings of Mensch Und Computer 2023*, MuC '23, p. 568–570., New York, NY, USA. Association for Computing Machinery. doi: [10.1145/3603555.3609316](https://doi.org/10.1145/3603555.3609316)

Bourque, P., Fairley, R. E. & Society, I. C. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0* (3rd). Washington, DC, USA: IEEE Computer Society Press.

Bozhinoski, D. & Birattari, M. (2018). Designing Control Software for Robot Swarms: Software Engineering for the Development of Automatic Design Methods. In *2018 IEEE/ACM 1st International Workshop on Robotics Software Engineering (RoSE)*, 33–35.

Bozhinoski, D., Di Ruscio, D., Malavolta, I., Pelliccione, P. & Crnkovic, I. (2019). Safety for mobile robotic system: A systematic mapping study from a software engineering perspective. *Journal of Systems and Software*, 150–179. doi: [10.1016/j.jss.2019.02.021](https://doi.org/10.1016/j.jss.2019.02.021)

Breitenhuber, G. (2020). Towards application level testing of ROS networks. In *2020 Fourth IEEE International Conference on Robotic Computing (IRC)*, 436–442. doi:

[10.1109/IRC.2020.00081](https://doi.org/10.1109/IRC.2020.00081)

Bretl, T. & Lall, S. (2008). Testing static equilibrium for legged robots. *IEEE Transactions on Robotics*, 24(4), 794–807. doi: [10.1109/TRO.2008.2001360](https://doi.org/10.1109/TRO.2008.2001360)

Brugali, D. & Hochgeschwender, N. (2018). Software Product Line Engineering for Robotic Perception Systems. *International Journal of Semantic Computing*, 89–107. doi: [10.1142/S1793351X18400056](https://doi.org/10.1142/S1793351X18400056)

Brugali, D. & Scandurra, P. (2009). Component-based robotic engineering (Part I) [Tutorial]. *IEEE Robotics & Automation Magazine*, 16(4), 84–96. doi: [10.1109/MRA.2009.934837](https://doi.org/10.1109/MRA.2009.934837)

BurguEo, L., Mayerhofer, T., Wimmer, M. & Vallecillo, A. (2018). Using physical quantities in robot software models. *Proceedings - International Conference on Software Engineering*, 23–28. doi: [10.1145/3196558.3196562](https://doi.org/10.1145/3196558.3196562)

Carlson, J. & Murphy, R. R. (2003). Reliability analysis of mobile robots. *Proceedings - IEEE International Conference on Robotics and Automation*, 1, 274–281. doi: [10.1109/robot.2003.1241608](https://doi.org/10.1109/robot.2003.1241608)

Chandorkar, A., Patkar, N., Di Sorbo, A. & Nierstrasz, O. (2022, 03). An Exploratory Study on the Usage of Gherkin Features in Open-Source Projects. 1159–1166. doi: [10.1109/SANER53432.2022.00134](https://doi.org/10.1109/SANER53432.2022.00134)

Chelimsky, D., Astels, D., Helmkamp, Z. D., North, D., Dennis, T. & Hellesoy, A. (2010). *The RSpec Book: Behaviour Driven Development with RSpec, Cucumber, and Friends*. Raleigh, NC: Pragmatic Bookshelf.

Chillarege, R. (1996). What Is Software Failure? *IEEE Transactions on Reliability*, 45(3), 354–. doi: [10.1109/TR.1996.536980](https://doi.org/10.1109/TR.1996.536980)

Cho, Y., Kim, H., Kim, D. K. & Park, S. (2012). An interaction-driven approach to identifying functional behaviors of service robot systems. *Proceedings - 2012 IEEE 17th International Conference on Engineering of Complex Computer Systems, ICECCS 2012*, 109–118. doi: [10.1109/ICECCS.2012.9](https://doi.org/10.1109/ICECCS.2012.9)

Coevoet, E., Morales-Bieze, T., Largilliere, F., Zhang, Z., Thieffry, M., Sanz-Lopez, M., Carrez, B., Marchal, D., Goury, O., Dequidt, J. & Duriez, C. (2017). Software toolkit for modeling, simulation, and control of soft robots. *Advanced Robotics*, 31(22), 1208–1224. doi: [10.1080/01691864.2017.1395362](https://doi.org/10.1080/01691864.2017.1395362)

Craig, J. J. (2018). *Introduction to Robotics: Mechanics and Control*. Hoboken, NJ.:Pearson.

Cucumber Project (2024). *Cucumber Documentation*. <https://cucumber.io/docs/>. Accessed: 2025-06-13.

Education, C. R. . (2021). *CORE Rankings Portal*. <https://www.core.edu.au/conference-portal>. Accessed: 2020-12-02.

Erich, F., Saksena, A., Biggs, G. & Ando, N. (2019). Design and Development of a Physical Integration Testing Framework for Robotic Manipulators. In *2019 IEEE/SICE International Symposium on System Integration (SII)*, 602–607., Paris, France. IEEE. doi: [10.1109/SII.2019.8700444](https://doi.org/10.1109/SII.2019.8700444)

Ernst, N. A., Kazman, R. & Bianco, P. (2018). Toward Rapid Composition with Confidence in Robotics Software. In *2018 IEEE/ACM 1st International Workshop on Robotics Software Engineering (RoSE)*, 44–47.

Estivill-Castro, V., Hexel, R. & Lusty, C. (2018). Continuous integration for testing full robotic behaviours in a GUI-stripped simulation. *CEUR Workshop Proceedings*, 2245, 453–464.

Fairley, R. E. D., Bourque, P. & Keppler, J. (2014). The impact of SWEBOK Version 3 on software engineering education and training. In *2014 IEEE 27th Conference on Software Engineering Education and Training (CSEET)*, 192–200. doi: [10.1109/CSEET.2014.6816804](https://doi.org/10.1109/CSEET.2014.6816804)

Farley, A., Wang, J. & Marshall, J. A. (2022). How to pick a mobile robot simulator: A quantitative comparison of CoppeliaSim, Gazebo, MORSE and Webots with a focus on accuracy of motion. *Simulation Modelling Practice and Theory*, 120, 102629. doi: <https://doi.org/10.1016/j.simpat.2022.102629>

Ferry, K. (2018). *Future of Work - The global talent crunch*. https://www.kornferry.com/content/dam/kornferry/docs/article-migration/FOWTalentCrunchFinal_Spring2018.pdf. Accessed: 2022-06-17.

Flückiger, L. & Utz, H. (2014). Service oriented robotic architecture for space robotics: Design, testing, and lessons learned. *Journal of Field Robotics*, 31(1), 176–191. doi: [10.1002/rob.21485](https://doi.org/10.1002/rob.21485)

Framework, R. (2025). *Robot Framework*. <https://robotframework.org/>. Accessed: 2025-11-19.

García, S., Struber, D., Brugali, D., Berger, T. & Pelliccione, P. (2020). Robotics Software Engineering - A Perspective from the Service Robotics Domain. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, p. 593–604., New York, NY, USA. Association for Computing Machinery. doi: [10.1145/3368089.3409743](https://doi.org/10.1145/3368089.3409743)

Gerasimou, S., Matragkas, N. & Calinescu, R. (2019). Towards systematic engineering of collaborative heterogeneous robotic systems. *Proceedings - 2019 IEEE/ACM 2nd International Workshop on Robotics Software Engineering, RoSE 2019*, 25–28. doi: [10.1109/RoSE.2019.00012](https://doi.org/10.1109/RoSE.2019.00012)

Gotlieb, A., Marijan, D. & Spieker, H. (2021). Testing Industrial Robotic Systems: A New Battlefield! In *Software Engineering for Robotics* (109–137). Cham: Springer International Publishing. doi: [10.1007/978-3-030-66494-7_4](https://doi.org/10.1007/978-3-030-66494-7_4)

Graham, D., Van Veenendaal, E. & Evans, I. (2008). *Foundations of Software Testing: ISTQB Certification*. Boston, MA: Course Technology Cengage Learning.

Groover, M. P. (2007). *Automation, Production Systems, and Computer-Integrated Manufacturing* (3rd). USA: Prentice Hall Press.

Han, K. H., Kim, S., Kim, Y. J. & Kim, J. H. (2001). Internet control architecture for internet-based personal robot. *Autonomous Robots*, 10(2), 135–147. doi: [10.1023/A:1008941101455](https://doi.org/10.1023/A:1008941101455)

Heimann, O. & Guhl, J. (2020). *Industrial Robot Programming Methods: A Scoping*

Review. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1, 696–703. doi: [10.1109/ETFA46521.2020.9211997](https://doi.org/10.1109/ETFA46521.2020.9211997)

Hossain, N., Rimon, M. I. H., Mimona, M. A., Mobarak, M. H., Ghosh, J., Islam, M. A. & Mahmud, M. Z. A. (2024). Prospects and challenges of sensor materials: A comprehensive review. *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, 7, 100496. doi: <https://doi.org/10.1016/j.prime.2024.100496>

Hunecke, B., Nguyen, M., Hochgeschwender, N. & Wrede, S. (2025). Specification and Execution of Robotic Acceptance Tests for Object Sorting. In *European Robotics Forum*, 200–205. Springer.

Iborra, A., Caceres, D. A., Ortiz, F. J., Franco, J. P., Palma, P. S. & Alvarez, B. (2009). Design of service robots. *IEEE Robotics Automation Magazine*, 16(1), 24–33. doi: [10.1109/MRA.2008.931635](https://doi.org/10.1109/MRA.2008.931635)

Icli, C., Stepanenko, O. & Bonev, I. (2020). New Method and Portable Measurement Device for the Calibration of Industrial Robots. *Sensors*, p. 5919. doi: [10.3390/s20205919](https://doi.org/10.3390/s20205919)

IFR (2024). *Executive Summary World Robotics 2023 Industrial Robots*. https://ifr.org/img/worldrobotics/Executive_Summary_WR_Industrial_Robots_2023.pdf. Accessed: 2024-04-06.

IFR, T. I. F. o. R. (2020). *Automated calibration process at Perschmann Calibration GmbH with the sensitive KUKA Robot*. Retrieved from <https://ifr.org/members-news/suppliers/automated-calibration-process-at-perschmann-calibration-gmbh-with-the-sensi>

Irshad, M., Britto, R. & Petersen, K. (2021). Adapting Behavior Driven Development (BDD) for large-scale software systems. *Journal of Systems and Software*, 177, 110944. doi: <https://doi.org/10.1016/j.jss.2021.110944>

ISO/IEC 25010 (2011). *ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

ISO/IEC 8373 (2012). *Robots and robotic devices — Vocabulary*. International Organization

for Standardization.

ISTQ (2021). *ISTQB® Glossary of Terms used in Software Testing*. International Software Testing Qualifications Board, Retrieved from <https://glossary.istqb.org>

Jawawi, D. N., Mamat, R. & Deris, S. (2007). A component-oriented programming for embedded mobile robot software. *International Journal of Advanced Robotic Systems*, 4(3), 371–380. doi: [10.5772/5678](https://doi.org/10.5772/5678)

Jia, Y. & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5), 649–678. doi: [10.1109/TSE.2010.62](https://doi.org/10.1109/TSE.2010.62)

Kim, M., Kim, S., Park, S., Choi, M. T., Kim, M. & Gomaa, H. (2006). UML-based service robot software development: A case study. *Proceedings - International Conference on Software Engineering, 2006(Icse)*, 534–543.

Kim, M., Lee, J., Kang, K. C., Hong, Y. & Bang, S. (2005). Re-engineering software architecture of home service robots: A case study. *Proceedings - 27th International Conference on Software Engineering, ICSE05*, 505–513. doi: [10.1109/icse.2005.1553596](https://doi.org/10.1109/icse.2005.1553596)

Kitchenham, B. & Brereton, P. (2013, 12). A Systematic Review of Systematic Review Process Research in Software Engineering. *Inf. Softw. Technol.*, 2049–2075. doi: [10.1016/j.infsof.2013.07.010](https://doi.org/10.1016/j.infsof.2013.07.010)

Kitchenham, B. A. & Charters, S. (2007). *Guidelines for performing Systematic Literature Reviews in Software Engineering* publication no. EBSE 2007-001. Keele University and Durham University Joint Report.

Koenig, N. & Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, Vol. 3, 2149–2154 vol.3. doi: [10.1109/IROS.2004.1389727](https://doi.org/10.1109/IROS.2004.1389727)

Kudelski, M., Gambardella, L. M. & Di Caro, G. A. (2013). RoboNetSim: An integrated framework for multi-robot and network simulation. *Robotics and Autonomous Systems*,

61(5), 483–496. doi: [10.1016/j.robot.2013.01.003](https://doi.org/10.1016/j.robot.2013.01.003)

KUKA (2021). *KUKA Aktiengesellschaft Germany*. <https://www.kuka.com/>. Accessed: 2020-12-02.

Layne, L. (2023). Robot-related fatalities at work in the United States, 1992-2017. *American journal of industrial medicine*. doi: [10.1002/ajim.23470](https://doi.org/10.1002/ajim.23470)

Le, B. S., Long, V. & Bui, T.-T. (2014, 11). Swarm Robotics Simulation Using Unity.

Li, H. (2016). *Communications for Control in Cyber Physical Systems: Theory, Design and Applications in Smart Grids*.

Li, Z., Li, S. & Luo, X. (2021). An overview of calibration technology of industrial robots. *IEEE/CAA Journal of Automatica Sinica*, 8(1), 23–36. doi: [10.1109/JAS.2020.1003381](https://doi.org/10.1109/JAS.2020.1003381)

Lubrano, E., Bouri, M. & Gavel, R. (2011, 06). Ultra-high-precision industrial robots calibration. 228 – 233. doi: [10.1109/ICRA.2011.5979746](https://doi.org/10.1109/ICRA.2011.5979746)

Malavolta, I., Lewis, G., Schmerl, B., Lago, P. & Garlan, D. (2020). How Do You Architect Your Robots? State of the Practice and Guidelines for ROS-Based Systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP '20*, p. 31–40., New York, NY, USA. Association for Computing Machinery. doi: [10.1145/3377813.3381358](https://doi.org/10.1145/3377813.3381358)

Maoz, S. & Ringert, J. O. (2018). On the software engineering challenges of applying reactive synthesis to robotics. *Proceedings - International Conference on Software Engineering*, 17–22. doi: [10.1145/3196558.3196561](https://doi.org/10.1145/3196558.3196561)

Marijan, D., Gotlieb, A. & Kumar Ahuja, M. (2019). Challenges of Testing Machine Learning Based Systems. In *2019 IEEE International Conference On Artificial Intelligence Testing (AITest)*, 101–102. doi: [10.1109/AITest.2019.00010](https://doi.org/10.1109/AITest.2019.00010)

McCall, J., Richards, P. & G.F. (1978). *Factors in Software Quality (3 volumes)*. RADC reports.

McKee, G., Fryer, J. & Schenker, P. (2001). Object-oriented concepts for modular robotics systems. In *Proceedings 39th International Conference and Exhibition on Technology of Object-Oriented Languages and Systems. TOOLS 39*, 229–238. doi: [10.1109/TOOLS.2001.941676](https://doi.org/10.1109/TOOLS.2001.941676)

Messina, E., Horst, J., Kramer, T., Huang, H. M. & Michaloski, J. (1999). Component specifications for robotics integration. *Autonomous Robots*, 6(3), 247–264. doi: [10.1023/A:1008980211618](https://doi.org/10.1023/A:1008980211618)

Michael, N., Fink, J. & Kumar, V. (2008). Experimental Testbed for Large Multirobot Teams: Verification and Validation. *IEEE Robotics and Automation Magazine*, 15(1), 53–61. doi: [10.1109/M-RA.2007.914924](https://doi.org/10.1109/M-RA.2007.914924)

Michel, O. (2004). WebotsTM: Professional Mobile Robot Simulation. *International Journal of Advanced Robotic Systems*. doi: [10.5772/5618](https://doi.org/10.5772/5618)

Minseong, K., Suntae, K., Sooyong, P., Mun-Taek, C., Munsang, K. & Gomaa, H. (2009). Service robot for the elderly: Software development with the COMET/UML method. *IEEE Robotics and Automation Magazine*, 16(1), 34–45. doi: [10.1109/MRA.2008.931636](https://doi.org/10.1109/MRA.2008.931636)

Mnyusiwalla, H., Triantafyllou, P., Sotiropoulos, P., Roa, M. A., Friedl, W., Sundaram, A. M., Russell, D. & Deacon, G. (2020). A Bin-Picking Benchmark for Systematic Evaluation of Robotic Pick-and-Place Systems. *IEEE Robotics and Automation Letters*, 5(2), 1389–1396. doi: [10.1109/LRA.2020.2965076](https://doi.org/10.1109/LRA.2020.2965076)

Molina, M., Carrera, A., Camporredondo, A., Bavle, H. & Rodriguez-ramos, A. (2020). Building the executive system of autonomous aerial robots using the Aerostack open-source framework. *International Journal of Advanced Robotic Systems*, 17(June), 1–20. doi: [10.1177/1729881420925000](https://doi.org/10.1177/1729881420925000)

Mossige, M., Gotlieb, A. & Meling, H. (2015). Testing robot controllers using constraint programming and continuous integration. *Information and Software Technology*, 57(1), 169–185. doi: [10.1016/j.infsof.2014.09.009](https://doi.org/10.1016/j.infsof.2014.09.009)

Mourão, E., Pimentel, J. F., Murta, L., Kalinowski, M., Mendes, E. & Wohlin, C. (2020). On the performance of hybrid search strategies for systematic literature reviews in software engineering. *Information and Software Technology*, 123, 106294. doi: [10.1016/j.infsof.2020.106294](https://doi.org/10.1016/j.infsof.2020.106294)

<https://doi.org/10.1016/j.infsof.2020.106294>

Naik, K. & Tripathy, P. (2018). *Software Testing and Quality Assurance: Theory and Practice* (2nd). Nova Jersey, EUA: Wiley Publishing.

Nguyen, M., Hochgeschwender, N. & Wrede, S. (2023). An Analysis of Behaviour-Driven Requirement Specification for Robotic Competitions. In *2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE)*, 17–24. doi: [10.1109/RoSE59155.2023.00008](https://doi.org/10.1109/RoSE59155.2023.00008)

Nordmann, A., Wrede, S. & Steil, J. (2015). Modeling of movement control architectures based on motion primitives using domain-specific languages. *Proceedings - IEEE International Conference on Robotics and Automation, 2015-June(June)*, 5032–5039. doi: [10.1109/ICRA.2015.7139899](https://doi.org/10.1109/ICRA.2015.7139899)

Ore, J.-P., Detweiler, C. & Elbaum, S. (2018). Toward Code-Aware Robotic Simulation. In *2018 IEEE/ACM 1st International Workshop on Robotics Software Engineering (RoSE)*, 40–43.

Ortiz, F. J., Insaurralde, C. C., Alonso, D., Sánchez, F. & Petillot, Y. R. (2015). Model-driven analysis and design for software development of autonomous underwater vehicles. *Robotica*, 33(8), 1731–1750. doi: [10.1017/S0263574714001027](https://doi.org/10.1017/S0263574714001027)

Pan, Z., Polden, J., Larkin, N., Duin, S. V. & Norrish, J. (2010). Recent Progress on Programming Methods for Industrial Robots. 1–8.

Papadakis, M., Shin, D., Yoo, S. & Bae, D.-H. (2018). Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults. In *Proceedings of the 40th International Conference on Software Engineering, ICSE '18*, p. 537–548., New York, NY, USA. Association for Computing Machinery. doi: [10.1145/3180155.3180183](https://doi.org/10.1145/3180155.3180183)

Paraschos, A., Spanoudakis, N. I. & Lagoudakis, M. G. (2012). Model-driven behavior specification for robotic teams. *11th International Conference on Autonomous Agents and Multiagent Systems 2012, AAMAS 2012: Innovative Applications Track*, 2(March 2014), 696–703.

Peng, L., Guan, F., Perneel, L. & Timmerman, M. (2016). EmSBot A modular framework supporting the development of swarm robotics applications. *International Journal of Advanced Robotic Systems*, 13(6), 1–15. doi: [10.1177/1729881416663662](https://doi.org/10.1177/1729881416663662)

Pidsadnyi, O. & Bubenkov, A. (2022). *Welcome to Pytest-BDD's documentation!* <https://pytest-bdd.readthedocs.io/en/stable/>. Accessed: 2022-09-03.

Pogliani, M., Maggi, F., Balduzzi, M., Quarta, D. & Zanero, S. (2020). Detecting Insecure Code Patterns in Industrial Robot Programs. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '20, p. 759–771., New York, NY, USA. Association for Computing Machinery. doi: [10.1145/3320269.3384735](https://doi.org/10.1145/3320269.3384735)

Pressman, R. (2010). *Software Engineering: A Practitioner's Approach* (7). New York, NY, USA: McGraw-Hill, Inc.

Pucciani, G. (2023). Gherkin and Behavior Driven Development. In *Boozang from the Trenches: Learn Test Automation with Boozang in an Enterprise Environment* (217–224). Berkeley, CA: Apress. doi: [10.1007/978-1-4842-9010-1_13](https://doi.org/10.1007/978-1-4842-9010-1_13)

Pyo, Y., Nakashima, K., Kuwahata, S., Kurazume, R., Tsuji, T., Morooka, K. & Hasegawa, T. (2015). Service robot system with an informationally structured environment. *Robotics and Autonomous Systems*, 74, 148–165. doi: [10.1016/j.robot.2015.07.010](https://doi.org/10.1016/j.robot.2015.07.010)

Quigley, M., Gerkey, B. & Smart, W. D. (2015). *Programming Robots with ROS: A Practical Introduction to the Robot Operating System* (1st). O'Reilly Media, Inc.

Ramaswamy, A., Monsuez, B. & Tapus, A. (2014). SafeRobots: A model-driven Framework for developing Robotic Systems. *IEEE International Conference on Intelligent Robots and Systems*, 1517–1524. doi: [10.1109/IROS.2014.6942757](https://doi.org/10.1109/IROS.2014.6942757)

Reynolds, J. (2023). *Robot kills factory worker: Man is crushed to death when machine fails to differentiate between human and a box of vegetables.* <https://www.dailymail.co.uk/news/article-12725423>. Accessed: 2024-06-17.

Rickert, M. & Gaschler, A. (2017). Robotics Library: An Object-Oriented Approach to Robot Applications. doi: [10.1109/IROS.2017.8202232](https://doi.org/10.1109/IROS.2017.8202232)

Ritschel, N., Holmes, R., Garcia, R. & Shepherd, D. (2019). Novice-friendly multi-armed robotics programming. *Proceedings - 2019 IEEE/ACM 2nd International Workshop on Robotics Software Engineering, RoSE 2019*, 29–32. doi: [10.1109/RoSE.2019.00013](https://doi.org/10.1109/RoSE.2019.00013)

Robert, C., Sotiropoulos, T., Waeselynck, H., Guiochet, J. & Vernhes, S. (2020, 3). The virtual lands of Oz: testing an agribot in simulation. *Empirical Softw. Engg.*, p. 2025–2054. doi: [10.1007/s10664-020-09800-3](https://doi.org/10.1007/s10664-020-09800-3)

Robotiq (2022). *Start production faster | Robotiq.* <https://robotiq.com/products/2f85-140-adaptive-robot-gripper>. Accessed: 2022-08-19.

Robots, K. (2022). *Discover our Gen3 robots | Kinova.* <https://www.kinovarobotics.com/product/gen3-robots>. Accessed:2022-08-19.

Rohmer, E., Singh, S. P. N. & Freese, M. (2013). V-REP: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 1321–1326. doi: [10.1109/IROS.2013.6696520](https://doi.org/10.1109/IROS.2013.6696520)

RosTest (2025). *ROS Wiki.* <https://wiki.ros.org/rostopic>. Accessed: 2025-11-19.

Roth, H., Ruehl, M. & Dueber, F. (2003). A robot simulation system for small and medium-sized companies. *IFAC Proceedings Volumes*, 36(17), 91–96. 7th IFAC Symposium on Robot Control (SYROCO 2003), Wroclaw, Poland, 1-3 September, 2003, doi: [https://doi.org/10.1016/S1474-6670\(17\)33375-X](https://doi.org/10.1016/S1474-6670(17)33375-X)

Sánchez, P., Alonso, D., Rosique, F., Álvarez, B. & Pastor, J. A. (2011). Introducing safety requirements traceability support in model-driven development of robotic applications. *IEEE Transactions on Computers*, 60(8), 1059–1071. doi: [10.1109/TC.2010.149](https://doi.org/10.1109/TC.2010.149)

Sanfilippo, F., Hatledal, L. I., Zhang, H., Fago, M. & Pettersen, K. Y. (2015). Controlling Kuka Industrial Robots: Flexible Communication Interface JOpenShowVar. *IEEE Robotics Automation Magazine*, 22(4), 96–109. doi: [10.1109/MRA.2015.2482839](https://doi.org/10.1109/MRA.2015.2482839)

Santos, S., Pimentel, T., Rocha, F. G. & Soares, M. S. (2024). Using Behavior-Driven Development (BDD) for Non-Functional Requirements. *Software*, 3(3), 271–283. doi: [10.3390/software3030014](https://doi.org/10.3390/software3030014)

Sauppé, A. & Mutlu, B. (2014). Design patterns for exploring and prototyping human-robot interactions. *Conference on Human Factors in Computing Systems - Proceedings*, 1439–1448. doi: [10.1145/2556288.2557057](https://doi.org/10.1145/2556288.2557057)

Seshia, S. A., Hu, S., Li, W. & Zhu, Q. (2017). Design Automation of Cyber-Physical Systems: Challenges, Advances, and Opportunities. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 36(9), 1421–1434. doi: [10.1109/T-CAD.2016.2633961](https://doi.org/10.1109/T-CAD.2016.2633961)

Shepherd, D. C., Kraft, N. A. & Francis, P. (2019). Visualizing the 'hidden' variables in robot programs. *Proceedings - 2019 IEEE/ACM 2nd International Workshop on Robotics Software Engineering, RoSE 2019*, 13–16. doi: [10.1109/RoSE.2019.00007](https://doi.org/10.1109/RoSE.2019.00007)

Silva, D. C., Abreu, P. H., Reis, L. P. & Oliveira, E. (2014). Development of a flexible language for mission description for multi-robot missions. *Information Sciences*, 288(1), 27–44. doi: [10.1016/j.ins.2014.07.047](https://doi.org/10.1016/j.ins.2014.07.047)

Silva, D. C., Braga, R. A., Reis, L. P. & Oliveira, E. (2012). Designing a meta-model for a generic robotic agent system using Gaia methodology. *Information Sciences*, 195, 190–210. doi: [10.1016/j.ins.2012.01.029](https://doi.org/10.1016/j.ins.2012.01.029)

Smart, J. (2014). *BDD in Action: Behavior-driven development for the whole software lifecycle*. Shelter Island, New York, United States: Manning Publications.

Solis Pineda, C. & Wang, X. (2011, 10). A Study of the Characteristics of Behaviour Driven Development. 383 – 387. doi: [10.1109/SEAA.2011.76](https://doi.org/10.1109/SEAA.2011.76)

Sommerville, I. (2000). *Software Engineering*. Boston, Massachusetts, EUA: Addison-Wesley.

SpecFlow Project (2024). *SpecFlow Documentation*. <https://docs.specflow.org/>. Accessed: 2025-06-13.

Spong, M. W., Hutchinson, S. & Vidyasagar, M. (2006). *Robot Modeling and Control*. Hoboken, NJ: John Wiley & Sons.

Ssanderson, L. M., Collins, J. W. & McGlothlin, J. D. (1986). Robot-related fatality involving a U.S. manufacturing plant employee: Case report and recommendations. *Journal of Occupational Accidents*, 8(1), 13–23. doi: [https://doi.org/10.1016/0376-6349\(86\)90027-1](https://doi.org/10.1016/0376-6349(86)90027-1)

Sun, Y., Falco, J., Cheng, N., Choi, H. R., Engeberg, E. D., Pollard, N., Roa, M. & Xia, Z. (2018). Robotic Grasping and Manipulation Competition: Task Pool. In Y. Sun & J. Falco (dir.). *Robotic Grasping and Manipulation*, 1–18., Cham. Springer International Publishing.

Timperley, C., Afzal, A., Katz, D. S., Hernandez, J. & Goues, C. L. (2018, apr). Crashing Simulated Planes is Cheap: Can Simulation Detect Robotics Bugs Early? In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 331–342., Los Alamitos, CA, USA. IEEE Computer Society. doi: [10.1109/ICST.2018.00040](https://doi.org/10.1109/ICST.2018.00040)

Todorov, E., Erez, T. & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. doi: [10.1109/IROS.2012.6386109](https://doi.org/10.1109/IROS.2012.6386109)

Travis, G. (2015). *Robot kills worker at Volkswagen plant in Germany*. <https://www.theguardian.com/world/2015/jul/02/robot-kills-worker-at-volkswagen-plant-in-germany>. Accessed: 2020-09-13.

Trivino, G., Mengual, L. & van der Heide, A. (2009). Towards an architecture for semi-autonomous robot telecontrol systems. *Information Sciences*, 179(23), 3973–3984. doi: [10.1016/j.ins.2009.08.007](https://doi.org/10.1016/j.ins.2009.08.007)

Tsarouhas, P. H. & Furlas, G. K. (2015). Reliability and Maintainability Analysis of a Robotic System for Industrial Applications: A Case Study. *International Journal of Performability Engineering*, 11(5), 453. doi: [10.23940/ijpe.15.5.p453.mag](https://doi.org/10.23940/ijpe.15.5.p453.mag)

Untch, R. H., Offutt, A. J. & Harrold, M. J. (1993). Mutation Analysis Using Mutant Schemata. In T. J. Ostrand & E. J. Weyuker (dir.). *Proceedings of the 1993 International Symposium on Software Testing and Analysis, ISSTA 1993, Cambridge, MA, USA, June 28-30, 1993*, 139–148. ACM.

Untch, R. H., Offutt, A. J. & Harrold, M. J. (1993). Mutation Analysis Using Mutant Schemata. In T. J. Ostrand & E. J. Weyuker (dir.). *Proceedings of the 1993 International*

Symposium on Software Testing and Analysis, ISSTA 1993, Cambridge, MA, USA, June 28-30, 1993, 139–148. ACM. doi: [10.1145/154183.154265](https://doi.org/10.1145/154183.154265)

Valero-Gomez, A., Gonzalez-Gomez, J. & Treviño, R. (2013). A new paradigm for open robotics research and education with the C++ OOML. *Autonomous Robots*. doi: [10.1007/s10514-013-9324-5](https://doi.org/10.1007/s10514-013-9324-5)

Von Mayrhauser, A. & Crawford-Hines, S. (1993). Automated testing support for a robot tape library. *Proceedings - International Symposium on Software Reliability Engineering, ISSRE*, 6–14. doi: [10.1109/ISSRE.1993.624269](https://doi.org/10.1109/ISSRE.1993.624269)

Wang, S. (2020). *Reliability Engineering for Long-term Deployment of Autonomous Service Robots*. University of California, San Diego.

Weintrop, D., Afzal, A., Salac, J., Francis, P., Li, B., Shepherd, D. C. & Franklin, D. (2018). Evaluating CoBlox: A Comparative Study of Robotics Programming Environments for Adult Novices. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (p. 1–12). New York, NY, USA: Association for Computing Machinery.

Witte, T. & Tichy, M. (2018). Checking consistency of robot software architectures in ROS. *Proceedings - International Conference on Software Engineering*, 1–8. doi: [10.1145/3196558.3196559](https://doi.org/10.1145/3196558.3196559)

Wohlin, C. (2014). Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14*, New York, NY, USA. Association for Computing Machinery.

Wynne, M. & Hellesoy, A. (2012). *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf.

Yun Koo Chung & Sun-Myung Hwang (2007). Software testing for intelligent robots. In *2007 International Conference on Control, Automation and Systems*, 2344–2349., Seoul, Korea (South). IEEE. doi: [10.1109/ICCAS.2007.4406752](https://doi.org/10.1109/ICCAS.2007.4406752)

Zheng, C., An, Y., Wang, Z., Wu, H., Qin, X., Eynard, B. & Zhang, Y. (2022). Hybrid offline

programming method for robotic welding systems. *Robotics and Computer-Integrated Manufacturing*, 73, 102238. doi: <https://doi.org/10.1016/j.rcim.2021.102238>

Zheng, X., Julien, C., Kim, M. & Khurshid, S. (2014). On the State of the Art in Verification and Validation in Cyber Physical Systems TR-ARiSE-2014-001. Retrieved from <https://api.semanticscholar.org/CorpusID:14604898>

Zhong, C. & DeLoach, S. A. (2011). Runtime Models for Automatic Reorganization of Multi-robot Systems. In *Proceedings of the 6th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, SEAMS '11, 20–29., New York, NY, USA. ACM. doi: [10.1145/1988008.1988012](https://doi.org/10.1145/1988008.1988012)